

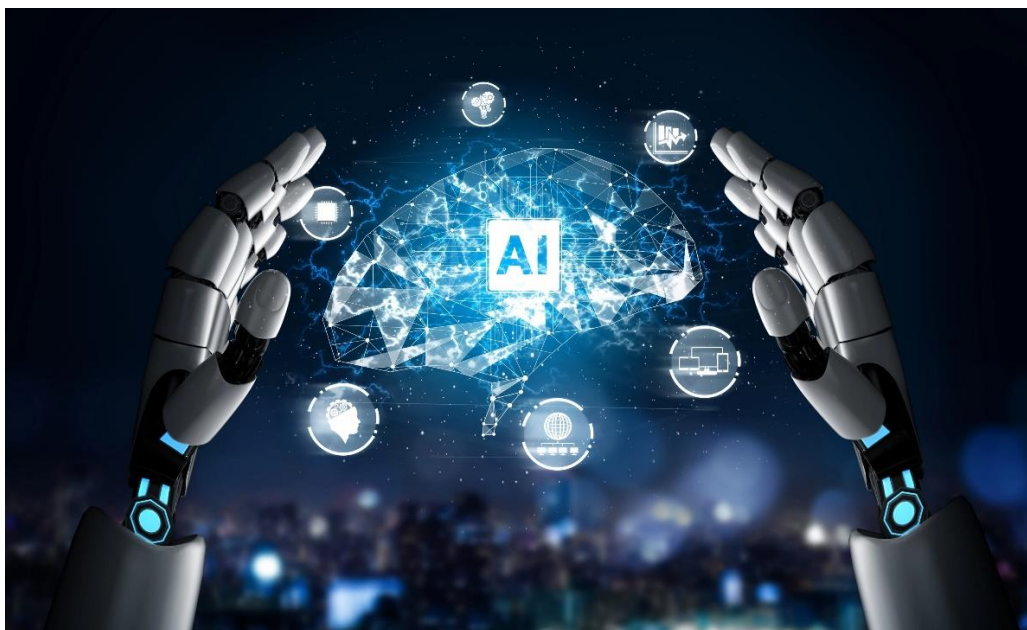
**O‘ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKASIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN’IY INTELLEKT”
KAFEDRASI**



“SUN’IY INTELLEKT ASOSLARI” FANIDAN

O‘QUV-USLUBIY MAJMUUA



Toshkent 2025

MA'RUZA MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

1-mashg'ulot. PyQt5 paketi va uning imkoniyatlari bilan tanishish.

O'quv savollari:

1. PyQt5 paketining imkoniyatlari. PyQt5 paketini o'rnatish.
2. PyQt5 paketini pip yordamida o'rnatish.
3. QtDesigner dasturini o'rnatish hamda uning imkoniyatlari bilan tanishish.

1. PyQt5 paketining imkoniyatlari. PyQt5 paketini o'rnatish

Python, boshqa dasturlash tillari kabi, grafikalar bilan ishlash qobiliyatiga ega. Quyida Pythonning eng yaxshi 5 ta grafik kutubxonasi haqida foydali ma'lumotlar keltirilgan:

1. PyQt5
2. Python Tkinter
3. PySide 2
4. Kivi
5. wxPython

Dasturlash tillaridan foydalanib turli dasturlar tuzish davomida GUI dasturlar tuzish degan tushunchaga duch kelinadi. Xo'sh GUI o'zi nima?

GUI (Graphics User Interface)- bu shakllar, hujjatlar, testlar va boshqalar kabi turli vaziyatlarda foydalanuvchilardan javob olish uchun dasturchi tomonidan ishlab chiqilgan interaktiv muhit. Bu foydalanuvchiga an'anaviy Buyruqlar qatori interfeysi (CLI) ga qaraganda chiroyli interaktiv ekranni taqdim etadi.

PyQt5 - bu Python uchun grafik foydalanuvchi interfeysi (GUI). Bu dasturchilar orasida juda mashhur va GUI kodlash yoki QT dizayner dasturi tomonidan ishlab chiqilishi mumkin. QT freymworki foydalanuvchi interfeyslarini yaratish uchun vidjetlarni bevosita sichqoncha yordamida tanlab kerakli joyga joylashtirish imkonini beruvchi vizual tizimdir.

Bu bepul va ochiq kodli bog'lovchi dasturiy ta'minot bo'lib, crossplatformali ilovalarni ishlab chiqish uchun xizmat qiladi. U Windows, Mac, Android, Linux, Arduino va Raspberry PI da qo'llanilishi mumkin.

PyQt5 ni o'rnatish uchun quyidagi buyruqdan foydalaniladi:

```
pip install pyqt5
```

Bu yerda oddiy koddan foydalanib, PyQt5 kutubxonasidan qanday foydalanishni ko'rsatib o'tilgan:

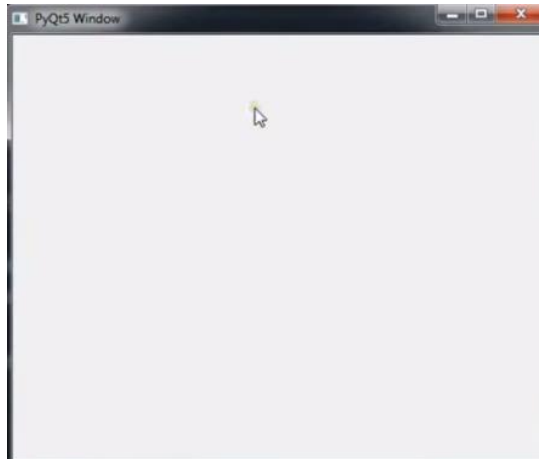
```
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys
class Window(QMainWindow):
    def __init__(self):
```

```

super().__init__()
self.setGeometry(300, 300, 600, 400)
self.setWindowTitle("PyQt5 window")
self.show()
app = QApplication(sys.argv)
window = Window()
sys.exit(app.exec_())

```

Dastur natijasi:



PyQt 5 kutubxonasi interfeysi ko'rinishi

PyQt – bu Sun'iy intellekt asoslari va Qt platformasini bir-biri bilan hamjihatlikda ishlashga imkon beruvchi mukammal bir kutubxona. U murakkab grafik interfeyslar uchun maxsus vidjetlar yaratish uchun o'rnatilgan vidjetlar va asboblarning boy tanlovini, shuningdek, ma'lumotlar bazalariga ulanish va ular bilan o'zaro ishlash uchun mustahkam SQL ma'lumotlar bazasini qo'llab-quvvatlaydi.

2. PyQt5 paketini pip yordamida o'rnatish

PyQt - Sun'iy intellekt asoslari uchun Qt grafik freymvorki uchun Python kengaytmasi sifatida amalga oshirilgan kengaytmalar (bog'lashlar) to'plami.

PyQt Britaniyaning Riverbank Computing kompaniyasi tomonidan ishlab chiqilgan python kutubxonasi. PyQt Qt tomonidan qo'llab-quvvatlanadigan barcha platformalarda ishlaydi: Linux, UNIX, macOS va Windows va shu kabi operatsion tizimlar. Hozirgi kungacha PyQt ning 3 ta versiyasi ishlab chiqilgan: PyQt4, PyQt5 va PyQt6. PyQt kutubxonasi GPL (pythonning 2 va 3 versiyalari) va tijorat litsenziyalari ostida tarqatiladi.

PyQt deyarli to'liq Qt imkoniyatlarini qamrab olgan. Bu esa 600 dan ortiq sinflar, 6000 dan ortiq funksiyalar va metodlar degani.

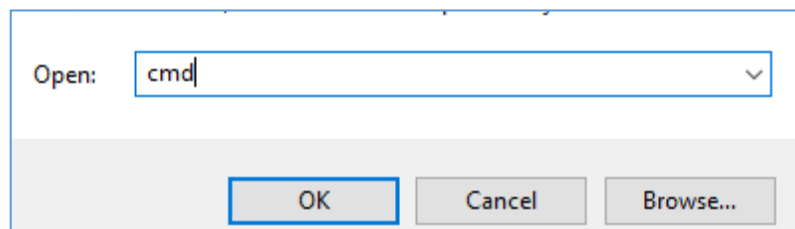
PyQt shuningdek, QtDesigner nomli GUI dizaynerini ham o'z ichiga oladi. QtDesigner dasturida yaratilgan *.ui formatli dizayn faylini Python faylga o'girib, qayta ishlash yoki to'g'ridan-to'g'ri *.ui fayli bilan *.py fayllari o'rtasida bog'lanish hosil qilish uchun **pyuic** kutubxonasidan foydalaniladi. QtDesigner dasturi PyQtni tez prototiplash uchun juda foydali vositaga aylantiradi. Bundan tashqari, Qt

Designer dasturiga Pythonda yozilgan yangi grafik boshqaruv elementlarini qo'shish mumkin.

PyQt5 paketini o'rnatish. Pythonning PyQt5 paketini Windows operatsion tizimiga o'rnatish uchun quyidagi amallarni bajarish kerak:

1-qadam: Python operatsion tizimga o'rnatilganligiga ishonch hosil qilish kerak. Buning uchun klaviaturadan **WIN+R** klaviaturalarini bir vaqtda bosiladi va ekranda **Выполнить** oynasi paydo bo'ladi. Bu oynaga **CMD** buyrug'i yoziladi va ok tugmachasini bosiladi:

Выполнить oynasi ko'rinishi



Shundan so'ng consol oynasi ochiladi va u yerga operatsion tizimda python o'rnatilganligini bilish maqsadida oddiy pythonning versiyasini bilib beruvchi buyruqni kiritamiz: **python -V**

Shundab so'ng, agar tizimda python allaqachon o'rnatilgan bo'lsa, buyruqning natijasi pythonning versiyasi bo'ladi :

```
C:\Users\PC10>python -V
Python 3.11.2
```

Python versiyasini aniqlash

Aks holda, xatolik yuzaga keladi va python o'rnatilishi kerak bo'ladi.

2-qadam. Kompyuterga PyQt5 o'rnatilmaganligiga ishonch hosil qilish.

Bunda foydalanuvchi PyQt avvaldan o'rnatilmaganligiga ishonch hosil qilish uchun quyidagi buyruqni kiritishi kerak.

Consolni ochiladi va u yerga pythonning o'rnatilgan barcha paketlari ro'yxatini chiqaruvchi buyruqni yoziladi: **pip list**.

Shunday qilib PyQt5 tizimga o'rnatilmagan yoki o'rnatilganligini bilib olinadi.

3-qadam. PyQt-ni o'rnatish. Demak operatsion tizimga python dasturi o'rnatilganligini va PyQt5 paketi o'rnatilmaganligiga ishonch hosil qilindi.

Endi PyQt5 paketini tizimga o'rnatish uchun foydalanuvchi quyida keltirilgan buyruqni konsolga kiritishi kerak: **pip install PyQt5**

```

PS C:\Users\ksahn> pip list
Package            Version
-----
charset-normalizer 2.0.9
cycler              0.11.0
fonttools           4.28.3
idna                 3.3
imutils             0.5.4
kiwisolver           1.3.2
matplotlib          3.5.1
mysql-connector-python 8.0.27
numpy                1.21.4
opencv-contrib-python 4.5.4.6
packaging            21.3
Pillow              8.4.0

```

O'rnatilgan paketlar ro'yxatini aniqlash.

Shundan so'ng hech qanday xatoliklar sodir bo'lmasa, PyQt5 tizimga muvaffaqiyatli o'rnatiladi va 2-qadamni takrorlash orqali foydalanuvchi PyQt o'rnatilganligini tekshirishi mumkin.

```

PS C:\Users\ksahn> pip install PyQt5
Collecting PyQt5
  Downloading https://files.pythonhosted.org/packages/3b/d3/76/93500af5d323160/PyQt5-5.13.0-5.13.0-cp35.cp36.cp37.cp38-none-win32.whl (49.7MB)
    100% |#####| 49.7MB 97kB/s
Requirement already satisfied: PyQt5_sip<13,>=4.19.14 in c:\python38\lib\site-packages (from PyQt5) (4.19.14)
Installing collected packages: PyQt5
Successfully installed PyQt5-5.13.0
You are using pip version 19.0.3, however version 19.1.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

```

PyQt 5 paketini o'rnatish

Windows muhitida Python uchun PyQtni muvaffaqiyatli o'rnatildi.

3. QtDesigner dasturini o'rnatish hamda uning imkoniyatlari bilan tanishish.

QtDesigner dasturi **pyqt5-tools** to'plami bilan birga o'rnatiladi. Va uni o'rnatish juda oddiy. Kompyuterning konsol oynasiga quyidagi buyruqni yoziladi va ishga tushiriladi:

```
pip install pyqt5-tools
```

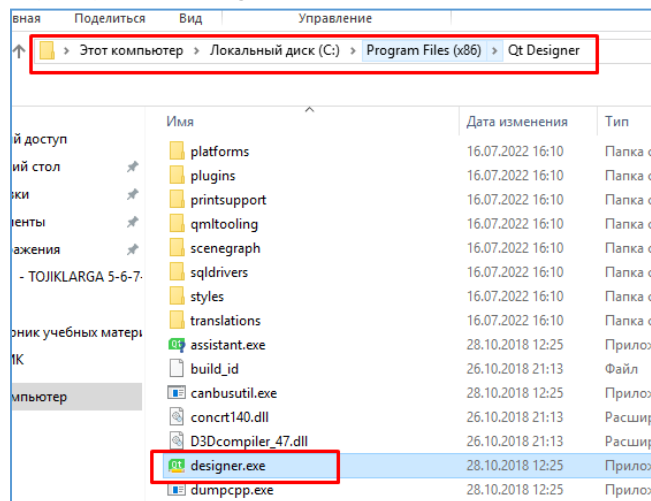
Agar yuqoridagi buyruq ishlamasa, unda quyidagi- pip buyrug'i pip3 buyrug'i bilan almashtirilgan buyruqni sinab ko'rish tavsiya etiladi:

```
pip3 install pyqt5-tools
```

QtDesignerni ishga tushiring.

Agar hammasi normada o'rnatilgan bo'lsa, unda bemaol dasturni ishga tushirib undan foydalanish mumkin bo'ladi. Buning uchun oldin QtDesigner dasturi o'rnatilgan joyni topish kerak. U odatda python o'rnatilgan kotalogda bo'ladi:

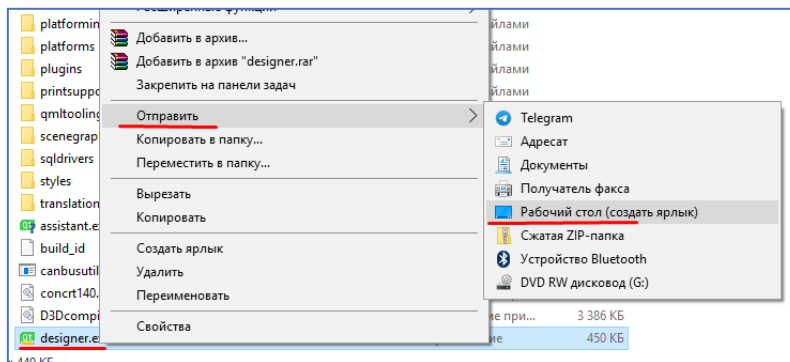
C:\Program Files (x86)\Qt Designer



Qt Designer dasturini ishga tushirish .

Odatda yorliq yaratish va uni ish stoliga joylashtirish tavsiya etiladi (38-rasm):

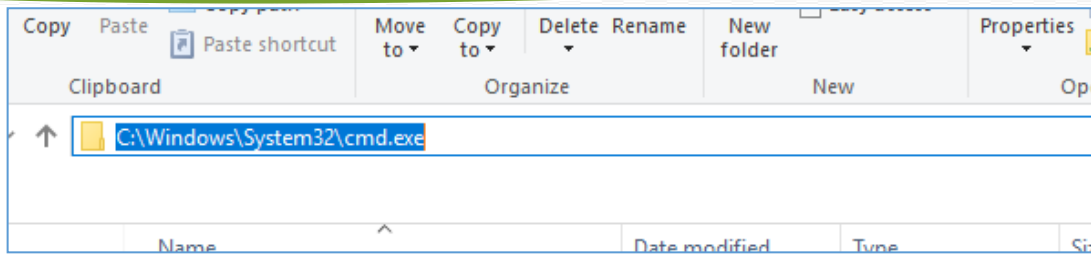
Endi dizayn dasturini ishga tushirish va python tilidagi dasturning grafik interaktiv qismini yaratishni boshlash mumkin.



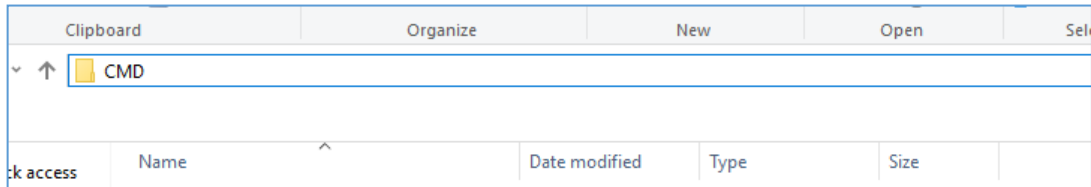
Qt Designer yorliq dasturini yaratish

QtDesigner dasturidan yaratilgan, ilova ko'rinishini o'zida saqlovchi faylni ichida python kodlarini joylashtirish uchun ushbu faylni python faylga o'girib olish talab qilinadi. Buning uchun quyidagi amallarni bajarish kerak bo'ladi:

1. QtDesigner dasturida yaratilgan **.ui** formatli faylni topiladi;
2. Ushbu fayl saqlangan kotalogdan turib konsolni ochiladi. Buning uchun kotlaogning **URL** manzili yoziladigan joyga **CMD** deb yoziladi va **enter** tugmasini bosish orqali konsolni kerakli kotologda ochish amali bajariladi (39-, 40-rasmlar):



Papkaning URL manzili yoziladigan joyi



CMD buyrug'i yozilishi kerak bo'lgan joy.

3. Ochilgan konsol oynasiga quyidagi buyruqni yoziladi. Faqat `ui_file_name` yozuvi o'rniga QtDesigner dasturining fayli nomi yozilsa, `python_file_name` o'rniga hosil bo'ladigan python fayl nomi yozilishi kerak bo'ladi:

```
pyuic5 -x "ui_file_name".ui -o "python_file_name".py
```

Shundan so'ng ushbu katalogda ko'rsatilgan nomdagi .py formatdagi fayl ham paydo bo'lganini ko'rishingiz mumkin. Va ushbu python faylning tarkibiga turli python kodlarini qo'shish, vidjetlarining qiymatlarini olish va ularni ishga tushirish uchun turli funksiyalar class, metodlar qo'shish imkoniyati mavjud.

Nazorat savollari :

1. Python PyQt 5 paketining xususiyatlarini tavsiflab bering .
2. Windows muhiti uchun PyQt5 paketi qanday o'rnatiladi?
3. QtDesigner dasturini o'rnatish qanday amalga oshiriladi?
4. QtDesigner dasturining .ui faylini .py formatli faylga o'girish uchun qanday amallarni bajarish kerak bo'ladi?
5. Nima uchun .ui formatli faylni python faylga og'irish kerak?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

2-mashg'ulot. PyQt5 kutubxonasi. QLabel vidjeti.

O'quv savollari:

1. QLabel vidjeti;
2. QLabel shrift, o'lcham va text xususiyatlari;

1. QLabel vidjeti

QLabel matn **satrlarini** ko'rsatish qobiliyatiga ega PyQt-dagi eng oddiy vidjetlardan biridir. Ushbu vidjet xohlagan vaqtda ushbu matnni olish va yangilash imkonini beruvchi ko'plab yordamchi funksiyalar va usullar bilan birga keladi.

QLabel bilan bog'liq barcha usullarning to'liq ro'yxatini ushbu sahifaning pastki qismida topish mumkin.

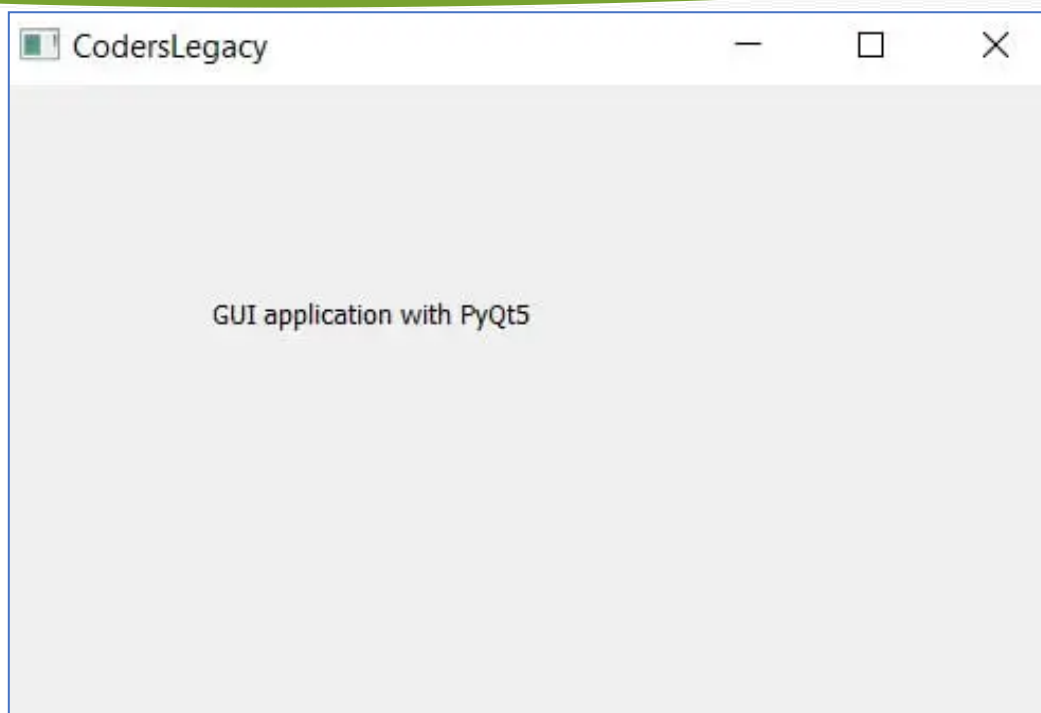
QLabel yaratish. Quyidagi misolda PyQt5 paketi yordamida oddiy QLabel yaratilishini ko'rsatib o'tilgan:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,500,300)
win.setWindowTitle("CodersLegacy")
label = QtWidgets.QLabel(win)
label.setText("GUI application with PyQt5")
label.adjustSize()
label.move(100,100)

win.show()
sys.exit(app.exec_())
```

Ushbu kodning chiqishi quyida ko'rsatilgan:



42-rasm. PyQt5 QLabel

QLabel bilan bog'liq kodni satr bo'yicha muhokama qilib chiqaylik.

```
label = QtWidgets.QLabel(win)
```

QtWidgets.QLabel() ga murojaat qilish GUI uchun foydalanish mumkin bo'lgan yorliq obyektini yaratadi. Biz yaratgan (win) oyna obyektini uning parametrlari sifatida berilishi yorliq ushbu oynaning bir qismi ekanligini anglatadi.

```
label.setText("GUI application with PyQt5")
```

Yuqoridagi oddiy kod labelga matn o'rnatadi.

```
label.adjustSize()
```

adjustSize() QLabel o'z o'lchamini avtomatik ravishda sozlashiga imkon beradi.

```
label.move(100,100)
```

Ushbu **move (100, 100)** labening dastur ekranining yuqori chap burchagidagi (100, 100) koordinatalarida paydo bo'lishini ta'minlaydi. Yodda tutingki, **ekranning** yuqori chap burchagida (0, 0) pozitsiya mavjud.

2. QLabel shrift, o'lcham va text xususiyatlari

Ushbu bo'limda hozirda QLabel da ko'rsatilgan matnni qanday yangilashni, shuningdek QLabelda ko'rsatilayotgan joriy matnni qanday qilib olish mumkinligini ko'rib chiqamiz. Bunday ishlarni qilish uchun QPushButton vidjetidan foydalanish kerak, u kerakli buyruqlarni o'z ichiga olgan funksiyalarni chaqiradi.

QLabelni yangilash uchun avvalgi **setText()** usulidan foydalaniladi va matn qiymatini olish uchun ushbu **text()** usulidan foydalaniladi:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys
```

```

def update():
    matn.setText("Yangilandi!")

def retrieve():
    print(matn.text())

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,500,300)
win.setWindowTitle("QLabel widgetining ishlashi")

matn = QtWidgets.QLabel(win)
matn.setText("PyQt5 paketida GUI dasturlari")
matn.adjustSize()
matn.move(100,100)

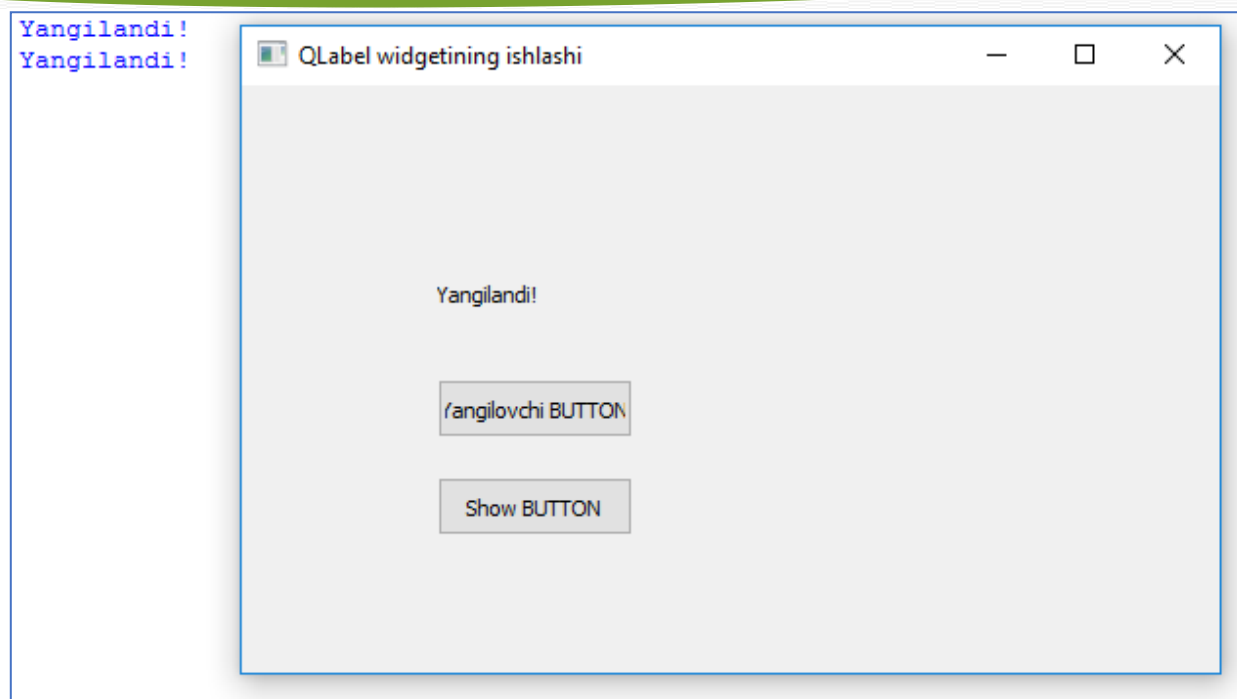
upd_btn = QtWidgets.QPushButton(win)
upd_btn.clicked.connect(update)
upd_btn.setText("Yangilovchi BUTTON")
upd_btn.move(100,150)

show_btn = QtWidgets.QPushButton(win)
show_btn.clicked.connect(retrieve)
show_btn.setText("Show BUTTON")
show_btn.move(100,200)

win.show()
sys.exit(app.exec_())

```

Yuqoridagi kodni ishga tushirgandan so‘ng ikkita QPushButton va QLabel vidgetlari orqali ishlovchi ilova paydo bo‘ladi. Unda QLabelga **“PyQt5 paketida GUI dasturlari”** yozuvi, buttonlarning biriga **“Yangilovchi BUTTON”**, ikkinchisiga esa **“Show BUTTON”** yozuvlari yozilgan bo‘ladi. Ilovaning ishlashi quyidagicha: **“Yangilovchi BUTTON”** tugmachasi bosilganda labelning matnini **“Yangilandi!”** yozuviga o‘zgartiruvchi update funksiyasini ishga tushiradi. **Show BUTTON** tugmachasi bosish orqali esa **“retrieve”** funksiyasini ishga tushirish orqali QLabel ning text xususiyatidagi yozuvni konsolga chiqariladi.



43-rasm. QLabel uchun yozilgan dasturning natijasi.

Label vidjeti uchun mavjud bo'lgan eng muhim metodlar:

Metod nomi	Tavsif
.setAlignment()	Label ichidagi matnni tekislash. U 4 xil qiymatlardan birini olishi mumkin: <i>Qt.AlignLeft</i> , <i>Qt.AlignRight</i> , <i>Qt.AlignCenter</i> va <i>Qt.AlignJustify</i> .
.setIndent()	Label matni uchun “tab”larni belgilaydi.
.setPixmap()	Labelga qo'yilgan rasmni ko'rsatish uchun ishlatiladi.
.text()	Label uchun (ko'rsatilgan) matn qiymatini qaytaradi.
.setText()	Labelda ko'rsatiladigan matnni o'rnatadi.
.selectedText()	Foydalanuvchi tomonidan tanlangan matnni qaytaradi. (Buning ishlashi uchun <i>textInteractionFlag</i> <i>TextSelectableByMouse</i> xususiyatlariga o'rnatilishi kerak).
.setWordWrap()	Label matnini avtomatik ravishda keying qatorga tushadigan xususiyatini yoqish.
.adjustSize()	Label o'lchamini ko'rsatilayotgan matnga avtomatik moslashtiradi.

Nazorat savollari:

1. QLabel vidjetidan qanday maqsadda foydalaniladi?
2. QLabel vidjetining `adjustSize()` metodining vazifasi qanday?
3. QLabel vidjetining `move()` metodining vazifasi qanday?
4. Label ichidagi matnni tekislash uchun qaysi metod ishlatiladi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

3-mashg'ulot. PyQt5 kutubxonasi. QLineEdit vidjeti.

O'quv savollari:

1. QLineEdit vidjeti;
2. setStyleSheet() metodi.

1. QLineEdit vidjeti

QLineEdit - bu matnning bir qatorini kiritish va tahrirlash imkonini beruvchi vidjet. Ushbu vidjetda Cencel va Repeat, Cut va Past hamda “Sudrab olib tashlash” funksiyalari mavjud.

```
import sys
from PyQt5.QtWidgets import (QWidget, QLabel,
                             QLineEdit, QApplication)

class Examples(QWidget):
    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):
        self.lbl = QLabel(self)
        qlineE = QLineEdit(self)

        qlineE.move(100, 100)
        self.lbl.move(100, 40)

        qlineE.textChanged[str].connect(self.changed)
        self.setGeometry(300, 300, 280, 170)
        self.setWindowTitle('QlineEdit Example')
        self.show()

    def changed(self, text):
        self.lbl.setText(Label text)
        self.lbl.adjustSize()

if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Examples()
    sys.exit(app.exec_())
```

Ushbu misolda QLineEdit vidjeti va QLabel ko'rsatilgan. Tahrirlash paneliga kiritgan matnimiz darhol teg vidjetida ko'rsatiladi.

```
qlineE = QLineEdit(self)
```

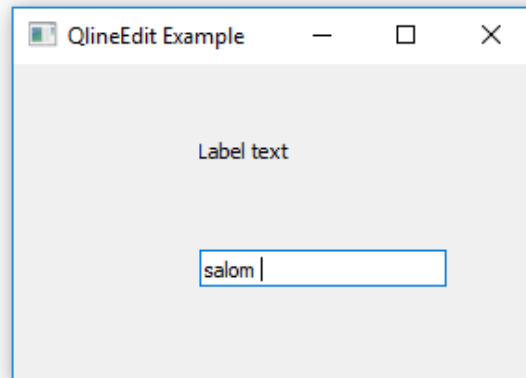
QLineEdit vidjetining `qline` obykti yaratildi.

```
qlineE.textChanged[str].connect(self.changed)
```

Agar QLineEdit vidjetidagi matn o'zgarsa, `changed()` funksiyasi ishga tushadi.

```
def changed(self, text):
    self.lbl.setText(text)
    self.lbl.adjustSize()
```

`changed` funksiyasining ichida kiritilgan matnni teg vidjetiga o'rnatish buyrug'i joylashtirilgan. Yorliq o'lchamini matn uzunligiga mos ravishda o'zgartirish uchun `adjustSize()` metodidan foydalanilgan .



Yuqoridagi dastur ko'rinishi

PyQt QLineEdit

PyQt QLineEdit bir qatorli matnli vidjet yaratish imkonini beradi. Odatda, siz QLineEditma'lumotlarni kiritish shaklida foydalanasiz .

Amalda siz ko'pincha QLineEditvidjetni vidjet bilan ishlatasiz QLabel.

Vidjet yaratish uchun QLineEditquyidagi amallarni bajaring.

Birinchidan, moduldan import QLineEditqiling PyQt6.QtWidgets:

from PyQt6.QtWidgets import QLineEditKod tili: Python (python)

Ikkinchidan, QLineEditfoydalanadigan yangi obyekt yarating:

- Hech qanday dalil yo'q.
- Faqat ota-ona vidjeti bilan.
- Yoki birinchi argument sifatida standart satr qiymati bilan.

Masalan:

line_edit = QLineEdit('Default Value', self)Kod tili: Python (python)

Bundan tashqari, siz quyidagi qo'shimcha xususiyatlardan foydalanishingiz mumkin:

Metod	Turi	Ta'ri
matn	ip	Satrn tahrirlash mazmuni
readOnly	Mantiqiy	To'g'ri yoki noto'g'ri. Agar rost bo'lsa, qatorni tahrir qilib bo'lmaydi
clearButtonEnabled	Mantiqiy	Aniq tugmani qo'shish to'g'ri

Metod	Turi	Ta'rifi
placeholderText	ip	Satrni tahrirlash bo'sh bo'lganda paydo bo'ladigan matn
maxLength	butun son	Maksimal kiritilishi mumkin bo'lgan belgilar sonini belgilang
echoMode	QLineEdit.EchoMode	Matnni ko'rsatish usulini o'zgartiring, masalan, parol

PyQt QLineEdit vidjetiga misollar

Keling, vidjetdan foydalanishga misollar keltiraylik QLineEdit.

1) Oddiy PyQt QLineEdit misoli

Quyidagi dastur vidjetni qanday yaratishni ko'rsatadi QLineEdit:

```
import sys
from PyQt6.QtWidgets import (
    QApplication,
    QWidget,
    QLineEdit,
    QVBoxLayout
)

class MainWindow(QWidget):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        self.setWindowTitle('PyQt QLineEdit Widget')
        self.setGeometry(100, 100, 320, 210)

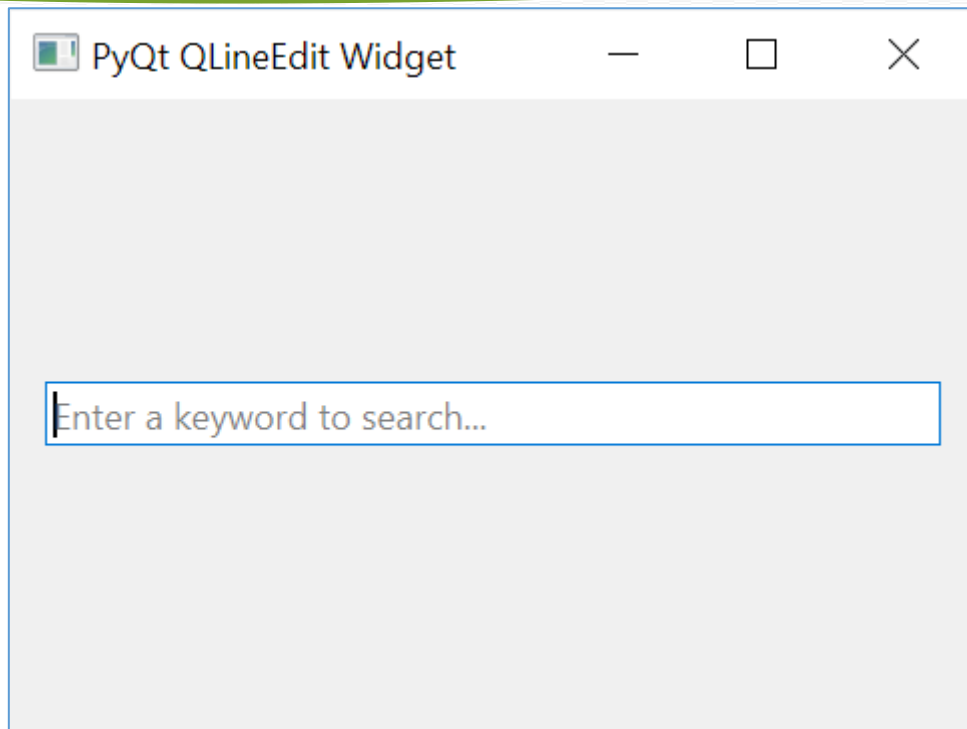
        search_box = QLineEdit(
            self,
            placeholderText='Enter a keyword to search...',
            clearButtonEnabled=True
        )

        # place the widget on the window
        layout = QVBoxLayout()
        layout.addWidget(search_box)
        self.setLayout(layout)

        # show the window
        self.show()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = MainWindow()
    sys.exit(app.exec())
```

Chiqish:



2) Parol yozuvini yaratish uchun PyQt QLineEdit dan foydalanish
Quyidagi dastur QLineEditparol kiritish sifatida yangi vidjet yaratadi:

```
import sys
from PyQt6.QtWidgets import (
    QApplication,
    QWidget,
    QLineEdit,
    QVBoxLayout
)

class MainWindow(QWidget):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        self.setWindowTitle('PyQt QLineEdit Widget')
        self.setGeometry(100, 100, 320, 210)

        password = QLineEdit(self, echoMode=QLineEdit.EchoMode.Password)

        # place the widget on the window
        layout = QVBoxLayout()
        layout.addWidget(password)
        self.setLayout(layout)

        # show the window
        self.show()

if __name__ == '__main__':
    app = QApplication(sys.argv)
```



```

window = MainWindow()
sys.exit(app.exec())

```

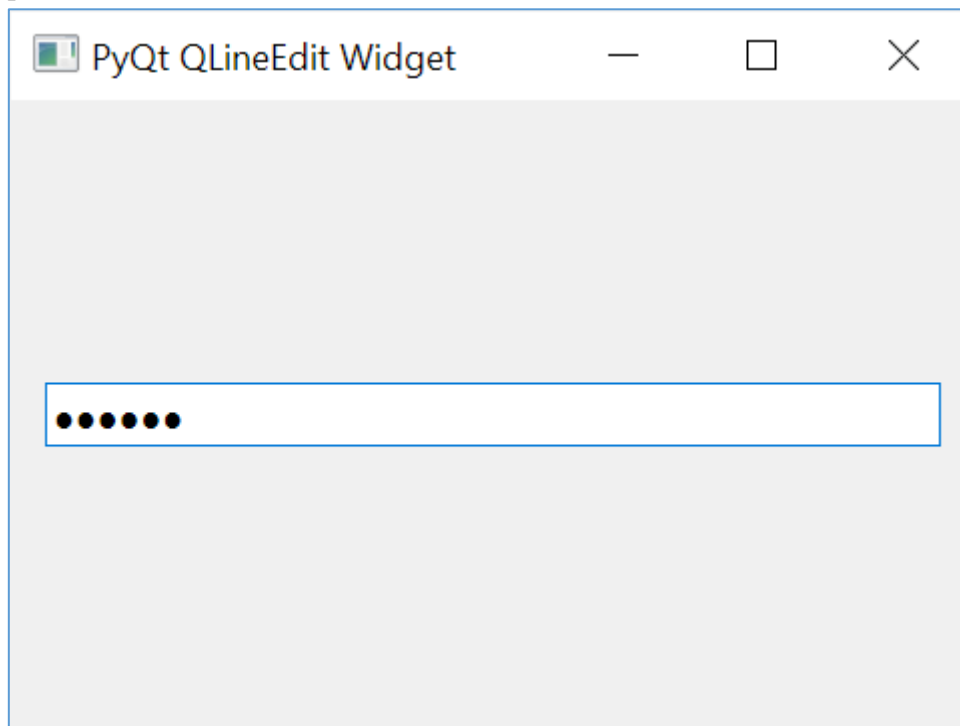
QLineEditVidjetni parol kiritishga aylantirish uchun siz echoModeni o'rnatasiz QLineEdit. EchoMode.Password:

```

password = QLineEdit(self, echoMode=QLineEdit.EchoMode.Password)

```

Chiqish:



3) PyQt QLineEdit-dan avtomatik to'ldirish xususiyati bilan foydalanish

Avtomatik to'ldirish xususiyati bilan yozuv yaratish uchun siz quyidagi amallarni bajaring:

Birinchidan, modulni import QCompleterqiling PyQt6.QtWidgets.

Ikkinchidan, QCompleteravtomatik to'ldirish funksiyasi uchun ishlatiladigan satrlar ro'yxati bilan vidjet yarating:

```

completer = QCompleter(word_list)

```

Uchinchidan, a yarating va to'ldiruvchi obyekt bilan QLineEdituning usulini chaqiring :setCompleter()

```

line_edit = QLineEdit(self)
line_edit.setCompleter(completer)

```

Masalan, quyidagi dastur QLineEditavtomatik to'ldirish xususiyatiga ega vidjetni ko'rsatadi:

```

import sys
from PyQt6.QtWidgets import (
    QApplication,
    QWidget,
    QLineEdit,
    QVBoxLayout,

```

```

        QCompleter
    )

    class MainWindow(QWidget):
        def __init__(self, *args, **kwargs):
            super().__init__(*args, **kwargs)

            self.setWindowTitle('PyQt QLineEdit Widget')
            self.setGeometry(100, 100, 320, 210)

            common_fruits = QCompleter([
                'Apple',
                'Apricot',
                'Banana',
                'Carambola',
                'Olive',
                'Oranges',
                'Papaya',
                'Peach',
                'Pineapple',
                'Pomegranate',
                'Rambutan',
                'Ramphal',
                'Raspberries',
                'Rose apple',
                'Starfruit',
                'Strawberries',
                'Water apple',
            ])
            fruit = QLineEdit(self)
            fruit.setCompleter(common_fruits)

            # place the widget on the window
            layout = QVBoxLayout()
            layout.addWidget(fruit)
            self.setLayout(layout)

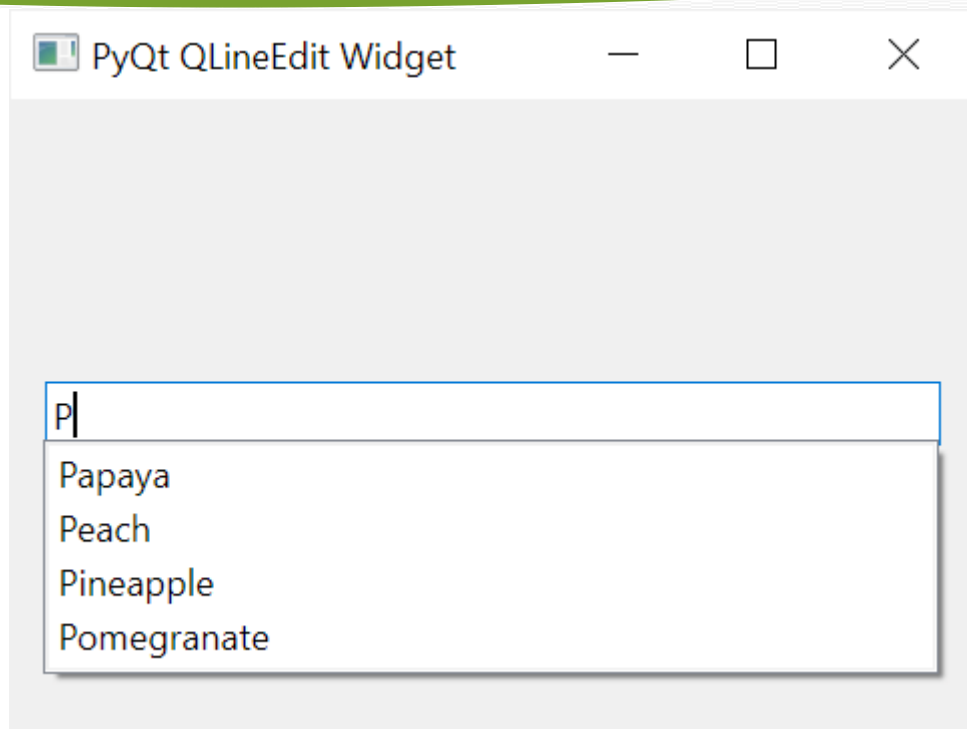
            # show the window
            self.show()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = MainWindow()
    sys.exit(app.exec())

```

Kod tili: Python (python)

Chiqish:



- QLineEditBir qatorli kirish vidjetini yaratish uchun foydalaning .
- echoModeMatnni ko'rsatish usulini o'zgartirish uchun xususiyatdan foydalaning .
- QLineEditAvtomatik to'ldirish funksiyasini qo'llab-quvvatlash uchun vidjetdan QCompleter vidjeti bilan foydalaning .

Nazorat savollari:

1. PyQt da yorliq yaratish uchun qaysi vidgetdan foydalaniladi?
2. QLineEdit vidjetining vazifasi qanday?
3. QLineEdit vidjetining xususiyatlari qanday?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

4-mashg'ulot. PyQt5 kutubxonasidan foydalanib turli qiyinchilikdagi masalalarning dasturlarini tuzish.

O'quv savollari:

1. Turli vidgetlardan foydalanib oddiy arifmetik dasturlar tuzish;
2. Turli vidgetlardan foydalanib ro'yxatlarga doir dasturlar tuzish.

Turli vidgetlardan foydalanib oddiy arifmetik dasturlar tuzish;

Har bir obyektga shuningdek, bosilgan, ochiladigan va kengaytiriladigan Pos 3 ta maxsus signal mavjud bo'lib, biz ularni maxsus slotlarga ulaymiz. Nihoyat, biz oyna hajmini yangi tarkibga moslashtirish uchun o'lchamini o'zgartirishni chaqiramiz. Bu, aslida, faqat deraza qisqarganida kerak bo'ladi - Qt uni avtomatik ravishda o'stiradi.

```
def init_map(self):
    # Add positions to the map
    for x in range(0, self.b_size):
        for y in range(0, self.b_size):
            w = Pos(x,y)
            self.grid.addWidget(w, y, x)
            # Connect signal to handle expansion.
            w.clicked.connect(self.trigger_start)
            w.revealed.connect(self.on_reveal)
            w.expandable.connect(self.expand_reveal)

    QTimer.singleShot(0, lambda: self.resize(1,1)) # <1>
```

Taymer `singleShot` biz voqea sikliga qaytganimizdan so'ng va Qt yangi tarkibdan xabardor bo'lganimizdan keyin o'lchamni o'zgartirishni ta'minlash uchun talab qilinadi.

Endi bizda pozitsion plitka obyektlari to'plami mavjud, biz o'yin taxtasining dastlabki sharoitlarini yaratishni boshlashimiz mumkin. Bu bir qator funksiyalarga bo'lingan. Biz ularni nomlaymiz `_reset` (etakchi pastki chiziq tashqi foydalanish uchun mo'ljallanmagan shaxsiy funksiyani ko'rsatadigan konventsiyadir). Asosiy funksiya `reset_map` uni sozlash uchun navbatma-navbat bu funksiyalarni chaqiradi.

Jarayon quyidagicha -

1. Maydondan barcha minalarni olib tashlang (va ma'lumotlarni qayta o'rnatish).
2. Maydonga yangi minalar qo'shing.
3. Har bir pozitsiyaga ulashgan minalar sonini hisoblang.
4. Boshlang'ich belgisini (raketa) qo'shing va dastlabki tadqiqotni ishga tushiring.
5. Taymerni qayta o'rnatish.

Buni amalga oshirish uchun kod:

```
def reset_map(self):
```

```

self._reset_position_data()
self._reset_add_mines()
self._reset_calculate_adjacency()
self._reset_add_starting_marker()
self.update_timer()

```

1 dan 5 gacha bo'lgan alohida qadamlar quyida har bir qadam uchun kod bilan batafsil tavsiflanadi.

Birinchi qadam xaritada har bir pozitsiya uchun ma'lumotlarni qayta o'rnatishdir. Biz doskadagi har bir pozitsiyani takrorlaymiz, `.reset()` har bir nuqtada vidjetni chaqiramiz. Funksiya kodi `.reset()` bizning maxsus sinfimizda belgilangan `Pos`, biz keyinroq batafsil ko'rib chiqamiz. Hozircha u minalarni, bayroqlarni tozalab, o'z o'rnini oshkor etilmagan holatga keltirishini bilish kifoya.

```

def _reset_position_data(self):
    # Clear all mine positions
    for x in range(0, self.b_size):
        for y in range(0, self.b_size):
            w = self.grid.itemAtPosition(y, x).widget()
            w.reset()

```

Endi barcha pozitsiyalar bo'sh, biz xaritaga minalar qo'shish jarayonini boshlashimiz mumkin. Minalar maksimal soni `n_mines` yuqorida tavsiflangan daraja sozlamalari bilan belgilanadi.

```

def _reset_add_mines(self):
    # Add mine positions
    positions = []
    while len(positions) < self.n_mines:
        x, y = random.randint(0, self.b_size-1), random.randint(0,
self.b_size-1)
        if (x, y) not in positions:
            w = self.grid.itemAtPosition(y, x).widget()
            w.is_mine = True
            positions.append((x, y))

    # Calculate end-game condition
    self.end_game_n = (self.b_size * self.b_size) - (self.n_mines + 1)
    return positions

```

Minalar joyida bo'lsa, endi biz har bir pozitsiya uchun "qo'shni" raqamini hisoblashimiz mumkin - berilgan nuqta atrofida 3x3 o'lchamdagi to'rdan foydalanib, yaqin atrofdagi minalar sonini. Maxsus funksiya shunchaki ma'lum va joylashuv `get_surrounding` atrofidagi pozitsiyalarni qaytaradi. Biz kon va do'kon bo'lgan bularning sonini hisoblaymiz. `xyis_mine == True`

Bu yerda qo'shni hisoblarni oldindan hisoblash keyinchalik ochish mantiqini soddalashtirishga yordam beradi. Ammo bu shuni anglatadiki, biz foydalanuvchiga

o'zining dastlabki harakatini tanlashiga ruxsat bera olmaymiz - biz buni "raketa atrofidagi dastlabki tadqiqot" deb tushuntirib, uni butunlay oqilona qilishimiz mumkin.

Hisobni kechiktirish orqali buni o'zingiz hal qilishga harakat qiling!

```
def _reset_calculate_adjacency(self):

    def get_adjacency_n(x, y):
        positions = self.get_surrounding(x, y)
        return sum(1 for w in positions if w.is_mine)

    # Add adjacencies to the positions
    for x in range(0, self.b_size):
        for y in range(0, self.b_size):
            w = self.grid.itemAtPosition(y, x).widget()
            w.adjacent_n = get_adjacency_n(x, y)
```

Birinchi harakat har doim haqiqiy bo'lishini ta'minlash uchun boshlang'ich belgisi ishlatiladi. Bu biz mina bo'lmagan pozitsiyani topmagunimizcha, tasodifiy pozitsiyalarni samarali sinab ko'rish orqali tarmoq bo'ylab *qo'pol kuch* qidirish sifatida amalga oshiriladi. Buning uchun qancha urinishlar ketishini bilmasligimiz sababli, uni uzluksiz halqaga o'rashimiz kerak.

Bu joy topilgach, biz uni boshlang'ich joy sifatida belgilaymiz va keyin atrofda barcha pozitsiyalarni o'rganishni ishga tushiramiz. Biz sikldan chiqib, tayyor holatni tiklaymiz.

```
def _reset_add_starting_marker(self):
    # Place starting marker.

    # Set initial status (needed for .click to function)
    self.update_status(STATUS_READY)

    while True:
        x, y = random.randint(0, self.b_size - 1), random.randint(0, self.b_size
- 1)

        w = self.grid.itemAtPosition(y, x).widget()
        # We don't want to start on a mine.
        if not w.is_mine:
            w.is_start = True
            w.is_revealed = True
            w.update()

        # Reveal all positions around this, if they are not mines either.
        for w in self.get_surrounding(x, y):
            if not w.is_mine:
                w.click()
            break

        # Reset status to ready following initial clicks.
        self.update_status(STATUS_READY)
```

Turli vidgetlardan foydalanib ro'yxatlarga doir dasturlar tuzish

O'yin strukturaviy o'yin bo'lib, individual plitka pozitsiyalari o'z holati haqida ma'lumotga ega bo'ladi. Bu shuni anglatadiki, **Pos**plitkalar o'zlarining o'yin mantiqlarini boshqarishga qodir.

Sinf nisbatan murakkab bo'lganligi sababli **Pos**, u bu erda bir necha qismlarga bo'linadi va ular o'z navbatida muhokama qilinadi. Dastlabki o'rnatish `__init__` bloki oddiy bo'lib, xva ypozitsiyasini qabul qiladi va uni obyektida saqlaydi. **Pos**pozitsiyalar yaratilgandan keyin hech qachon o'zgarmaydi.

O'rnatishni yakunlash uchun `.reset()` barcha obyekt atributlarini sukut bo'yicha, nol qiymatlariga qaytaradigan funksiya chaqiriladi. Bu konni *boshlang'ich pozitsiyasi emas , mina emas , oshkor qilinmagan va belgilanmagan* deb belgilaydi . Biz qo'shni hisobni ham tiklaymiz.

```
class Pos(QWidget):

    expandable = pyqtSignal(int,int)
    revealed = pyqtSignal(object)
    clicked = pyqtSignal()

    def __init__(self, x, y, *args, **kwargs):
        super(Pos, self).__init__(*args, **kwargs)

        self.setFixedSize(QSize(20, 20))
        self.x = x
        self.y = y
        self.reset()

    def reset(self):
        self.is_start = False
        self.is_mine = False
        self.adjacent_n = 0
        self.is_revealed = False
        self.is_flagged = False

        self.update()
```

O'yin o'yin maydonidagi plitkalar bilan sichqonchani o'zaro ta'siriga qaratilgan, shuning uchun sichqonchani bosishlarini aniqlash va ularga munosabat bildirish asosiy hisoblanadi. Qt da biz sichqonchani bosishlarini aniqlash orqali ushlaymiz `mouseReleaseEvent`. Buni bizning shaxsiy vidjetimiz uchun qilish uchun **Pos** biz sinfda ishlov beruvchini aniqlaymiz. Bu `QMouseEvent` sodir bo'lgan voqealarni o'z ichiga olgan ma'lumot bilan qabul qilinadi. Bunday holda, biz faqat sichqonchani chap tugmasi yoki o'ng tugmasi bilan bo'shatilganligi bilan qiziqamiz.

Sichqonchani chap tugmachasini bosish uchun biz kafelning bayroqlanganligini yoki allaqachon ochilganligini tekshiramiz. Agar shunday bo'lsa, biz chertishni e'tiborsiz qoldiramiz - bayroqchali plitalarni "xavfsiz" qilib, tasodifan

bosish mumkin emas. Agar kafel belgilanmagan bo'lsa, biz shunchaki `.click()` usulni ishga tushiramiz (keyinroq ko'ring).

Sichqonchaning o'ng tugmasi bilan ko'rsatilmagan plitkalarni bosish uchun biz `.toggle_flag()` bayroqni yoqish va o'chirish usulini chaqiramiz.

```
def mousePressEvent(self, e):

    if(e.button()==Qt.RightButton and not self.is_revealed):
        self.toggle_flag()

    elif (e.button() == Qt.LeftButton):
        # Block clicking on flagged mines.
        if not self.is_flagged and not self.is_revealed:
            self.click()
```

Ishlovchi tomonidan chaqiriladigan usullar `mousePressEvent` quyida tavsiflanadi.

Ishlovchi `.toggle_flag` shunchaki `.is_flagged` o'ziga teskarisini o'rnatadi (`True` bo'ladi `False`, `False` bo'ladi `True`) uni yoqish va o'chirish effektiga ega. `.update()` E'tibor bering, biz holatni o'zgartirib, qayta chizishga majburlash uchun qo'ng'iroq qilishimiz kerak. `.clicked` Shuningdek, biz taymerni ishga tushirish uchun ishlatiladigan maxsus signalimizni chiqaramiz - chunki bayroqni qo'yish kvadratni ochish emas, balki boshlang'ich sifatida ham hisoblanishi kerak.

Usul `.click()` sichqonchaning chap tugmachasini bosadi va o'z navbatida kvadratni ochishni boshlaydi. Agar bunga qo'shni minalar soni `Pos` nolga teng bo'lsa, biz `.expandable` o'rganilayotgan hududni avtomatik ravishda kengaytirish jarayonini boshlash uchun signalni ishga tushiramiz (keyinroq ko'ring). Nihoyat, biz yana `.clicked` o'yin boshlanishini bildirish uchun chiqaramiz.

Nihoyat, `.reveal()` usul kafel allaqachon aniqlanganligini tekshiradi va agar `.is_revealed` bo'lmasa `True`. `.update()` Vidjetni qayta bo'yashni boshlash uchun yana qo'ng'iroq qilamiz.

Signalning ixtiyoriy emissiyasi `.revealed` faqat o'yin oxiri to'liq xaritasini ochish uchun ishlatiladi. Har bir ochilish qaysi plitkalar aniqlanishi mumkinligini aniqlash uchun qo'shimcha qidiruvni ishga tushirganligi sababli, butun xaritani ochish juda ko'p ortiqcha qo'ng'iroqlarni keltirib chiqaradi. Bu erda signalni bostirish orqali biz bundan qochamiz.

```
def toggle_flag(self):
    self.is_flagged = not self.is_flagged
    self.update()

    self.clicked.emit()

def click(self):
    self.reveal()
    if self.adjacent_n == 0:
```



```

        self.expandable.emit(self.x, self.y)

    self.clicked.emit()

    def reveal(self, emit=True):
        if not self.is_revealed:
            self.is_revealed = True
            self.update()

            if emit:
                self.revealed.emit(self)

```

Va nihoyat, biz vidjetimiz uchun joriy joylashuv holatini ko'rsatishni boshqarish uchun maxsus `paintEvent` usulni aniqlaymiz. `Pos[bobda]` tasvirlanganidek, vidjet tuvalini maxsus bo'yashni amalga oshirish uchun biz chizishimiz kerak bo'lgan chegaralarni - bu holda vidjetning tashqi chegarasini ko'rsatadigan belgini `QPainter` olamiz `.event.rect()`

Ochilgan plitkalar, kafelning *boshlang'ich pozitsiyasi*, *bomba* yoki *bo'sh joy* bo'lishiga qarab turlicha chiziladi. Birinchi ikkitasi mos ravishda raketa va bomba piktogrammalari bilan ifodalanadi. `QRect` Ular yordamida plitka ichiga tortiladi `.drawPixmap`. `QImage` E'tibor bering, biz o'tish orqali o'tish orqali doimiylarni piksel xaritalariga aylantirishimiz kerak `QPixmap`.

Siz shunday deb o'ylashingiz mumkin: "Nima uchun ularni faqat obyektlar sifatida saqlamaslik kerak, chunki biz aynan shu narsadan foydalanamiz? Afsuski, siz o'zingiz ishga tushmasdan oldin obyektlar `QPixmap` yarata olmaysiz. `QPixmapQApplication`

Bo'sh pozitsiyalar uchun (raketalar emas, bombalar emas) biz ixtiyoriy ravishda qo'shni raqamni ko'rsatamiz, agar u noldan katta bo'lsa. Matnni matnga chizish uchun biz, hizalama bayroqlari va qator sifatida chizish uchun raqamni o'tkazishdan `QPainter` foydalanamiz. Foydalanish qulayligi uchun biz har bir raqam uchun standart rangni belgilab oldik (da saqlangan) `.drawText()` `QRectNUM_COLORS`

Ochilmagan plitkalar uchun biz to'rtburchakni ochiq kul rang bilan to'ldirib, plitka chizamiz va quyuqroq kul rangning 1 pikselli chegarasini chizamiz. Agar o'rnatilgan bo'lsa, biz va plitkadan `.is_flagged` foydalanib, kafelning yuqori qismiga bayroq belgisini ham chizamiz `.drawPixmapQRect`

```

def paintEvent(self, event):
    p = QPainter(self)
    p.setRenderHint(QPainter.Antialiasing)

    r = event.rect()

    if self.is_revealed:
        if self.is_start:

```

```

        p.drawPixmap(r, QPixmap(IMG_START))

    elif self.is_mine:
        p.drawPixmap(r, QPixmap(IMG_BOMB))

    elif self.adjacent_n > 0:
        pen = QPen(NUM_COLORS[self.adjacent_n])
        p.setPen(pen)
        f = p.font()
        f.setBold(True)
        p.setFont(f)
        p.drawText(r, Qt.AlignHCenter | Qt.AlignVCenter,
str(self.adjacent_n))

    else:
        p.fillRect(r, QBrush(Qt.lightGray))
        pen = QPen(Qt.gray)
        pen.setWidth(1)
        p.setPen(pen)
        p.drawRect(r)

    if self.is_flagged:
        p.drawPixmap(r, QPixmap(IMG_FLAG))

```

Mexanika

Biz odatda ma'lum bir nuqta atrofidagi barcha plitkalarni olishimiz kerak, shuning uchun bizda bu maqsad uchun maxsus funksiya mavjud. U nuqta atrofida 3x3 o'lchamdagi to'r bo'ylab oddiy takrorlanadi va biz to'r chetlarida chegaradan chiqmasligimizni tekshirib ko'ring ($0 \leq x \leq \text{self.b_size}$). Qaytarilgan ro'yxatda **Poshar** bir atrofdagi joydan vidjet mavjud.

```

def get_surrounding(self, x, y):
    positions = []

    for xi in range(max(0, x - 1), min(x + 2, self.b_size)):
        for yi in range(max(0, y - 1), min(y + 2, self.b_size)):
            if not (xi == x and yi == y):
                positions.append(self.grid.itemAtPosition(yi, xi).widget())

    return positions

```

Ushbu **expand_reveal** nolga qo'shni minalar bo'lgan kafelni bosishga javoban ishga tushiriladi. Bunday holda, biz bosish atrofidagi maydonni nolga qo'shni minalar bo'lgan har qanday bo'shliqqa kengaytirishni xohlaymiz, shuningdek, bu kengaytirilgan maydonning chegarasi atrofidagi har qanday kvadratlarni (bu minalar emas) aniqlamoqchimiz.

Biz keyingi iteratsiyada tekshiriladigan pozitsiyalarni o'z ichiga olgan ro'yxat, ko'rsatiladigan plitka vidjetlarini o'z ichiga olgan **to_expand** ro'yxat

va sikldan qachon chiqishni aniqlash uchun bayroqdan boshlaymiz. Birinchi marta yangi vidjetlar qo'shilmasa, sikl to'xtaydi `.to_revealany_addedto_reveal`

Loop ichida biz `any_added` ni `False` ga qayta o'rnatamiz, ro'yxatni bo'shatamiz va takrorlash uchun `to_expand` ni vaqtincha `1` o'zgaruvchisida saqlab qolamiz. Har bir joy uchun biz 8 ta qo'shni vidjetni olamiz. Agar ushbu vidjetlardan biri **mina bo'lmasa** va `to_reveal` ro'yxatida bo'lmasa, uni `to_reveal` ga qo'shamiz. Bu kengaytirilgan maydonning barcha qirralari oshkor bo'lishini ta'minlaydi. Agar pozitsiyada qo'shni minalar soni `0` bo'lsa, keyingi iteratsiyada tekshiriladigan koordinatalarni `to_expand` ga qo'shamiz.

`to_reveal` ga faqat oshkor qilinmagan plitkalarni qo'shish va `to_expand` orqali faqat hali ko'rilmagan plitkalarni kengaytirish natijasida, biz bir xil kafelga bir necha marta tashrif buyurmasligimizga ishonch hosil qilamiz.

```
def expand_reveal(self, x, y):
    """
    Iteratsiyani boshlang'ich nuqtadan tashqariga kengaytirib,
    yangi joylarni navbatga qo'shamiz. Bu bir necha callback'lara
    tayanmasdan, barchasini bir martada kengaytirish imkonini beradi.
    """
    to_expand = [(x, y)]
    to_reveal = []
    any_added = True

    while any_added:
        any_added = False
        # avvalgi to_expand'ni l ga saqlab, to_expand'ni bo'shatamiz
        to_expand, l = [], to_expand

        for x, y in l:
            positions = self.get_surrounding(x, y)
            for w in positions:
                if not w.is_mine and w not in to_reveal:
                    to_reveal.append(w)
                    if w.adjacent_n == 0:
                        to_expand.append((w.x, w.y))
                        any_added = True

        # Topilgan barcha pozitsiyalarni oshkor qilamiz.
        for w in to_reveal:
            w.reveal()
```

2. Dasturni testlash

O'yin yakun holati plitka ustiga bosilgandan so'ng oshkor qilish jarayonida aniqlanadi. Ikkita mumkin bo'lgan natija bor:

1. Plitka mina bo'lsa — o'yin tugaydi.
2. Plitka mina bo'lmasa — `self.end_game_n` (bo'sh plitkalar soni) kamaytiriladi.

Bu qiymat 0 ga yetguncha davom etadi; 0 bo'lganda g'alaba jarayoni (game_won) boshlanadi. Muvaffaqiyat ham, muvaffaqiyatsizlik ham xaritani oshkor qilish (reveal_map) va tegishli holatni o'rnatish orqali yakunlanadi.

```
def on_reveal(self, w):
    if w.is_mine:
        self.game_over()
    else:
        # qolgan bo'sh joylarni kamaytiramiz
        self.end_game_n -= 1
        if self.end_game_n == 0:
            self.game_won()

def game_over(self):
    self.reveal_map()
    self.update_status(STATUS_FAILED)

def game_won(self):
    self.reveal_map()
    self.update_status(STATUS_SUCCESS)
```

Nazorat savollari:

1. PyQt5 kutubxonasi nima va u qaysi maqsadlarda qo'llaniladi?
2. PyQt5 kutubxonasining asosiy komponentlari (QWidget, QMainWindow, QApplication, va boshqalar) haqida gapirib bering.
3. PyQt5 dasturida asosiy oynani (Main Window) yaratish jarayonini tushuntiring.
4. QPushButton vidjeti qanday yaratiladi va unga bosilganda bajariladigan amal qanday ulanadi?
5. PyQt5 da QLineEdit va QTextEdit vidjetlarining farqi nimada?
6. QLabel yordamida dastur oynasiga matn yoki rasm qanday joylashtiriladi?
7. Signal va Slot tushunchasi PyQt5 da qanday ishlaydi? Misol keltiring.
8. PyQt5 da layoutlar (QHBoxLayout, QVBoxLayout, QGridLayout) nima vazifa bajaradi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

5-mashg'ulot. PyQt5 modal dialog. QMessageBox vidjeti bilan ishlash.

O'quv savollari:

1. QMessageBox vidjetining vazifasi.
2. QMessageBox vidjetining asosiy xususiyatlari.
3. Statik funksiyalari.
4. Piktogramma va QPixmap xususiyatlari.

QMessageBox vidjetining vazifasi

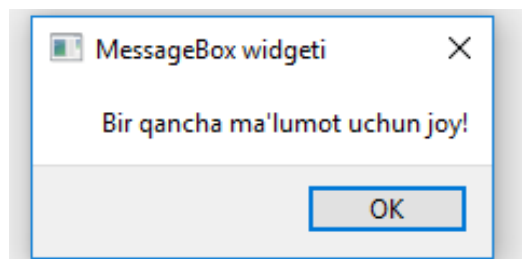
QMessageBox - bu dialoglar yaratish uchun ishlatiladigan foydali PyQt5 vidjeti. Ushbu dialog oynalari turli maqsadlarda ishlatilishi va turli xil xabarlar va tugmalarni ko'rsatish uchun ko'p jihatdan moslashtirilgan bo'lishi mumkin.

Quyida oddiy QMessageBox vidjetini yaratmoqda. Ammo bu hech qanday qo'shimcha funksiyalar yoki sozlashlarsiz faqat MessageBox vidjetining o'zining tinchlik xolatidagi ko'rinishi bo'ladi.

```
def display():
    msg = QMessageBox(wind)
    msg.setWindowTitle("MessageBox widgeti")
    msg.setText("Bir qancha ma'lumot uchun joy!")

    x = msg.exec_()
```

Va quyida chiqish. Agar siz ekranda oddiy xabarni ko'rsatmoqchi bo'lsangiz, buni qilishning yo'li.



61-rasm. QMessageBox vidjetining o'zining ko'rinishi

QMessageBox vidjetining asosiy xususiyatlari

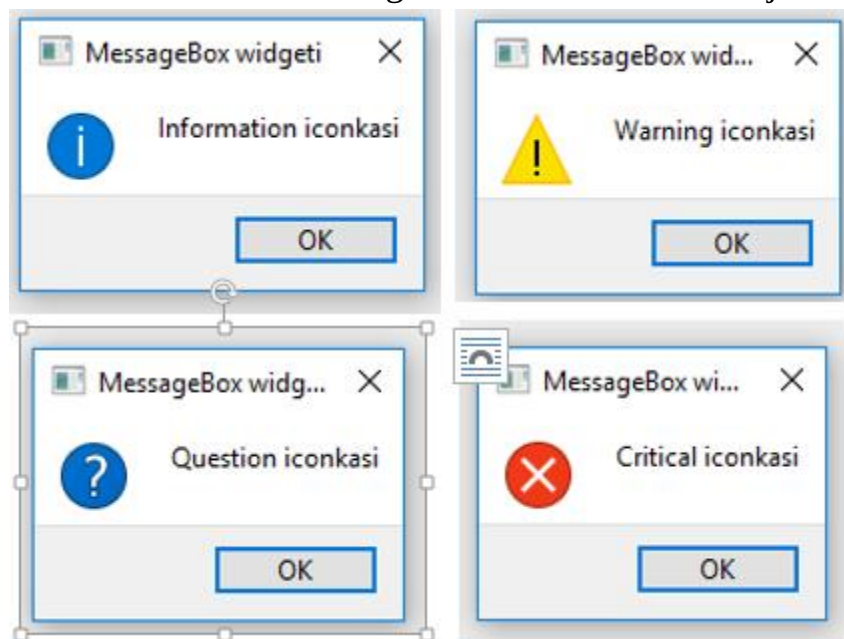
QMessageBox – da **.setIcon()** metodini chaqirish orqali foydalanish mumkin bo'lgan 4 xil turdagi piktogramma mavjud bo'lib, ko'rsatmoqchi bo'lgan xabar turiga qarab qaysi birini ishlatishni hal qilinadi.

- QMessageBox.Question
- QMessageBox.Information
- QMessageBox.Warning
- QMessageBox.Critical

Yuqoridagilardan birini quyida ko'rsatilganidek **setIcon()** funksiyasiga o'tkazish kifoya.

```
msg.setIcon(QMessageBox.Question)
```

Quyida ko'rsatilishi mumkin bo'lgan to'rt xil xabarlar mavjud.



62-rasm. QMessageBox pictogrammalarining turlari

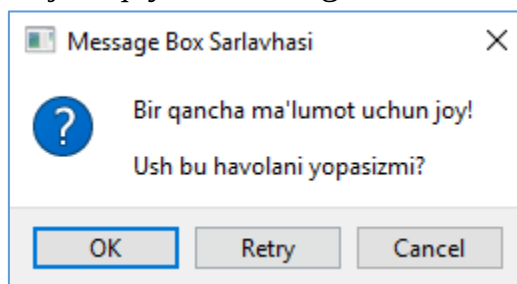
QMessageBox tugmalari

Hozirgacha biz QMessageBox taqdim etgan standart "OK" tugmasidan foydalanib kelmoqdamiz. Biroq, QMessageBox foydalanish uchun o'ndan ortiq turli xil tugmalar taklif qiladi. Quyida ba'zi tugmalardan foydalanish misoli keltirilgan.

setStandardButtons() funksiyadan foydalanib xohlagan tugma turini o'ratish mumkin.

```
def display():
    msg = QMessageBox(wind)
    msg.setWindowTitle("Message Box Sarlavhasi")
    msg.setText("Bir qancha ma'lumot uchun joy!")
    msg.setIcon(QMessageBox.Question)
    msg.setStandardButtons(QMessageBox.Cancel
        | QMessageBox.Ok
        | QMessageBox.Retry)
    msg.setInformativeText("Ush bu havolani yopasizmi?")
    x = msg.exec_()
```

Ushbu dasturning natijasi quyida keltirilgan:



63-rasm. QMessageBox ga doir oddiy dastur natijasi

Buni allaqachon payqagan bo'lishingiz mumkin, lekin tugmalar tartibi ularning xabarlar oynasida qanday ko'rinishiga ta'sir qilmaydi.

Default Button

Yuqoridagi rasmga qarab "OK" tugmasi atrofida ko'k kontur borligini payqash mumkin. Bu ushbu tugma **Default Button** ekanligini bildiradi.

setDefaultButton() funksiyasidan foydalanib, standart tugmani o'zgartirish mumkin. Kodga quyidagi qatorni qo'shish ko'k rangni "Cancel" tugmasi atrofida bo'lishiga olib keladi.

```
msg.setDefaultButton(QMessageBox.Cancel)
```

Bu yerda **QMessageBox** vidjetida foydalanish mumkin bo'lgan turli xil tugma turlarining to'liq ro'yxati keltirilgan:

- QMessageBox.Ok
- QMessageBox.No
- QMessageBox.Yes
- QMessageBox.Cancel
- QMessageBox.Close
- QMessageBox.Abort
- QMessageBox.open
- QMessageBox.Retry
- QMessageBox.Ignore
- QMessageBox.Save
- QMessageBox.Retry
- QMessageBox.Apply
- QMessageBox.Help
- QMessageBox.Reset
- QMessageBox.SaveAll
- QMessageBox.YesToAll
- QMessageBox.NoToAll

3. Statik funksiyalari

Odatda QMessageBox-da Matnni ko'rsatadigan faqat bitta maydon mavjud. Aslida esa, qo'shimcha matn qo'shish uchun yana ikkita qo'shimcha maydon mavjud.

Birinchisi, QMessageBox oynasining o'zida qo'shimcha matn bo'limi. Bu qo'shish mumkin bo'lgan qo'shimcha matn qatoriga o'xshaydi. Ushbu yangi matn maydonini qo'shish uchun faqat setInformativeText() funksiyasiga murojaat qilish kerak.

Ikkinchisi QMessageBox-dan kengaytirilgan maydonda ko'rsatiladi. Bu **Detailed Text** – ya'ni "Batafsil matn" deb ataladi. Ushbu bo'limni o'rnatish avtomatik ravishda ushbu maydonni ko'rsatish uchun ishlatiladigan tugmani qo'shadi. Ushbu

batafsil matn yoziladigan maydonni qo'shish uchun faqat **setDetailedText()** funksiya kerak bo'ladi.

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

def show_popup():
    msg = QMessageBox(win)
    msg.setWindowTitle("Message Box sarlavhasi")
    msg.setText("Bu qandaydir matn")
    msg.setIcon(QMessageBox.Question)

    msg.setStandardButtons(QMessageBox.Cancel|QMessageBox.Ok)
    msg.setDefaultButton(QMessageBox.Ok)

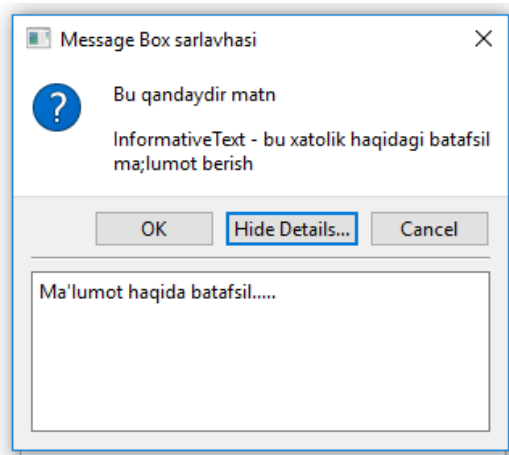
    msg.setDetailedText("Ma'lumot haqida batafsil.....")
    msg.setInformativeText("InformativeText - bu xatolik haqidagi batafsil ma'lumot berish")
    x = msg.exec_()

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,300,300)
win.setWindowTitle("QMessageBox widgeti haqida")

button = QtWidgets.QPushButton(win)
button.setText("A Button")
button.clicked.connect(show_popup)
button.move(100,100)

win.show()
sys.exit(app.exec_())
```

Yuqoridagi dasturning ishga tushirilganidan keyin quyidagi messagebox oynasi paydo bo'ladi:



QMessageBox widgetining to'liq ko'rinishi

Hide Details/Show Details tugmasini bosish mos ravishda xabarlar qutisi ostidagi qo'shimcha maydonni yashirish/ko'rsatish mumkin.

QMessageBox qiymatlarini olish

Yuqorida juda ko'p turli xil sozlashlar va xususiyatlarni muhokama qilindi, ammo aslida bu turli xususiyat va tugmalarning hech birini kodga bog'lanmadi.

Misol uchun, agar MessageBoxda 3 xil tugma bo'lsa, qaysi biri bosilganligini qanday bilish mumkin? Buning uchun quyida ko'rsatilgan kodyaqqol javob bo'la oladi:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

def show_popup():
    msg = QMessageBox(win)
    msg.setWindowTitle("Message Box sarlavhasi")
    msg.setText("Bu qandaydir matn")
    msg.setIcon(QMessageBox.Question)
    msg.setStandardButtons(QMessageBox.Cancel|QMessageBox.Ok)
    msg.setDefaultButton(QMessageBox.Ok)

    msg.setDetailedText("Ma'lumot haqida batafsil.....")
    msg.setInformativeText("InformativeText - bu xatolik haqidagi batafsil ma'lumot berish")
    msg.buttonClicked.connect(popup)
    x = msg.exec_()

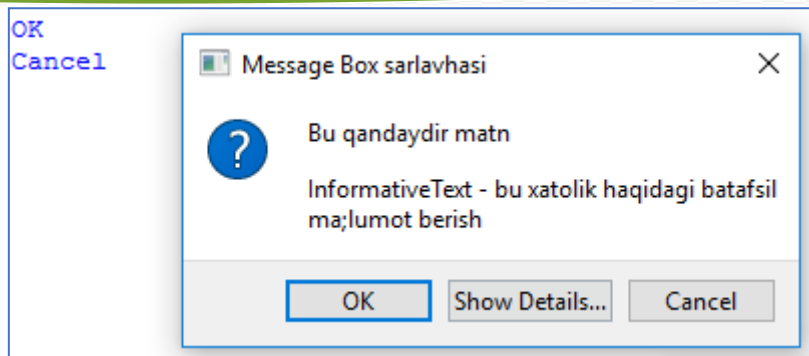
def popup(i):
    print(i.text())

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,300,300)
win.setWindowTitle("QMessageBox widgeti haqida")

button = QtWidgets.QPushButton(win)
button.setText("A Button")
button.clicked.connect(show_popup)
button.move(100,100)

win.show()
sys.exit(app.exec_())
```

buttonClicked.connect() usulidan foydalanib MessageBoxdagi birorta tugma bosilganda funksiyani ishga tushirish mumkin. Bunday holda, messageboxni **popup** deb nomlangan funksiyaga bog'landi. popup() funksiyas bitta parametr qabul qiladi.



QMessageBox vidjetining to'liq imkoniyatlari bilan ko'rinishi

MessageBoxning tugmasi bosilganda, u avtomatik ravishda bosilgan tugmani **buttonClicked.connect()** da e'lon qilingan funksiyaga uzatadi. Ushbu tugmadagi **text()** funksiyasini chaqirish esa tugmaning matnini qaytaradi.

Nazorat savollari:

1. QMessageBox vidjeti qanday vazifani bajaradi?
2. QMessageBox qanday ishga tushiriladi?
3. QMessageBox ning qanday xususiyatlari mavjud?
4. QMessageBox vidjetida qanday pictogrammalar mavjud?
5. QMessageBoxga o'rnatish mumkin bo'lgan standart buttonlarni keltiring?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

6-mashg'ulot. PyQt da rasmlar va menyular.

O'quv savollari:

1. PyQt paketi yordamida rasm joylashtirish usullari.
2. Menu yaratish. Menu vidjetining xususiyatlari.

1. PyQt paketi yordamida rasm joylashtirish usullari

QPixmap vidjeti

QPixmap – bu tasvirlarni ekranga chiqarish uchun ishlatiladigan vidjetlardan biri. Quyidagi misolda tasvirni ekranda ko'rsatish uchun **QPixmap** metodidan foydalanilgan.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QPixmap

class Example(QWidget):
    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        h_box = QHBoxLayout(self)
        pixmap = QPixmap("blue_black.png")

        lbl = QLabel(self)
        lbl.setPixmap(pixmap)

        h_box.addWidget(lbl)
        self.setLayout(h_box)

        self.move(300, 200)
        self.setWindowTitle('Blue and Black')
        self.show()

if __name__ == '__main__':

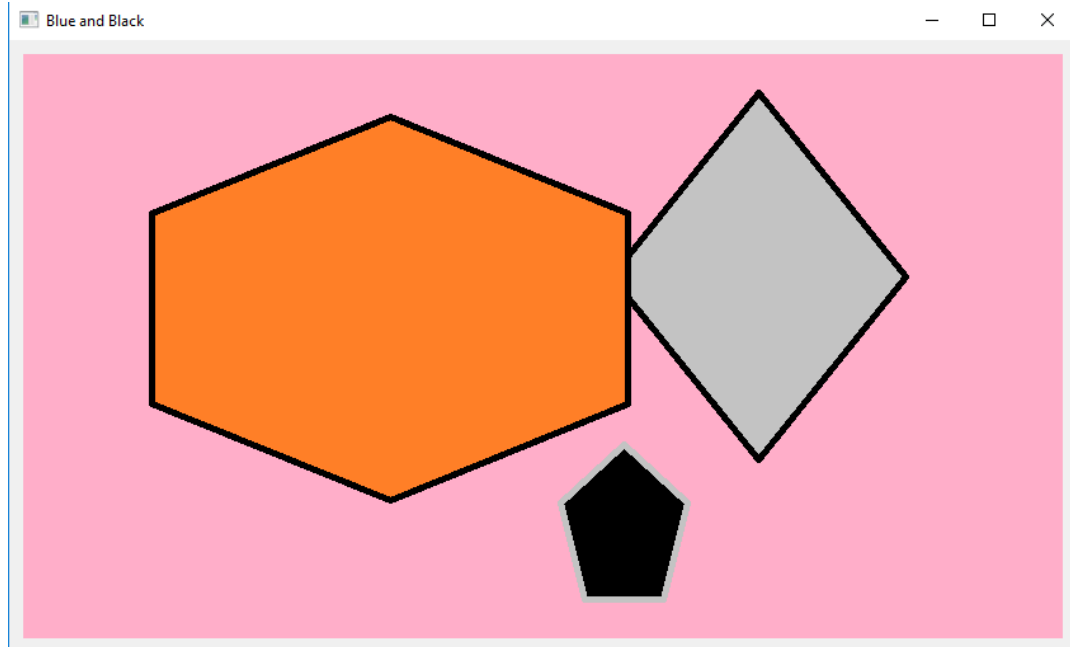
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())
```

Ushbu dasturda `blue_black.png` nomli rasmni oynada ko'rsatish uchun quyidagi amallarni bajarildi:

1. `pixmap = QPixmap("blue_black.png")` - **QPixmap** sinfi obyektini yaratildi.

2. `lbl = QLabel(self)`

`lbl.setPixmap(pixmap)` - rasmni QLabel vidjetidan foydalanib ekranga joylashtirildi.



71-rasm. QPixmap vidjetiga rasm joylashtirilgandagi ko'rinishi

2. Menu yaratish. Menu vidjetining xususiyatlari.

PyQt5 ga bag'ishlangan bobning ushbu qismida menyu va asboblari paneli (asboblari paneli) vidjetlaridan foydalanishni ko'rib chiqiladi.

Menyu - bu menyu satrida joylashgan buyruqlar guruhi. Asboblari panelida ilovadagi ba'zi umumiy buyruqlar uchun tugmalar mavjud.

Menyu paneli

Menyu paneli GUI ilovasining umumiy qismidir. Bu turli xil menyularda joylashgan buyruqlar guruhi.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QIcon

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        exitAction = QAction(QIcon('exit.png'), '&Exit', self)
        exitAction.setShortcut('Ctrl+Q')
        exitAction.setStatusTip('Exit application')
        exitAction.triggered.connect(qApp.quit)
```

```

self.statusBar()

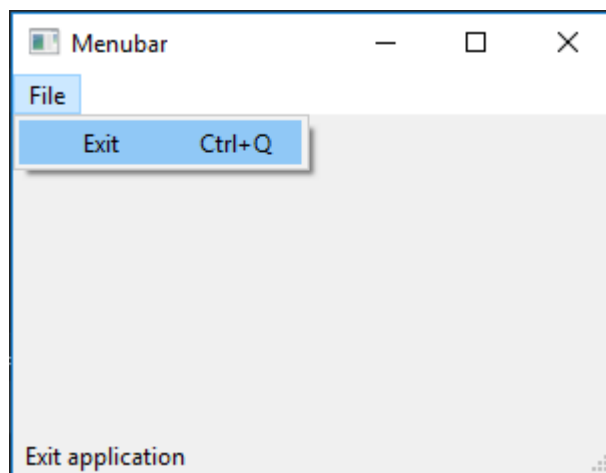
menubar = self.menuBar()
fileMenu = menubar.addMenu('&File')
fileMenu.addAction(exitAction)

self.setGeometry(300, 300, 300, 200)
self.setWindowTitle('Menubar')
self.show()

if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```



Menubar widgetining ilovada ko'rinishi.

Yuqoridagi misolda bitta menubar vidjeti yordamida menyu panelini yaratildi. Ushbu menyu tarkibida dasturni to'xtatadigan bitta buyruq kiritilgan. Ushbu buyruqni menular panelidan sichqoncha bilan tanlash yoki klaviaturadan Ctrl + q tugmalari yordamida ishga tushirish ham mumkin.

Datur tahlili:

```

exitAction = QAction(QIcon('exit.png'), '&Exit', self)
exitAction.setShortcut('Ctrl+Q')
exitAction.setStatusTip('Exit application')

```

Ushbu uchta qatorda tegishli belgi bilan hodisa yaratilgan. Bundan tashqari, ushbu hodisa uchun tugmalar birikmasi aniqlanadi. Uchinchi qator kursorni menyu bandi ustiga olib borilganda satr yonida maslahatchiga o'xshagan yozuv yaratadi.

```
exitAction.triggered.connect(qApp.quit)
```

Ushbu maxsus harakatni tanlaganimizda, signal ishga tushadi. Signal **QApplication** vidjetining **quit()** metodini ishga tushiradi va ilovani yakunlaydi.

```

menubar = self.menuBar()
fileMenu = menubar.addMenu('&File')
fileMenu.addAction(exitAction)

```

Bu yerga **menyuBar()** vidjeti yordamida bitta menular satri yaratilyapdi. fileMenu o'zgaruvchisiga menular satri fa File nomli menu qo'shish buyrug'i yuklanmoqda. Va uchinchi satrda Fayl menyusining tarkibida dasturdan chiqish amalini qo'shilmoqda.

Asboblari paneli (toolbar)

Menyular ilovada foydalanishimiz mumkin bo'lgan barcha buyruqlarni guruhlaydi.

Asboblari paneli eng ko'p ishlatiladigan buyruqlarga tezkor kirishni ta'minlaydi.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QIcon

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

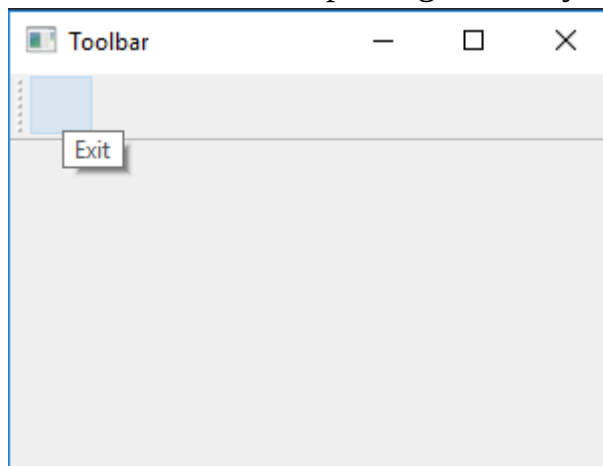
    def initUI(self):
        exitAction = QAction(QIcon('exit24.png'), 'Exit', self)
        exitAction.setShortcut('Ctrl+Q')
        exitAction.triggered.connect(qApp.quit)

        self.toolbar = self.addToolBar('Exit')
        self.toolbar.addAction(exitAction)

        self.setGeometry(300, 300, 300, 200)
        self.setWindowTitle('Toolbar')
        self.show()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())
```

Bu yerdagi deyarli hamma narsa status paneliga o'xshaydi.



Asboblari panelining ilovada ko'rinishi.

Hammasini birlashtirish

Ushbu bo‘limning oxirgi misolida menyu satri, asboblari paneli va holat panelini yaratildi. Shuningdek, markaziy vidjetni ham ular bilan birlashtirilgan.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QIcon

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        textEdit = QTextEdit()
        self.setCentralWidget(textEdit)

        exitAction = QAction(QIcon('exit24.png'), 'Exit', self)
        exitAction.setShortcut('Ctrl+Q')
        exitAction.setStatusTip('Exit application')
        exitAction.triggered.connect(self.close)

        self.statusBar()

        menubar = self.menuBar()
        fileMenu = menubar.addMenu('&File')
        fileMenu.addAction(exitAction)

        toolbar = self.addToolBar('Exit')
        toolbar.addAction(exitAction)

        self.setGeometry(300, 300, 350, 250)
        self.setWindowTitle('Main window')
        self.show()

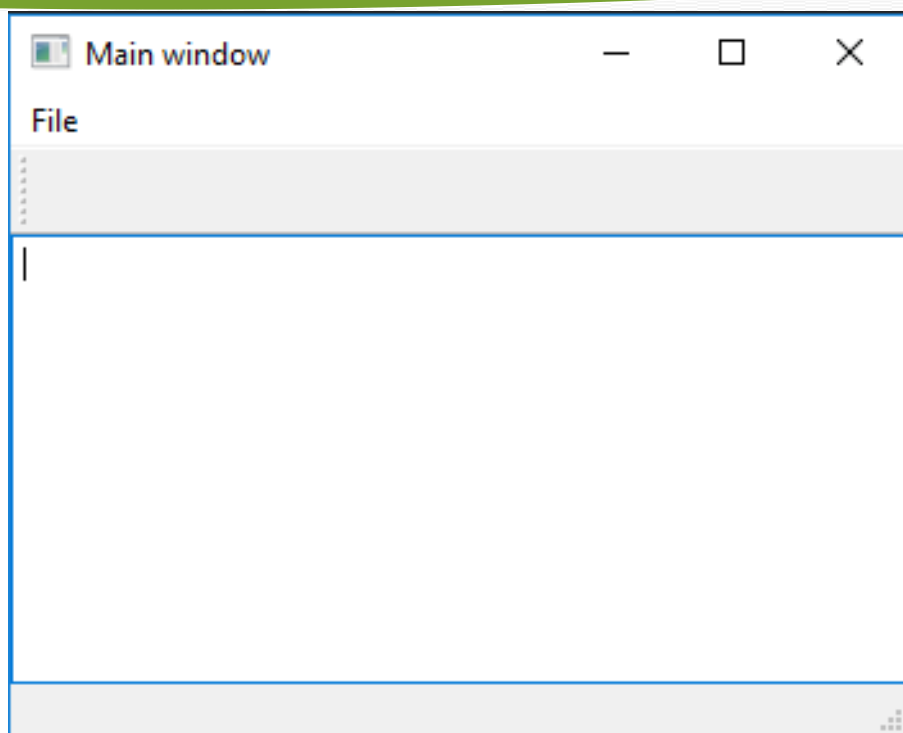
if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())
```

Ushbu kod misoli menyu, asboblari paneli va holat paneli bilan klassik GUI ilovasining skeletini yaratadi.

```
textEdit = QTextEdit()
self.setCentralWidget(textEdit)
```

Bu yerda matnni tahrirlash vidjetini yaratilmoqda. Uni **QMainWindow** ning markaziy vidjetiga aylantiriladi. Markaziy vidjet qolgan barcha bo‘sh joyni egallaydi.



Menu va uning asboblar paneli ko'rinishi.

Nazorat savollari:

1. Menuni hosil qilishi uchun qanday widgetdan foydalaniladi?
2. Statusbar widgeti nima vazifani bajaradi?
3. Menular panelini qaysi widgetdan foydalanib o'rnatiladi?
4. QPixmap vidjetining imkoniyatlaridan qanday foydalaniladi?
5. QPixmap vidjetining qanday xususiyatlari mavjud?
6. QPixmap vidjeti yordamida rasm joylashtirish qanday amalga oshiriladi?

O'quv savollari:

- ## 1. Yaratiladigan matn muharriri uchun kerakli uskunalarni tanlash

Bizga nimalar kerak bo‘ladi?

Birinchi navbatda Windowsda PyQt paketini va Qt Designer dasturini yuklab olish hamda oʻrnatish talab qilinadi. QtDesigner dasturi PyQt paketi bilan birgalikda oʻrnatiladi.

Linux : Sizga kerak bo'lgan hamma narsa, ehtimol, tarqatish omborlarida. Qt Designer ilova markazidan o'rnatilishi mumkin, lekin PyQt terminal orqali o'rnatilishi kerak. Bizga kerak bo'lgan hamma narsani bitta buyruq bilan o'rnatishingiz mumkin, masalan:

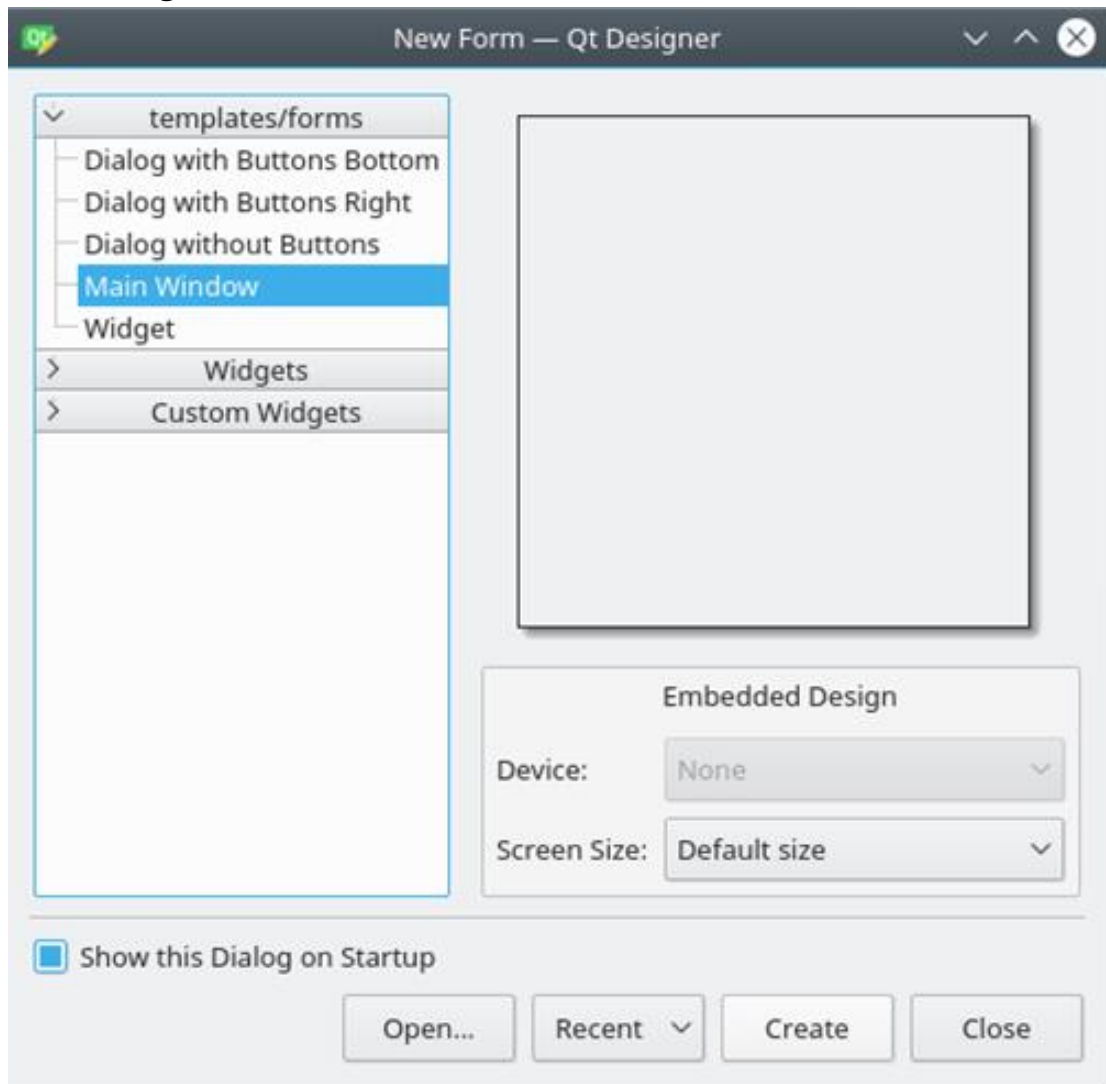
```
$ sudo apt install python3-gt5 pyqt5-dev-tools qtccreator
```

Error: one **input** ui-file must be specified

Dastur Dizaynini yaratish

170

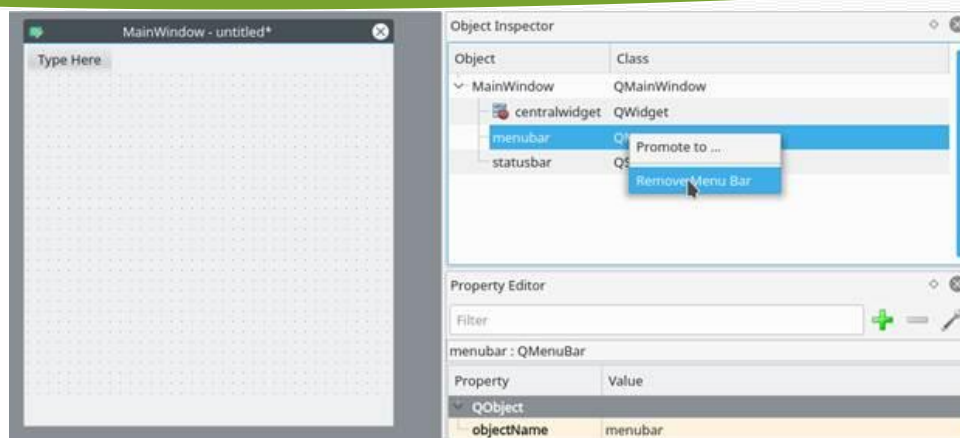
Qt Designer dasturini ishga tushuriladi, yangi ochilgan oynada kichik dialog oynasi paydo bo'ladi. U yerdan **“Main Window”** bo'limi tanlab, **Create** tugmasini bosing.



Shundan so'ng sizda **forma** - oyna dizayn uchun shablon paydo bo'lishi kerak, uning o'lchamini o'zgartirish mumkin va vidjet oynasidan obyektlarni qayerga hohlasangiz qo'shishingiz mumkin bo'ladi. QtDesigner dasturi interfeysi bilan tanishib chiqing, bu juda ham oddiy tuzulishga ega.

Endi asosiy oynamiz hajmini biroz o'zgartiramiz. Bizga bunchalik katta bo'lishi shart emas. Va keling, avtomatik qo'shilgan menyu va holat panelini ham olib tashlaymiz, chunki ular bizning ilovamizda foydali bo'lmaydi.

Barcha shakl elementlari va ularning ierarxiyasi sukut bo'yicha Qt Designer oynasining o'ng tomonidagi *“Object Inspector”* deb nomlangan oynada ko'rsatiladi. Ushbu oynada obyektlarni sichqonchani o'ng tugmasi bilan osongina o'chirishingiz mumkin. Yoki ularni asosiy shaklda tanlashingiz va klaviaturangizdagi **DEL** tugmasini bosishingiz mumkin.



Natijada, bizda deyarli bo'sh shakl mavjud. Qolgan yagona obyekt - **centralwidget**, lekin bizga kerak bo'ladi, shuning uchun biz u bilan hech narsa qilmaymiz.

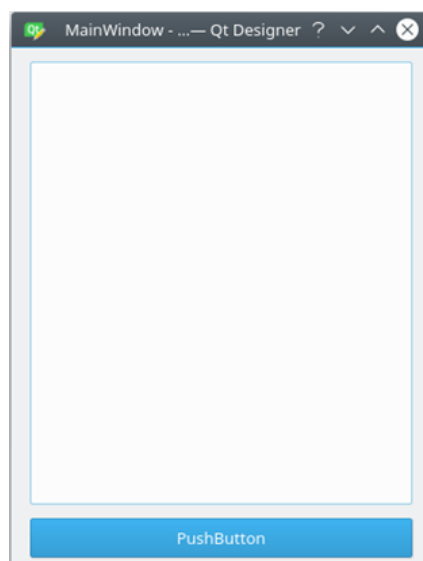
2. Tanlangan uskunalarni matn muharriri ekraniga joylashtirish

Endi *WidgetBox* dan *ListWidget* ni (*ListView* emas) va *PushButton* larni asosiy shaklning biror joyiga joylashtiring.

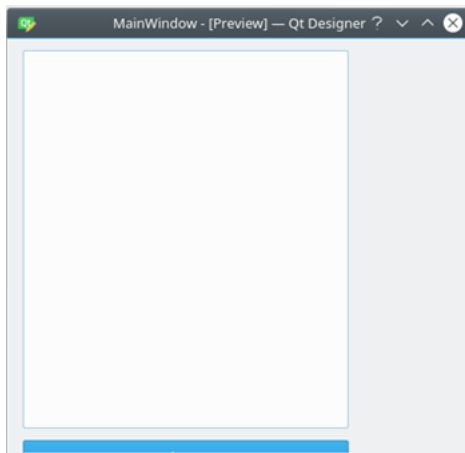
Maketlar. Ilovangizdagi elementlar uchun qat'iy pozitsiyalar va o'lchamlardan foydalanish o'rniga, maketlardan foydalangan ma'qul. Ruxsat etilgan pozitsiyalar va o'lchamlar sizga yaxshi ko'rinadi (oyna o'lchamini o'zgartirmaguningizcha), lekin boshqa mashinalar va/yoki operatsion tizimlarda aynan bir xil bo'lishiga hech qachon ishonch hosil qila olmaysiz.

Maketlar - bu vidjetlar uchun konteynerlar bo'lib, ularni boshqa elementlarga nisbatan o'z o'rnida ushlab turadi. Shuning uchun, oyna hajmi o'zgartirilganda, vidjetlarning o'lchami ham o'zgaradi.

Keling, maketlardan foydalanmasdan birinchi shaklimizni yarataylik. Shakldagi ro'yxat va tugmani torting va ularning o'lchamlarini quyidagicha ko'rinishda o'zgartiring:

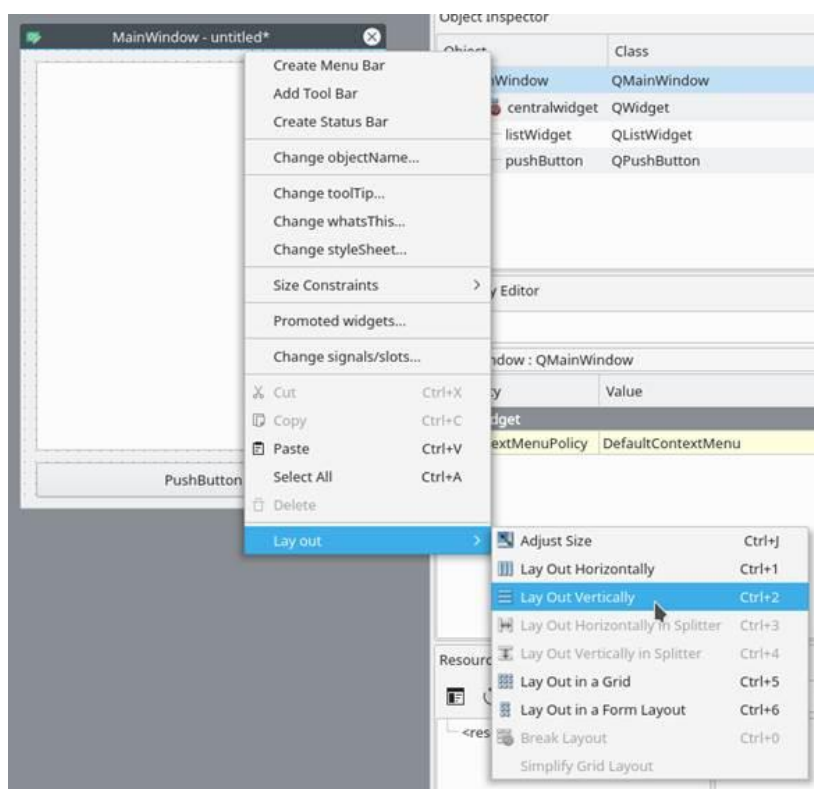


Endi Qt Designer menyusida *Formni* bosing, so'ng *Preview-ni* tanlang va yuqoridagi skrinshotga o'xshash narsani ko'rasiz. Yaxshi ko'rinadi, shunday emasmi? Ammo oyna o'lchamini o'zgartirganda nima sodir bo'ladi:



Asosiy oynaning o'lchami o'zgargan va tugma deyarli ko'rinmas bo'lsada, bizning obyektlarimiz bir xil joylarda qoldi va o'z o'lchamlarini saqlab qoldi. Shuning uchun ko'p hollarda maketlardan foydalanishga arziydi. Albatta, masalan, obyekt uchun qattiq yoki minimal/maksimal kenglik kerak bo'lgan paytlar mavjud. Ammo umuman olganda, dasturni ishlab chiqishda maketlardan foydalanish yaxshiroqdir.

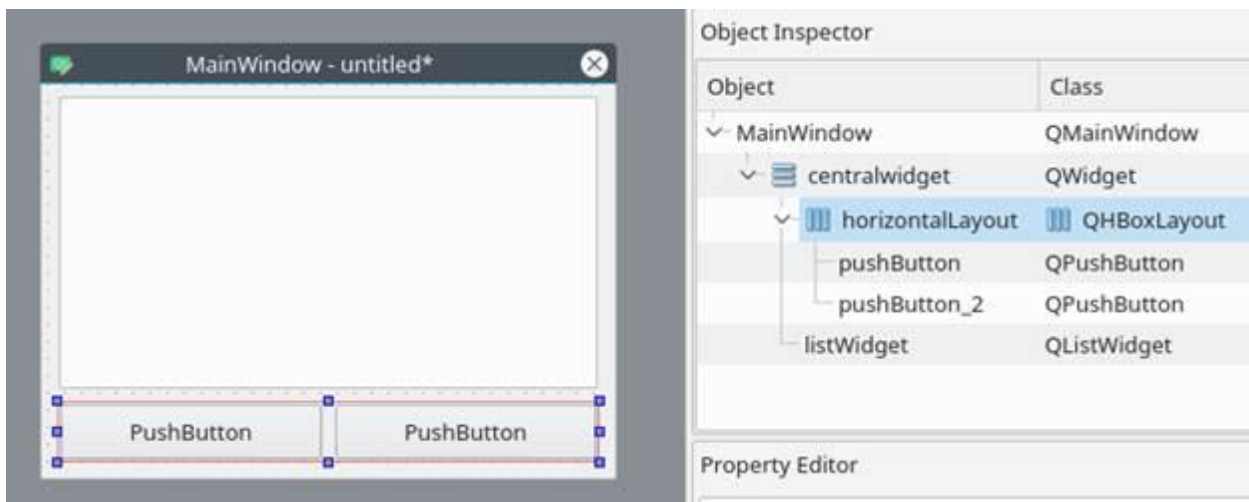
Asosiy oyna allaqachon maketlarni qo'llab-quvvatlaydi, shuning uchun formamizga hech narsa qo'shishimiz shart emas. *Obyekt* inspektoridagi **Main Window** ning o'ng tugmasini bosing va **Lay Out** → **Lay out vertically** ni tanlang. Shuningdek, siz formadagi bo'sh joyni sichqonchaning o'ng tugmasi bilan bosishingiz va bir xil variantlarni tanlashingiz mumkin:



3. Matn muharririr ekrani dizaynini shakllantirish

Sizning elementlaringiz o'zgarishlar kiritilishidan oldingi tartibda bo'lishi kerak, ammo agar ular bo'lmasa, ularni kerakli joyga sudrab olib boring.

Biz vertikal joylashtirishdan foydalanganimiz sababli, biz qo'shgan barcha elementlar vertikal ravishda joylashtiriladi. Istalgan natijani olish uchun joylashtirishlarni birlashtira olasiz. Masalan, ikkita tugmani gorizontal ravishda vertikalga qo'yish quyidagicha ko'rinadi:



Agar siz asosiy oynada elementni ko'chira olmasangiz, buni *Object Inspector* oynasida qilishingiz mumkin .

Yakuniy urinishlar

Endi vertikal joylashtirish tufayli bizning elementlarimiz to'g'ri hizalanadi. Qolgan yagona narsa (lekin shart emas) elementlarning nomini va ularning matnini o'zgartirishdir.

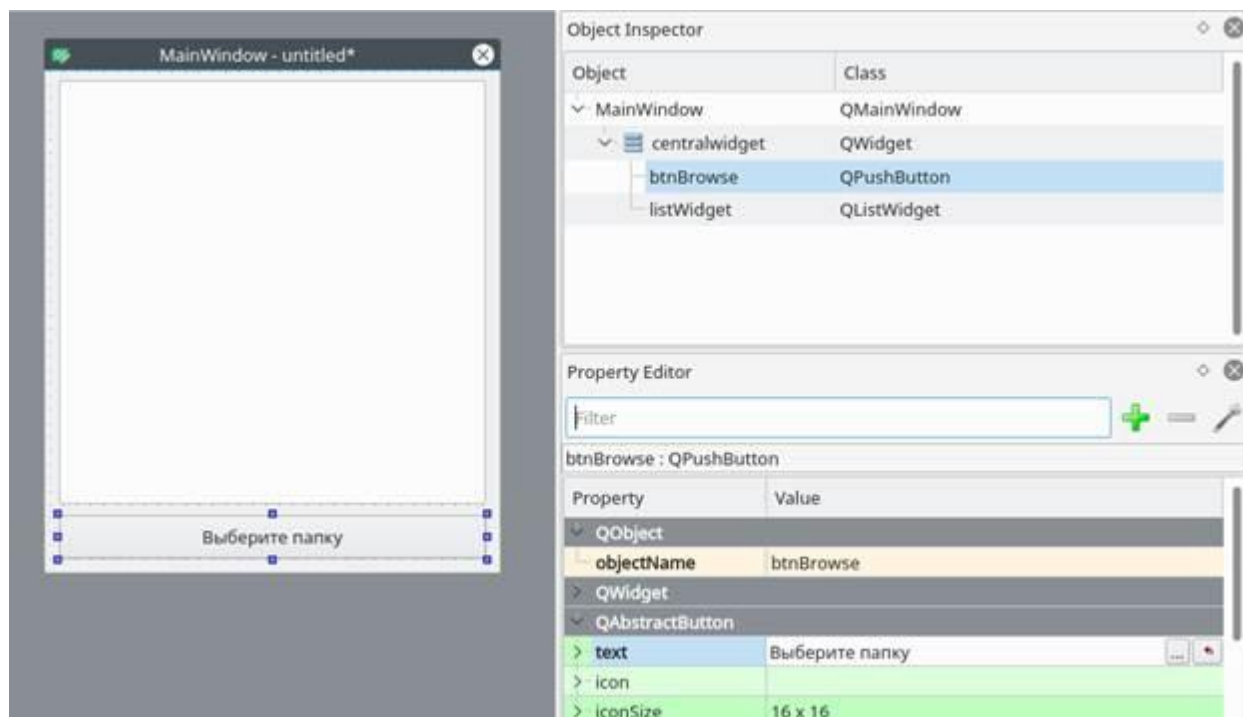
Ro'yxat va tugma bo'lgan bu kabi oddiy dasturda nomlarni o'zgartirish shart emas, chunki baribir undan foydalanish oson. Biroq, elementlarning to'g'ri nomlanishi siz boshidanoq o'rganishingiz kerak bo'lgan narsadir.

Element xususiyatlari **Property Editor** bo'limida o'zgartirilishi mumkin.

Ko'rsatma: Ish jarayonini tezlashtirish uchun siz Qt Designer interfeysiga tez-tez ishlatiladigan elementlarning o'lchamini o'zgartirishingiz, ko'chirishingiz yoki qo'shishingiz mumkin. View menyusi elementi orqali interfeysning yashirin/yopiq (скрытые/закрытые) qismlarini qo'shishingiz mumkin.

Shaklga qo'shgan tugmani bosing. Endi *Property Editor* da siz ushbu elementning barcha xususiyatlarini ko'rishingiz kerak. Biz hozirda QAbstractButton bo'limidagi objectName va matnga qiziqamiz. Bo'lim nomini bosish orqali *Property Editor* da bo'limlarni yopishingiz mumkin.

ObjectName qiymatini *btnBrowse* ga o'zgartiring va matnni *Jildni* tanlang.
Bu shunday bo'lishi kerak:



Nazorat savollari:

1. Pythonda gui bilan ishlash uchun qanday kutubxonalari mavjud?
2. PyQt5 paketini qanday o'rnatiladi va chaqirib ishlatish qanday amalga oshiriladi?
3. pyuic5 buyrug'i nima uchun foydalaniladi?
4. Papkalarni ochish uchun qanday kutubxonadan foydalaniladi?
5. Kataloglar ustida qanday amallar bajarish mumkin?
6. Faylni saqlash uchun qanday amal bajariladi?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

8-mashg'ulot. Matn muharriri dasturini yozish.

O'quv savollari:

1. PyQt5 da matn muharriri dizaynidagi elementlarning funksiyalari uchun dastur yozish.
2. Dasturni testlash.

PyQt5 da matn muharriri dizaynidagi elementlar funksiyalari uchun dastur yozish

Ro'yxat obyeksi nomi **listWidget** bo'lishi kerak — bu to'g'ri. Dizaynni loyiha papkasida **design.ui** nomi bilan saqlang.

Dizaynni kodga aylantirish

.ui fayllarini Python kodida to'g'ridan-to'g'ri ishlatish mumkin, ammo yanada qulay usul ham mavjud — ya'ni .ui faylini Python fayliga aylantirish. Buning uchun terminal yoki buyruqlar satrida **pyuic5** buyrug'idan foydalanamiz.

Masalan, design.ui faylini design.py nomli Python fayliga o'tkazish uchun quyidagi buyruqni kiriting:

```
$ pyuic5 path/to/design.ui -o output/path/to/design.py
```

Kod yozishni boshlash

Endi bizda **QtDesigner** dasturida yaratilgan dizaynning Python faylga aylantirilgan versiyasi — design.py mavjud. Endi biz uning mantiqiy qismini, ya'ni asosiy dastur qismini yozishni boshlaymiz.

Birinchi navbatda, design.py joylashgan papkada **main.py** faylini yarating.

QtDesigner dizaynidan foydalanish

Python GUI ilovasi uchun quyidagi modullar kerak bo'ladi:

```
import sys
from PyQt5 import QtWidgets
```

Shuningdek, avval yaratilgan dizayn faylini ham import qilamiz:

```
import design # Bu bizning o'zgartirilgan dizayn faylimiz
```

Dizayn fayli har safar o'zgartirilganda qayta yaratiladi, shuning uchun uni tahrirlamaymiz. Buning o'rniga yangi sinf yaratamiz — **ExampleApp**, u design.py dagi barcha funksiyalar bilan birlashtiriladi:

```
class ExampleApp(QtWidgets.QMainWindow, design.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)
```


Bu sinfdagi biz interfeys elementlari bilan o‘zaro ishlaymiz, signal-slot (ulanish) funksiyalarini qo‘shamiz va dastur mantiqini yozamiz. Endi esa bu sinfni ishga tushirish uchun **main()** funksiyasini yozamiz:

```
def main():
    app = QtWidgets.QApplication(sys.argv)
    window = ExampleApp()
    window.show()
    app.exec_()
```

Va uni quyidagicha chaqiramiz:

```
if __name__ == '__main__':
    main()
```

Natijada main.py quyidagicha ko‘rinadi:

```
import sys
from PyQt5 import QtWidgets
import design

class ExampleApp(QtWidgets.QMainWindow, design.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

def main():
    app = QtWidgets.QApplication(sys.argv)
    window = ExampleApp()
    window.show()
    app.exec_()

if __name__ == '__main__':
    main()
```

Agar ushbu kodni python3 main.py orqali ishga tushirsangiz — ilova oynasi ochiladi. Ammo hozircha tugmalar hech qanday amal bajarmaydi, shuning uchun funktsionallik qo‘shamiz.

Ilovaga funktsionallik qo‘shish

Eslatma: Quyidagi barcha kodlar ExampleApp sinfi ichida yoziladi.

Avvalo “papkani tanlash” tugmasidan boshlaymiz. Tugma bosilganda funktsiyani ishga tushirish uchun **clicked.connect()** metodidan foydalanamiz:

```
self.btnBrowse.clicked.connect(self.browse_folder)
```

Bu qatorni `__init__` metodiga joylashtiramiz.

Tushuntirish:

- self.btnBrowse — Qt Designer’da aniqlangan tugma nomi.
- clicked — tugma bosilganda sodir bo‘ladigan hodisa (signal).
- connect() — hodisani funktsiyaga bog‘laydigan metod.
- self.browse_folder — biz yozadigan metod nomi.

Papka tanlash oynasi

Papka tanlash oynasini chaqirish uchun quyidagidan foydalanamiz:

```
directory = QtWidgets.QFileDialog.getExistingDirectory(self, "Papkani tanlash")
```

Agar foydalanuvchi katalogni tanlasa, directory o'zgaruvchisiga tanlangan papkaning manzili yoziladi; agar oynani yopsa — qiymat None bo'ladi. Shuning uchun biz if directory: bilan shart qo'yamiz.

Papka tarkibini olish uchun os moduli kerak bo'ladi:

```
import os
```

Papka ichidagi fayllarni olish uchun:

```
os.listdir(path)
```

listWidget ga element qo'shish uchun addItem() metodidan, hammasini tozalash uchun clear() metodidan foydalanamiz.

Natijada browse_folder funksiyasi quyidagicha bo'ladi:

```
def browse_folder(self):
    self.listWidget.clear()
    directory = QtWidgets.QFileDialog.getExistingDirectory(self, "Papkani tanlash")

    if directory:
        for file_name in os.listdir(directory):
            self.listWidget.addItem(file_name)
```

Yakuniy kod

Quyida Python GUI ilovasining to'liq kodi keltirilgan:

```
import sys
import os
from PyQt5 import QtWidgets
import design

class ExampleApp(QtWidgets.QMainWindow, design.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)
        self.btnBrowse.clicked.connect(self.browse_folder)

    def browse_folder(self):
        self.listWidget.clear()
        directory = QtWidgets.QFileDialog.getExistingDirectory(self, "Papkani tanlash")

        if directory:
            for file_name in os.listdir(directory):
                self.listWidget.addItem(file_name)

def main():
    app = QtWidgets.QApplication(sys.argv)
    window = ExampleApp()
```

```
window.show()
app.exec_()

if __name__ == '__main__':
    main()
```

Bu PyQt5 va Qt Designer yordamida Python GUI ilovasini ishlab chiqishning asosiy bosqichlari edi. Endi siz dizaynni o'z g'oyalaringizga moslashtirishingiz va pyuic5 yordamida .ui fayllarini Python kodiga oson aylantirishni bilasiz.

Nazorat savollari:

1. Pythonda gui bilan ishlash uchun qanday kutubxonalari mavjud?
2. PyQt5 paketini qanday o'rnatiladi va chaqirib ishlatish qanday amalga oshiriladi?
3. Pyuic5 buyrug'i nima uchun foydalaniladi?
4. Papkalarni ochish uchun qanday kutubxonadan foydalaniladi?
5. Kataloglar ustida qanday amallar bajarish mumkin?
6. Faylni saqlash uchun qanday amal bajariladi?

MA'RUZA MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

1-mashg'ulot. Tarmoqda ma'lumot almashuvchi klient-server dasturini tuzish.

O'quv savollari:

1. Socket modulining asosiy metodlari bilan tanishish.
2. TCP protokoli asosida klient-server ulanishini tashkil qilish.
3. Ma'lumotlarni uzatish va qabul qilish jarayonini amalda bajarish.
4. Klient va server dasturlarida xatoliklarni aniqlash va bartaraf etish.

1. Socket modulining asosiy metodlari bilan tanishish

Python tarmoq xizmatlariga kirishning ikki darajasini ta'minlaydi. Past darajada, siz asosiy operatsion tizimdagi asosiy rozetka yordamiga kirishingiz mumkin, bu sizga ulanishga yo'naltirilgan va ulanishsiz protokollar uchun mijozlar va serverlarni amalga oshirish imkonini beradi.

Python, shuningdek, FTP, HTTP va boshqalar kabi ma'lum amaliy qatlam tarmoq protokollariga yuqori darajadagi kirishni ta'minlaydigan kutubxonalarga ega.

Ushbu bo'lim sizga Internetdagi eng mashhur kontsepsiya - Socket Programming haqida tushuncha beradi.

Soketlar nima?

Soketlar ikki tomonlama aloqa kanalining so'nggi nuqtalari hisoblanadi. Soketlar jarayon ichida, bitta mashinadagi jarayonlar o'rtasida yoki turli qit'alardagi jarayonlar o'rtasida aloqa qilishi mumkin.

Soketlar bir nechta turli turdagi quvurlar uchun amalga oshirilishi mumkin: Unix domen soketlari, TCP, UDP va boshqalar. *Soket* kutubxonasi umumiy transportlarni boshqarish uchun maxsus sinflarni, shuningdek, qolganlarini boshqarish uchun umumiy interfeysni taqdim etadi.

Soketlarning o'z lug'ati bor.

t/r	Muddati va tavsifi
1	Domen Transport mexanizmi sifatida foydalaniladigan protokollar oilasi. Bu qiymatlar AF_INET, PF_INET, PF_UNIX, PF_X25 va boshqalar kabi konstantalardir.
2	turi Ikki so'nggi nuqta o'rtasidagi aloqa turi, odatda ulanishga yo'naltirilgan protokollar uchun SOCK_STREAM va ulanishsiz protokollar uchun SOCK_DGRAM.
3	protokol

	Odatda nolga teng, bu domen va turdagi protokol variantini aniqlash uchun ishlatilishi mumkin.
4	xost nomi Tarmoq interfeysi identifikatori - • Xost nomi, to'rt nuqtali manzil yoki ikki nuqta (va ehtimol nuqta) belgisidagi IPV6 manzili bo'lishi mumkin bo'lgan qator. • INADDR_BROADCAST manzilini bildiruvchi "<broadcast>" qatori. • INADDR_ANY ni belgilaydigan nol uzunlikdagi qator yoki • Xost bayt tartibida ikkilik manzil sifatida talqin qilingan butun son.
5	port Har bir server bir yoki bir nechta portga qo'ng'iroq qilayotgan mijozlarni tinglaydi. Port Fixnum port raqami, port raqami qatori yoki xizmat nomi bo'lishi mumkin.

Domen

Transport mexanizmi sifatida foydalaniladigan protokollar oilasi. Bu qiymatlar AF_INET, PF_INET, PF_UNIX, PF_X25 va boshqalar kabi konstantalardir.

type

Ikki so'nggi nuqta o'rtasidagi aloqa turi, odatda ulanishga yo'naltirilgan protokollar uchun SOCK_STREAM va ulanishsiz protokollar uchun SOCK_DGRAM.

protokol

Odatda nolga teng, bu domen va turdagi protokol variantini aniqlash uchun ishlatilishi mumkin.

xost nomi

Tarmoq interfeysi identifikatori -

Xost nomi, to'rt nuqtali manzil yoki ikki nuqta (va ehtimol nuqta) belgisidagi IPV6 manzili bo'lishi mumkin bo'lgan qator.

INADDR_BROADCAST manzilini bildiruvchi "<broadcast>" qatori.

INADDR_ANY ni belgilaydigan nol uzunlikdagi qator yoki

Xost bayt tartibida ikkilik manzil sifatida talqin qilingan butun son.

port

Har bir server bir yoki bir nechta portga qo'ng'iroq qilayotgan mijozlarni tinglaydi. Port Fixnum port raqami, port raqami qatori yoki xizmat nomi bo'lishi mumkin.

socket moduli

Soket yaratish uchun umumiy sintaksisga ega *socket* modulida mavjud *socket.socket()* funksiyasidan foydalanishingiz kerak.

```
s = socket.socket(socket_family, socket_type, protocol=0)
```

Bu erda variantlarning tavsifi:

- **socket_family** AF_UNIX yoki AF_INET, avvalroq tushuntirilganidek.
- **socket_type** - SOCK_STREAM yoki SOCK_DGRAM.
- **protokol** - odatda ko'rsatilmaydi, standart 0.

socket_family AF_UNIX yoki AF_INET, avvalroq tushuntirilganidek.

socket_type - SOCK_STREAM yoki SOCK_DGRAM.

protokol - odatda ko'rsatilmaydi, standart 0.

Soket obyektiga ega bo'lganingizdan so'ng, mijoz yoki server dasturini yaratish uchun kerakli funksiyalardan foydalanishingiz mumkin. Quyida talab qilinadigan xususiyatlar ro'yxati -

Server soket usullari

t/r	Usul va tavsif
1	s.bind() Ushbu usul manzilni (xost nomi, port raqamlari juftligi) rozetkaga bog'laydi.
2	s.tinglash() Ushbu usul TCP tinglovchisini o'rnatadi va ishga tushiradi.
3	s.accept() Bu passiv ravishda TCP mijoz ulanishini qabul qiladi, ulanish o'rnatilguncha kutadi (blokirovka).

s.bind()

Ushbu usul manzilni (xost nomi, port raqamlari juftligi) rozetkaga bog'laydi.

s.tinglash()

Ushbu usul TCP tinglovchisini o'rnatadi va ishga tushiradi.

s.accept()

Bu passiv ravishda TCP mijoz ulanishini qabul qiladi, ulanish o'rnatilguncha kutadi (blokirovka).

Mijoz soket usullari

t/r	Usul va tavsif
1	s.connect() Ushbu usul TCP serveriga ulanishni faol ravishda boshlaydi.

s.connect()

Ushbu usul TCP serveriga ulanishni faol ravishda boshlaydi.

Umumiy soket usullari

t/r	Usul va tavsif
1	s.recv() Ushbu usul TCP xabarini oladi
2	s.send() Ushbu usul TCP xabarini yuboradi
3	s.recvfrom() Bu usul UDP xabarini oladi
4	s.sendto() Bu usul UDP xabarini yuboradi
5	s.close() Ushbu usul rozetkani yopadi
6	socket.getostname() Xost nomini qaytaradi.

s.recv()

Ushbu usul TCP xabarini oladi

s.send()

Ushbu usul TCP xabarini yuboradi

s.recvfrom()

Bu usul UDP xabarini oladi

s.sendto()

Bu usul UDP xabarini yuboradi

s.close()

Ushbu usul rozetkani yopadi

socket.getostname()

Xost nomini qaytaradi.

oddiy server. Internet-serverlarni yozish uchun biz rozetka obyektini yaratish uchun rozetka modulida mavjud **soket funksiyasidan foydalanamiz**. Soket obyekti soket serverini sozlash uchun boshqa funksiyalarni chaqirish uchun ishlatiladi.

Endi berilgan hostda xizmatigiz uchun *portni* belgilash uchun **bog'lash (hostname, port) funksiyasini chaqiring**.

Keyin qaytarilgan obyekt *qabul qilish usulini chaqiring*. Ushbu usul mijoz siz ko'rsatgan portga ulanishini kutadi va keyin ushbu mijozga ulanishni ifodalovchi *Connection obyekti*ni qaytaradi.

```
#!/usr/bin/python          # This is server.py file

import socket              # Import socket module

s = socket.socket()        # Create a socket object
host = socket.gethostname() # Get local machine name
port = 12345              # Reserve a port for your service.
s.bind((host, port))      # Bind to the port

s.listen(5)               # Now wait for client connection.
while True:
    c, addr = s.accept()   # Establish connection with client.
    print 'Got connection from', addr
    c.send('Thank you for connecting')
    c.close()              # Close the connection
```

Oddiy mijoz. Berilgan port 12345 va berilgan xostga ulanishni ochadigan juda oddiy mijoz dasturini yozamiz. *Python soket* moduli funksiyasidan foydalangan holda soket mijozini yaratish juda oddiy .

Socket.connect(hostname, port) portdagi xost *nomi bilan* TCP ulanishini ochadi . Ochiq rozetkaga ega bo'lganingizdan so'ng, siz undan istalgan kiritish-chiqarish obyekti kabi o'qishingiz mumkin. Ishingiz tugagach, faylni yopganingizdek, uni yopishni unutmang.

Quyidagi kod juda oddiy mijoz bo'lib, u berilgan xost va portga ulanadi, rozetkadan barcha mavjud ma'lumotlarni o'qiydi va keyin chiqadi.

```
#!/usr/bin/python          # This is client.py file

import socket              # Import socket module

s = socket.socket()        # Create a socket object
host = socket.gethostname() # Get local machine name
port = 12345              # Reserve a port for your service.

s.connect((host, port))
print s.recv(1024)
s.close()                  # Close the socket when done
```

Endi bu server.py ni fonda ishga tushiring va natijani ko'rish uchun yuqoridagi client.py ni ishga tushiring.

```
# Following would start a server in background.
```

```
$ python server.py &
```

```
# Once server is started run client as follows:
```

```
$ python client.py
```

Bu quyidagi natijani beradi:

```
Got connection from ('127.0.0.1', 48437)
```

```
Thank you for connecting
```

Python Internet modullari

Python Network/Internet dasturlashdagi ba'zi muhim modullar ro'yxati.

protokol	Umumiy funktsiya	Port raqami.	Python moduli
HTTP	veb-sahifalar	80	httplib, urllib, xmlrpclib
NNTP	Usenet yangiliklari	119	nntplib
FTP	Fayl uzatish	20	ftplib, urllib
SMTP	Email yuborish	25	smtplib
POP3	Email qabul qilinmoqda	110	poplib
IMAP4	Email qabul qilinmoqda	143	imaplib
telnet	Buyruqlar qatorlari	23	telnetlib
gofer	Hujjatlarni topshirish	70	gopher, urllib

Iltimos, FTP, SMTP, POP va IMAP protokollari bilan ishlash uchun yuqorida ko'rsatilgan barcha kutubxonalarni tekshiring.

2) TCP protokoli asosida klient–server ulanishini tashkil qilish

TCP nima? U ishonchli, oqimli (stream) transport protokoli. Paketlar yo'qolsa qayta jo'natadi, tartibni kafolatlaydi, lekin xabar chegaralarini saqlamaydi (stream — pastda bu muhim bo'ladi).

Asosiy rollar:

- **Server:** socket() → bind() → listen() → accept() ketma-ketligi bilan portga “quloq soladi”.

- **Klient:** socket() → connect() bilan serverga ulanadi.

• **3-bosqichli qo'l siqish** (SYN → SYN-ACK → ACK) natijasida TCP sessiya ochiladi.

Tavsiyalar:

- Test uchun HOST='127.0.0.1', PORT=5000 kabi lokal portdan foydalaning.
- Qayta ishga tushirishda "Address already in use" bo'lmasligi uchun SO_REUSEADDR ni yoqing.
- Uzun sessiyalar uchun SO_KEEPALIVE ni yoqish foydali.
- Kechikishni kamaytirish kerak bo'lsa TCP_NODELAY (Nagle off) ni sozlash mumkin.

Minimal server (multi-client, thread bilan):

```
# server.py
import socket, threading, struct

HOST, PORT = '127.0.0.1', 5000

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionResetError("Peer closed")
        buf += chunk
    return buf

def recv_msg(sock):
    hdr = recv_exact(sock, 4) # 4 baytli uzunlik (network
byte order)
    (ln,) = struct.unpack('!I', hdr)
    return recv_exact(sock, ln)

def send_msg(sock, data: bytes):
    sock.sendall(struct.pack('!I', len(data)) + data)

def handle_client(conn, addr):
    conn.settimeout(20) # himoya: muzlab qolmasin
    try:
        while True:
            data = recv_msg(conn) # xabarni o'qiyviz
            if not data:
                break
            # Bu yerda biznes-mantiq: masalan, ECHO (katta harfga o'girish)
            reply = data.upper()
            send_msg(conn, reply)
    except (socket.timeout, ConnectionResetError):
        pass
    finally:
        conn.shutdown(socket.SHUT_RDWR)
```

```

        conn.close()

def main():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
        s.bind((HOST, PORT))
        s.listen(100)
        print(f"Listening on {HOST}:{PORT}")
        while True:
            conn, addr = s.accept()
            threading.Thread(target=handle_client, args=(conn, addr),
daemon=True).start()

if __name__ == "__main__":
    main()

```

Minimal klient:

```

# client.py
import socket, struct

HOST, PORT = '127.0.0.1', 5000

def send_msg(sock, data: bytes):
    sock.sendall(struct.pack('!I', len(data)) + data)

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionResetError("Peer closed")
        buf += chunk
    return buf

def recv_msg(sock):
    ln, = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)

with socket.create_connection((HOST, PORT), timeout=10) as s:
    for text in ["salom", "tcp sinovi", "xayr"]:
        send_msg(s, text.encode('utf-8'))
        print("Server:", recv_msg(s).decode('utf-8'))

```

Ishga tushirish:

1. python server.py
2. boshqa terminalda python client.py

3) Ma'lumotlarni uzatish va qabul qilishni amalda bajarish

TCP **stream** bo'lgani uchun "bir send = bir recv" **emas**. Xabar chegaralarini o'zingiz belgilashingiz kerak. Amaliy usullar:

1. **Uzunlik-prefiks** (yuqoridagi kod): xabar boshiga 4 bayt uzunlik (!l – network byte order) qo'shiladi. Eng ko'p qo'llanadigan, sodda va ishonchli usul.

2. **Delimiter** (masalan, ¥n bilan tugaydigan satrlar): loglar, buyruqlar uchun qulay, lekin kontent ichida delimiter bo'lmashligini kafolatlash kerak.

3. **Protobuf/JSON**: JSON jo'natsangiz ham yuqoridagidek uzunlik bilan jo'natish yaxshi amaliyot (aks holda recv bo'linib ketishi mumkin).

4. **Katta fayllar**: fayl hajmini oldindan jo'nating, so'ngra recv_exact bilan bo'lak-bo'lak qabul qiling; yoki sendfile() / memoryview dan foydalanib samaradorlikni oshiring.

Misol – fayl yuborish (klientdan serverga):

- **Klient:** `header = struct.pack('!l', len(filename_bytes)) + filename_bytes + struct.pack('!Q', file_size)`
- **Server:** avval sarlavhani o'qiydi, so'ng file_size baytni siklda qabul qilib faylga yozadi.

Koddagi muhim punktlar:

- **sendall** ishlating — u butun bufer jo'natilmaguncha qaytmaydi.
- **recv** ni aynan kerakli uzunlikka yetguncha takrorlang (recv_exact).
- **Kodlash:** matnlar uchun utf-8 (yuborishda .encode(), qabulda .decode()).
- **Timeout** qo'ying (settimeout) — "osilib qolish"dan saqlaydi.
- **Shutdown/close** tartibi: avval shutdown(SHUT_RDWR), so'ng close().

4) Klient va server dasturlarida xatoliklarni aniqlash va bartaraf etish

Keng tarqalgan xatoliklar va yechimlar

Muammo / xato	Sabab	Aniqlash	Yechim
ConnectionRefusedError	Server ishlamayapti yoki port bloklangan	Klient ulana olmaydi	Serverni ishga tushiring; IP/port to'g'riligini tekshiring; firewall'ni ruxsat qiling
Address already in use	Port band (oldingi instansiya to'liq yopilgan)	bind() da xato	SO_REUSEADDR; eski jarayonni yoping; portni o'zgartiring
ConnectionResetError / BrokenPipeError	Ikkinchi tomon ulanishni yopdi	O'qish/yozishda istisno	Protokol bo'yicha to'g'ri yopish; qayta urinish (retry) strategiyasi
socket.timeout	Tarmoq sust yoki peer javob bermadi	recv/connect kutishi tugadi	settimeout qiymatini oshiring; qayta urinish; vaqtinchalik xatoni loglab, foydalanuvchiga xabar bering

“Truncated/garbled message”	Framing yo‘q (stream bo‘linib ketdi)	Noto‘liq xabar	Uzunlik-prefiks yoki delimiter bilan framingni joriy qiling
Kodlash xatolari	utf-8/cp1251 aralash	.decode() da xato	Standartni bir xil qiling; binar bilan ishlaganda hech qachon avtomatik .decode() qilmang
Sekin yoki notekis ishlash	Nagle, kichik bo‘laklar, locklar	Kechikish katta	Paketlashni birlashtiring; kerak bo‘lsa TCP_NODELAY; oqimni buferlang

Diagnostika va kuzatuv

- **Loglar:** har xabar uchun `client_id`, uzunlik, vaqt, xatolik kodlarini yozing.
- **Net vositalar:**
 - `netstat -an | find "5000"` (Windows) yoki `ss -tlnp/lsof -i :5000` (Linux) — port holati.
 - `telnet 127.0.0.1 5000` yoki `nc 127.0.0.1 5000` — ochiqligini tekshirish.
 - **Wireshark/tcpdump** — paket darajasida tahlil (SYN/SYN-ACK/ACK, RST, retransmit).
- **Sog‘lomlik signallari:** serverga **heartbeat/ping** RPC qo‘shing.
- **Backpressure:** server band bo‘lsa, mijozga “busy” javobi qaytaring yoki navbatga qo‘ying.

Himoya va barqarorlik

- **Kirishni cheklash:** IP-filtrlash, autentifikatsiya (token).
- **Resurslar:** maksimal ulanishlar, bufer o‘lchamlari, fayl hajmi limitlari.
- **TLS/SSL:** agar internet orqali ishlasa, ssl moduli bilan socketni o‘rab shifrlang.
- **Arxitektura:** ko‘p mijozli yukda `asyncio` (event loop) yoki `worker-pool` tanlang.

Istisnolarni to‘liq qo‘lga olish (shablon):

```
try:
    # tarmoq amali
except (ConnectionResetError, BrokenPipeError) as e:
    logger.warning("Peer disconnected: %s", e)
    # retry yoki sessiyani yumshoq yopish
except socket.timeout:
    logger.warning("Timeout; retry...")
except OSError as e:
    logger.exception("OS error: %s", e)
finally:
    try:
        sock.shutdown(socket.SHUT_RDWR)
    except OSError:
        pass
    sock.close()
```

Qisqa chek-ro‘yxat (cheat-sheet)

- ☐ **Server:** bind → listen → accept; **Klient:** connect.
- ☐ SO_REUSEADDR, SO_KEEPALIVE, kerak bo'lsa TCP_NODELAY.
- ☐ **Framing:** 4 baytli uzunlik prefiksi (struct.pack('!l', len)), **sendall/recv_exact**.
- ☐ utf-8 kodlash; fayllar uchun binar oqim.
- ☐ settimeout; istisnolarni tutish; loglash.
- ☐ **Yopish:** shutdown → close.
- ☐ **Test:** telnet/nc, netstat/ss, Wireshark.

Nazorat savollari:

1. Tarmoq dasturlash deganda nimani tushunasiz?
2. Tarmoqda ishlash uchun python tilining qaysi paketi bilan ishlanadi?
3. Qanday metodlarni bilasiz?
4. Bind() metodining vazifasi qanday?
5. connect metodining vazifasi qanday?
6. Protokol nima?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

2-mashg'ulot. Socket moduli bilan ishlash.

O'quv savollari:

1.Socket modulining asosiy metodlari bilan tanishish.

2.socket(), .bind, .listen, .accept(), .connect(), .send(), recv(), .close() metodlari bilan ishlash.

1. Socket modulining asosiy metodlari bilan tanishish

Soket - bu jarayonlar o'rtasida ma'lumot almashinuvini ta'minlash uchun dasturlash interfeysi.

Socket turlari:

- Server - xabarlarni qabul qiluvchi socket.
- Mijoz - xabarlarni yuboradigan socket.

Socketlar protokollarning transport qatlamida ishlaydi va shunga mos ravishda 2 tur mavjud:

- **Oqim** (TCP asosida, kodda ko'rsatilgan **SOCK_STREAM**) - TCP protokoli asosida o'rnatilgan ulanishga ega socketlar, ikki tomonlama bo'lishi mumkin bo'lgan baytlar oqimini uzatadi - ya'ni Ilova ma'lumotlarni qabul qilishi va yuborishi mumkin.
- **Datagram** (UDP asosida, kodda ko'rsatilgan **SOCK_DGRAM**) - ular o'rtasida aniq ulanishni talab qilmaydigan socketlar. Xabar belgilangan socketga yuboriladi va shunga mos ravishda belgilangan socketdan qabul qilinishi mumkin.

Soket tarkibiasllida IP manzil va portdan iborat bo'ladi.

IP manzil - IP protokoli yordamida qurilgan kompyuter tarmog'idagi tugunning yagona tarmoq manzili. IPv4 protokoli versiyasida IP manzili uzunligi 4 bayt (masalan, 192.168.0.3), IPv6 protokoli versiyasida esa IP manzili 16 bayt (masalan, 2001:0db8:85a3:0000:0000: 8a2e: 0370: 7334). IP-manzil noyob bo'lishi kerak.

Port - transport qatlami protokollari (TCP, UDP va boshqalar) sarlavhalarida yozilgan natural son. Port bir xil xost ichidagi paketni qabul qilish jarayonini aniqlash uchun ishlatiladi.

Python socketlar bilan ishlash uchun o'rnatilgan `socket` kutubxonasidan foydalanadi. Modulning asosiy funksiyalaridan biri bu `socket()` ulanish bilan ishlash uchun tegishli funksiyalarga ega bo'lgan socket tipidagi obyektini qaytaradigan funksiyadir.:

```
class socket.socket
sock = socket.socket()
```

2 .socket(), .bind(), .listen(), .accept(), .connect(), .send(), .recv(), .close() metodlari bilan ishlash

- **socket.bind(address)** - Socketsni manzilga bog'laydi (IP manzil va portni ishga tushiradi). Bundan oldin socket ulanmagan bo'lishi kerak.
- **socket.listen([backlog])** - Ulanishlarni qabul qilish uchun serverni almashtiradi. "backlog (int)" parametri server qabul qiladigan ulanishlar sonidir.
- **socket.accept()** - Ulanishni qabul qiladi va mijozdan xabar kutayotganda dasturni bloklaydi. Natijada kortejni qaytaradi:
 - **conn**: ma'lumotlarni yuborish/qabul qilish uchun ishlatilishi mumkin bo'lgan ulanish obyekti (socket);
 - **address**: mijozning manzili.
- **socket.recv(bufsize[, flags])** - socketdan ma'lumotlarni ikkilik formatda (baytlar to'plami) o'qiydi va qaytaradi. Parametr - bitta xabardagi baytlarning maksimal soni. **bufsize (int)**
- **socket.send(bytes[, flags])** - Mijozga ma'lumotlarni yuboradi va yuborilgan baytlar sonini qaytaradi. Parametr ikkilik ma'lumotlardir. **bytes (bytes)**
- **socket.close()** - Socketsni yopadi.

Sockets bilan ishlash ko'p jihatdan fayl obyekti bilan ishlashga o'xshaydi. Printsip - ochiq ulanish - ma'lumotlar - yopiq ulanish deb hisoblanadi.

Nazorat savollari:

1. port deb nimaga aytiladi?
2. Socket nima ?
3. Socketning qanday turlari mavjud?
4. Socket obyektining qanday funksiyalari mavjud?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

3-mashg'ulot. Pythonda TCP klient-server dasturini tuzish.

O'quv savollari:

1. Socket modulining asosiy metodlari yordamida client–server dasturini tuzish.
2. TCP ulanishini o'rnatish va xabar almashishni sinash (connect/send/recv).
3. Xatoliklarni qayta ishlash va ulanish holatini boshqarish (timeout/reconnect/close).

1. Socket modulining asosiy metodlari yordamida client-server dasturini tuzish.

Python-da o'rnatilgan socket moduli tarmoq orqali muloqot qilish uchun funktsionallikni ta'minlaydi. Ushbu modul socket sinfi shaklida so'rovlarni yuborish va qabul qilish uchun past darajadagi interfeysda aniqlanadi. Va yaratayotgan narsadan qat'iy nazar - server yoki mijoz dasturi, so'rovlarni yuborish uchun socket obyektini yaratish kerak bo'ladi:

```
import socket
sock = socket.socket()
```

socket bilan ishlashni tugatgandan so'ng, uni `close()` metodi yordamida yopish kerak.

```
import socket
client = socket.socket()
client.close()
```

Socket bilan keyingi harakatlar qanday rozetka yaratilayotganiga bog'liq bo'lib, server yoki mijoz uchun yaratiladi. Bunday holda, Python socketlarida eng oddiy mijozni yaratishni ko'rib chiqiladi.

Serverga bog'lanish. `connect()` metodi serverga ulanish uchun ishlatiladi.

```
socket.connect(address)
```

Ushbu **connect** metodi parametr sifatida server manzilini qabul qiladi. Manzil odatda kortej sifatida ifodalanadi

```
(host, port)
```

Birinchi element string sifatida xost hisoblanadi. Bu, masalan, "127.0.0.1" shaklidagi IP-manzil yoki lokal xost nomi bo'lishi mumkin. Ikkinchi parametr raqamli port raqamidir. Port 0 dan 65535 gacha bo'lgan 2 baytlik qiymatdir. Masalan:

```
import socket
client = socket.socket()
client.connect(("www.kun.uz", 80))
print("Connected...")
client.close()
```

Bu yerda birinchi parametr sifatida: "www.kun.uz" manziliga va ikkinchi parametr sifatida 80-portga ulanishga harakat qilmoqda. Ya'ni, `www.kun.uz` 80-portda ishlaydigan oddiy veb-saytga ulanishga harakat qilinmoqda (odatda veb-server 80-

portda ishlaydi). Ulangandan so'ng, shunchaki konsolga satr chiqariladi va socketni yopiladi.

Server bilan o'zaro ulanish.

Ulanish o'rnatilgandan so'ng serverga ma'lumotlarni yuborish va undan qabul qilish mumkin. Ma'lumotlarni yuborish uchun **socket.send()** usuli qo'llaniladi, u parametr sifatida yuboriladigan ma'lumotlar to'plamini oladi.

```
socket.send(bytes)
```

Soketdan ma'lumotlarni qabul qilish uchun **socket.recv()** usuli qo'llaniladi.

```
bytes = socket.recv(bufsize[, flags])
```

Bu zarur parametr sifatida bir vaqtning o'zida boshqa socketdan olinishi mumkin bo'lgan baytlardagi maksimal bufer hajmini oladi. Bufer hajmini 2 ga karrali qilish yaxshidir, masalan, 512, 1024 va boshqalar. Qaytish qiymati boshqa socketdan olingan baytlar to'plamidir.

Masalan, "www.kun.uz" serveriga ma'lumotlarni yuboriladi va undan qaytgan javobni qabul qilinadi:

```
import socket
client = socket.socket()
client.connect(("www.kun.uz", 80))
message = "GET / HTTP/1.1\r\nHost:www.kun.uz\r\n Connection: close\r\n\r\n"
print("Connecting...")
client.send(message.encode())
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Bunday holda, satr veb-sayt tushunadigan standart HTTP protokoli sarlavhalari bilan yuboriladi. "GET/HTTP/1.1\r\nHost: www.kun.uz HTTP so'rov formati birinchi navbatda so'rov turini, so'ralgan manbaga yo'lni va maxsus protokol versiyasidan iborat so'rov qatorini () o'z ichiga oladi . Ya'ni, bu yerda xabarda GET so'rovi "/" yo'li bo'ylab yuborilganligini bildiriladi (ya'ni www.kun.uz saytining ildiziga). Bu holda HTTP / 1.1 protokoli qo'llaniladi. So'rov qatori ikki karek to'plami va qaytariladigan belgilar qatori \r\n bilan tugashi kerak.

Bundan tashqari, HTTP so'rovida sarlavhalar bo'lishi mumkin. Shunday qilib, bu holda "Xost" sarlavhasini yuboriladi, bu esa xost manziliga ishora qiladi. Bu holda bu "www.kun.uz:80". Va shuningdek, bu holatda, "close" qiymatiga ega bo'lgan "connection" sarlavhasini yuboriladi - bu qiymat serverga ulanishni yopishni buyuradi.

Bu yerda satrlarni emas, faqat baytlarni yuborish mumkinligi sababli, uni yuborilganida satrni baytlar to'plamiga aylantiriladi. Buning uchun **bytes** sinf obyektini qaytaradigan **encode()** usuli qo'llaniladi.

```
client.send(message.encode())
```

Serverga xabar yuborilganidan so'ng, recv() funksiyasidan foydalanib ma'lumotlarni olishga harakat qilinadi. Bunday holda, olingan ma'lumotlar uchun bufer 1024 bayt hajmiga ega bo'ladi. Usulning natijasi olingan ma'lumotlardir. Biroq,

socket ma'lumotlarni baytlar to'plami shaklida oladi (aniqrog'i, obyekt bytes). Ularni satrga aylantirish uchun decode() usulidan foydalaning:

```
data = client.recv(1024) # serverdan ma'lumotlarni olish
print("Server sent: ", data.decode())
```

Va agar ushbu dasturni Python-da ishga tushirilsa, quyidagi kabi javob olish mumkin:

```
Connected...
Server sent: HTTP/1.1 301 Moved Permanently
Connection: close
Content-Length: 0
Server: Varnish
Retry-After: 0
Location: https://www.python.org/
Accept-Ranges: bytes
Date: Tue, 02 May 2023 17:52:32 GMT
Via: 1.1 varnish
X-Served-By: cache-bma1653-BMA
X-Cache: HIT
X-Cache-Hits: 0
X-Timer: S1683049953.637569,VS0,VE0
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

Shuningdek, server bizga http sarlavhalarida veb-sayt <https://www.kun.uz/> manziliga ko'chirilganligini ko'rsatadigan qatorni yuboradi.

Server dasturini yaratish

Python-da serverni, shuningdek mijozni yaratish uchun socket sinfi ham ishlatiladi, ammo bu mijoz socketi bilan ishlashdan biroz farq qiladi.

Serverni ulanish

Server kiruvchi ulanishlarni kutib turadi (listening), ularni qandaydir tarzda qayta ishlaydi va javob yuboradi. Server o'z ishini boshlashi uchun, avvalo, u ulanishlarni tinglaydigan manzilni aniqlab olishi kerak. Buning uchun **bind()** metodi qo'llaniladi.

```
socket.bind(address)
```

Bu usul server ishga tushadigan manzilni qabul qiladi. Odatiy bo'lib, manzil ikki elementdan iborat to'plamdir:

```
(host, port)
```

Birinchi element string sifatida xost hisoblanadi. Bu, masalan, "127.0.0.1" shaklidagi IP-manzil yoki mahalliy xost nomi bo'lishi mumkin. Ikkinchi parametr raqamli port raqamidir. Port 0 dan 65535 gacha bo'lgan 2 baytlik qiymatni ifodalaydi. Bir xil manzilda (bir xil mashinada) bir nechta turli tarmoq ilovalari ishga tushirilishi mumkinligi sababli, port ushbu ilovalarni farqlash imkonini beradi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
```

```
server.bind((hostname, port))
```

Bu yerda server 12345-portda ishlaydi. E'tibor bering, barcha portlar ham bepul bo'lmashligi mumkin. Ammo, qoida tariqasida, band bo'lgan portlar unchalik ko'p emas.

Manzil sifatida joriy xost nomidan foydalanamiz. Uni olish uchun funksiyadan foydalaniladi **socket.gethostname()** (odatda bu joriy kompyuterning nomi).

Ulanishlarni tinglash

Serverni bog'lagandan so'ng ulanishlarni tinglash uchun uni ishga tushirish kerak. Buning uchun **listen()** metodi qo'llaniladi.

```
socket.listen([backlog])
```

Bu backlog parametrini qabul qiladi. backlog – bu socket uchun ruxsat etilgan navbatdagi kiruvchi ulanishlarning maksimal soni. Ya'ni, mijozlar ulanganda, ular navbatda turishadi va server joriy mijozni qayta ishlaguncha kutishadi. Agar navbatda server tomonidan qayta ishlanishini kutayotgan mijozlar soni allaqachon belgilangan bo'lsa, barcha yangi mijozlar rad etiladi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)
```

Mijozni qabul qilish va qayta ishlash

accept() metodi kiruvchi ulanishlarni qabul qilish uchun ishlatiladi. Bu usul 2 ta tuple tipli qiymatni qaytaradi

```
(conn, address)
```

Birinchi element conn - server mijoz bilan aloqa qiladigan boshqa socket obyektini ifodalaydi. Ikkinchi element address - ulangan mijozning manzili hisoblanadi. Shuni ta'kidlash kerakki, mijoz bilan o'zaro aloqalar tugagandan so'ng, conn socketini close() usuli bilan yopish kerak bo'ladi.

Tuplening birinchi elementi - conn yordamida mijozga xabar yuborish yoki aksincha ma'lumotlarni qabul qilish mumkin. Soketdan ma'lumotlarni qabul qilish uchun **socket.recv()** usuli qo'llaniladi.

```
bytes = socket.recv(bufsize)
```

Bu zarur parametr sifatida bir vaqtning o'zida boshqa socketdan olinishi mumkin bo'lgan baytlardagi maksimal bufer hajmini oladi. Qaytish qiymati boshqa socketdan olingan baytlar to'plamidir.

Ma'lumotlarni yuborish uchun **socket.send()** metodi qo'llaniladi, u parametr sifatida yuboriladigan ma'lumotlar to'plamini oladi.

```
socket.send(bytes)
```

Endi bir misolni ko'rib chiqaylik. Server.py faylida serverni quyidagi kod bilan aniqlaylik:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server starts")

con, addr = server.accept()
print("connection: ", conn)
print("client address: ", addr)

message = "Hello Client!"
con.send(message.encode())
con.close()
print("Server ends")
server.close()
```

Bu yerda server mijozni qabul qiladi, uning ulanishi haqidagi ma'lumotlarni chiqaradi va mijozga "Hello Client!" qatorini qaytarib yuboradi.

Va client.py faylida quyidagi mijoz kodini aniqlanadi:

```
import socket
client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Bizning serverimiz va mijozimiz bitta kompyuterda ishlaganligi sababli, **socket.gethostname()** server kodidagi kabi ulanish uchun server manzilini aniqlash uchun funksiya va port 12345 ishlatiladi. Serverga ulangandan so'ng undan ma'lumotlarni olinadi va uni konsolda ko'rsatadi.

Avval server kodini ishga tushiraylik. Va keyin mijoz kodini ishga tushiriladi. Natijada, server ulanishni qabul qiladi, u haqidagi ma'lumotlarni konsolda ko'rsatadi va mijozga xabar yuboradi:

```
c:\python>python server.py
Server starts
connection: <socket.socket fd=432, family=2, type=1, proto=0,
laddr=('192.168.0.102', 12345), raddr=('192.168.0.102', 61824)>
client address: ('192.168.0.102', 61824)
Server ends
```

Xususan, bu yerda server va mijoz 192.168.0.102 da ishlayotganini va mijoz 61824 portidan foydalanayotganini ko'ramiz.

Va mijoz serverdan xabar oladi va uni konsolga chop etadi:

```
c:\python>python client.py
Server sent: Hello Client!
```

Bir nechta mijozlar bilan ishlash

Bunday holda, server bitta mijozga xizmat qiladi va ishlashni to'xtatadi. Agar biz server ko'p mijozlar bilan ishlashni istasak, unda cheksiz sikldan foydalanishga to'g'ri keladi:

```
import socket
from datetime import datetime

server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server running")
while True:
    con, addr = server.accept()
    print("client address: ", addr)
    message = datetime.now().strftime("%H:%M:%S")
    con.send(message.encode())
    con.close()
```

Bunday holda, masalan, o'rnatilgan modul `datetime` va `datetime.now().strftime()` funksiyasidan foydalanib, joriy vaqtni satr sifatida olish, keyinchalik uni mijozga yuborish mumkin. Natijada, mijoz so'rov bilan joriy vaqthaqidagi ma'lumotlarni ham oladi.

Ikki tomonlama aloqa

Yuqoridagi misollarda aloqa bir tomonlama edi - server ma'lumotlarni yuboradi va mijoz uni qabul qiladi. Keling, mijoz ham, server ham ma'lumotlarni jo'natgan va qabul qilganda eng oddiy ikki tomonlama aloqani ko'rib chiqaylik. Serverga mijozdan bir nechta satr olishiga ruxsat beraylik, uni o'zgartirib so'ngra mijozga qaytarib yuboraylik:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server running")
while True:
    con, _ = server.accept()
    data = con.recv(1024)
    message = data.decode()
    print(f"Client sent: {message}")
    message = message[::-1]
```

```
con.send(message.encode())
con.close()
```

Mijoz konsoldan satr kiritish va uni serverga yuborish uchun kodni aniqlasin:

```
import socket

client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
message = input("Input a text: ")
client.send(message.encode())
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Yuqoridagi ishning natijasi:

Client:

```
c:\python>python client.py
Input a text: hello
Server sent: olleh
c:\python>
```

Server:

```
c:\python>python server.py
Server running
Client sent: hello
```

2) TCP ulanishini o'rnatish va xabar almashishni sinash (connect/send/recv)

Minimal server (echo)

```
# server.py
import socket

HOST, PORT = '127.0.0.1', 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen(1)
    print(f"Listening on {HOST}:{PORT}")
    conn, addr = s.accept()
    with conn:
        print("Client:", addr)
        while True:
            data = conn.recv(4096)      # 0 => peer yopdi
            if not data:
                break
            conn.sendall(data)          # echo
```

Minimal klient

```
# client.py
```

```
import socket

HOST, PORT = '127.0.0.1', 5000
msg = b"salom, tcp!"

with socket.create_connection((HOST, PORT), timeout=5) as s:
    s.sendall(msg)                # barcha baytlar jo'natiladi
    reply = s.recv(4096)          # javobni o'qish
    print("Server javobi:", reply.decode('utf-8'))
```

Test: birinchi terminalda python server.py, ikkinchisida python client.py.

Eslatma: TCP **stream**; bitta send = bitta recv bo'lishi **shart emas**. Barqaror format uchun **framing** kiriting (quyida).

Satrli protokol uchun framing (uzunlik-prefiks)

```
import struct

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return buf

def recv_msg(sock) -> bytes:
    (ln,) = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)
```

Bu yordamchilarni server/klientga qo'shsangiz, xabarlar bo'linib ketmaydi.

3) Xatoliklarni qayta ishlash va ulanish holatini boshqarish (timeout/reconnect/close)

Mustahkam klient shabloni

```
import socket, time, struct, random

HOST, PORT = '127.0.0.1', 5000
BASE, CAP = 0.5, 8.0 # backoff (sekund)

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
```

```

        buf += chunk
    return buf

def recv_msg(sock):
    ln, = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)

def connect_with_retry():
    delay = BASE
    while True:
        try:
            sock = socket.create_connection((HOST, PORT), timeout=5)
            sock.settimeout(10) # I/O timeout
            return sock
        except (ConnectionRefusedError, TimeoutError, OSError) as e:
            print("Ulanmadi:", e)
            time.sleep(delay + random.random()*0.2) # jitter
            delay = min(delay*2, CAP) # eksponent backoff

def run_client():
    s = connect_with_retry()
    try:
        for text in ["ping", "hello", "bye"]:
            try:
                send_msg(s, text.encode())
                reply = recv_msg(s).decode()
                print("reply:", reply)
            except (socket.timeout, ConnectionError, OSError) as e:
                print("I/O xato:", e, "→ qayta ulanish")
                try:
                    s.shutdown(socket.SHUT_RDWR)
                except OSError:
                    pass
                s.close()
                s = connect_with_retry() # avtomatik reconnect
    finally:
        try:
            s.shutdown(socket.SHUT_RDWR) # muloyim yopish
        except OSError:
            pass
        s.close()

if __name__ == "__main__":
    run_client()

```

Amaliy qoidalar (muxtasar)

- **Timeoutlar:**

- `create_connection(..., timeout=5)` — ulanish limiti.
- `sock.settimeout(10)` — har bir send/recv uchun I/O limiti.

- **Reconnect:** xatoda **eksponent backoff + jitter** qo'llang; cheksiz sikl yoki maksimal urinish soni.
- **Holatni tekshirish:**
 - `recv()` **0 bayt** qaytarsa — peer socketni yopgan.
 - `socket.timeout` — tarmoq sust; qayta urinish yoki foydalanuvchiga xabar.
 - `ConnectionResetError/BrokenPipeError` — qarshi tomon to'satdan uzdi.
- **Yopish tartibi:** `shutdown(SHUT_RDWR)` → `close()` (har doim `try/except` bilan).
- **SO_REUSEADDR/KEEPALIVE:** serverda `SO_REUSEADDR`; uzoq sessiyalar uchun `SO_KEEPALIVE`.
- **Framing:** uzunlik-prefiks yoki delimiter (¥n) — **majburiy**; aks holda xabarlar bo'linadi/qo'shiladi.
- **Loglash:** ulanish holati, xatolik turlari, qayta urinishlar sonini yozib boring.

Tezkor diagnostika

- Port tinglayaptimi?
 - **Linux:** `ss -tlnp | grep 5000` yoki `lsof -i :5000`
 - **Win:** `netstat -ano | find "5000"`
- Soddashtirilgan test: `nc -l 5000` (server o'rnida) yoki `nc 127.0.0.1 5000` (klient o'rnida).
- Paket darajasi: Wireshark — SYN/SYN-ACK/ACK, RST, retransmitlarni ko'ring.

Nazorat savollari:

1. Server bilan aloqani qanday amalga oshiriladi?
2. `socket.bind()` metodi nima amalni bajaradi?
3. `socket.gethostname()` metodi qanday vazifani bajaradi?
4. `listen()` metodining ishlash tamoyili qanday?
5. `socket.recv()` metodidan nima maqsadda foydalaniladi?
6. Socket nima?
7. `decode()` funksiyasi nima maqsadda foydalaniladi?
8. Serverga ma'lumot jo'natish qanday amalga oshiriladi?
9. Serverdan ma'lumotni qanday qabul qilin olinadi?
10. `Connect()` metodi qanday parametrlarni qabul qiladi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

4-mashg'ulot. TCP klient-server dasturini testlash.

O'quv savollari:

1. TCP client-server dasturi yordamida ma'lumot almashish.
2. TCP ulanishini o'rnatish va testlash.
3. TCP client va serverdagi xatoliklarni aniqlash va bartaraf etish.

1. TCP client-server dasturi yordamida ma'lumot almashish.

Serverni ulanish

Server kiruvchi ulanishlarni kutib turadi (listening), ularni qandaydir tarzda qayta ishlaydi va javob yuboradi. Server o'z ishini boshlashi uchun, avvalo, u ulanishlarni tinglaydigan manzilni aniqlab olishi kerak. Buning uchun **bind()** metodi qo'llaniladi.

```
socket.bind(address)
```

Bu usul server ishga tushadigan manzilni qabul qiladi. Odatiy bo'lib, manzil ikki elementdan iborat to'plamdir:

```
(host, port)
```

Birinchi element string sifatida xost hisoblanadi. Bu, masalan, "127.0.0.1" shaklidagi IP-manzil yoki mahalliy xost nomi bo'lishi mumkin. Ikkinchi parametr raqamli port raqamidir. Port 0 dan 65535 gacha bo'lgan 2 baytlik qiymatni ifodalaydi. Bir xil manzilda (bir xil mashinada) bir nechta turli tarmoq ilovalari ishga tushirilishi mumkinligi sababli, port ushbu ilovalarni farqlash imkonini beradi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
```

Bu yerda server 12345-portda ishlaydi. E'tibor bering, barcha portlar ham bepul bo'lmasligi mumkin. Ammo, qoida tariqasida, band bo'lgan portlar unchalik ko'p emas.

Manzil sifatida joriy xost nomidan foydalanamiz. Uni olish uchun funksiyadan foydalaniladi **socket.gethostname()** (odatda bu joriy kompyuterning nomi).

Ulanishlarni tinglash

Serverni bog'lagandan so'ng ulanishlarni tinglash uchun uni ishga tushirish kerak. Buning uchun **listen()** metodi qo'llaniladi.

```
socket.listen([backlog])
```

Bu backlog parametrini qabul qiladi. backlog – bu socket uchun ruxsat etilgan navbatdagi kiruvchi ulanishlarning maksimal soni. Ya'ni, mijozlar ulanganda, ular navbatda turishadi va server joriy mijozni qayta ishlaguncha kutishadi. Agar navbatda

server tomonidan qayta ishlanishini kutayotgan mijozlar soni allaqachon belgilangan bo'lsa, barcha yangi mijozlar rad etiladi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)
```

Mijozni qabul qilish va qayta ishlash

accept() metodi kiruvchi ulanishlarni qabul qilish uchun ishlatiladi. Bu usul 2 ta tuple tipli qiymatni qaytaradi

```
(conn, address)
```

Birinchi element conn - server mijoz bilan aloqa qiladigan boshqa socket obyektini ifodalaydi. Ikkinchi element address - ulangan mijozning manzili hisoblanadi. Shuni ta'kidlash kerakki, mijoz bilan o'zaro aloqalar tugagandan so'ng, conn socketini close() usuli bilan yopish kerak bo'ladi.

Tuplening birinchi elementi - conn yordamida mijozga xabar yuborish yoki aksincha ma'lumotlarni qabul qilish mumkin. Socketdan ma'lumotlarni qabul qilish uchun **socket.recv()** usuli qo'llaniladi.

```
bytes = socket.recv(bufsize)
```

Bu zarur parametr sifatida bir vaqtning o'zida boshqa socketdan olinishi mumkin bo'lgan baytlardagi maksimal bufer hajmini oladi. Qaytish qiymati boshqa socketdan olingan baytlar to'plamidir.

Ma'lumotlarni yuborish uchun **socket.send()** metodi qo'llaniladi, u parametr sifatida yuboriladigan ma'lumotlar to'plamini oladi.

```
socket.send(bytes)
```

Endi bir misolni ko'rib chiqaylik. Server.py faylida serverni quyidagi kod bilan aniqlaylik:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server starts")

con, addr = server.accept()
print("connection: ", con)
print("client address: ", addr)

message = "Hello Client!"
con.send(message.encode())
con.close()
print("Server ends")
```

```
server.close()
```

Bu yerda server mijozni qabul qiladi, uning ulanishi haqidagi ma'lumotlarni chiqaradi va mijozga "Hello Client!" qatorini qaytarib yuboradi.

Va client.py faylida quyidagi mijoz kodini aniqlanadi:

```
import socket
client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Bizning serverimiz va mijozimiz bitta kompyuterda ishlaganligi sababli, **socket.gethostname()** server kodidagi kabi ulanish uchun server manzilini aniqlash uchun funksiya va port 12345 ishlatiladi. Serverga ulangandan so'ng undan ma'lumotlarni olinadi va uni konsolda ko'rsatadi.

Avval server kodini ishga tushiraylik. Va keyin mijoz kodini ishga tushiriladi. Natijada, server ulanishni qabul qiladi, u haqidagi ma'lumotlarni konsolda ko'rsatadi va mijozga xabar yuboradi:

```
c:\python>python server.py
Server starts
connection: <socket.socket fd=432, family=2, type=1, proto=0,
laddr=('192.168.0.102', 12345), raddr=('192.168.0.102', 61824)>
client address: ('192.168.0.102', 61824)
Server ends
```

Xususan, bu yerda server va mijoz 192.168.0.102 da ishlayotganini va mijoz 61824 portidan foydalanayotganini ko'ramiz.

Va mijoz serverdan xabar oladi va uni konsolga chop etadi:

```
c:\python>python client.py
Server sent: Hello Client!
```

Bir nechta mijozlar bilan ishlash

Bunday holda, server bitta mijozga xizmat qiladi va ishlashni to'xtatadi. Agar biz server ko'p mijozlar bilan ishlashni istasak, unda cheksiz sikldan foydalanishga to'g'ri keladi:

```
import socket
from datetime import datetime

server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server running")
```

```

while True:
    con, addr = server.accept()
    print("client address: ", addr)
    message = datetime.now().strftime("%H:%M:%S")
    con.send(message.encode())
    con.close()

```

Bunday holda, masalan, o'rnatilgan modul `datetime` va `datetime.now().strftime()` funksiyasidan foydalanib, joriy vaqtni satr sifatida olish, keyinchalik uni mijozga yuborish mumkin. Natijada, mijoz so'rov bilan joriy vaqthaqidagi ma'lumotlarni ham oladi.

Ikki tomonlama aloqa

Yuqoridagi misollarda aloqa bir tomonlama edi - server ma'lumotlarni yuboradi va mijoz uni qabul qiladi. Keling, mijoz ham, server ham ma'lumotlarni jo'natgan va qabul qilganda eng oddiy ikki tomonlama aloqani ko'rib chiqaylik. Serverga mijozdan bir nechta satr olishiga ruxsat beraylik, uni o'zgartirib so'ngra mijozga qaytarib yuboraylik:

```

import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server running")
while True:
    con, _ = server.accept()
    data = con.recv(1024)
    message = data.decode()
    print(f"Client sent: {message}")
    message = message[:-1]
    con.send(message.encode())
    con.close()

```

Mijoz konsoldan satr kiritish va uni serverga yuborish uchun kodni aniqlasin:

```

import socket

client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
message = input("Input a text: ")
client.send(message.encode())
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()

```

Yuqoridagi ishning natijasi:

Client:

```
c:\python>python client.py
```

```
Input a text: hello
Server sent: olleh
c:\python>
```

Server:

```
c:\python>python server.py
Server running
Client sent: hello
```

2) TCP ulanishini o'rnatish va testlash**Minimal ECHO server**

```
# server.py
import socket

HOST, PORT = '127.0.0.1', 5000
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen(1)
    print(f"Listening on {HOST}:{PORT}")
    conn, addr = s.accept()
    with conn:
        print("Client:", addr)
        while True:
            data = conn.recv(4096)          # 0 bayt → mijoz yopdi
            if not data:
                break
            conn.sendall(data)              # echo
```

Minimal klient

```
# client.py
import socket

HOST, PORT = '127.0.0.1', 5000
with socket.create_connection((HOST, PORT), timeout=5) as s:
    s.sendall(b"salom, tcp!")
    print("Server javobi:", s.recv(4096).decode('utf-8'))
```

Testlash:

1. python server.py (server terminalida)
2. python client.py (boshqa terminalda)
3. Agar “salom, tcp!” qaytsa — ulanish OK.

TCP **stream**: bitta send = bitta recv bo'lishi **shart emas**. Barqaror xabar almashuv uchun framing qo'llang (quyida).

Framing: uzunlik-prefiks (tavsiya etiladi)

```
import struct

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)
```

```
def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return buf

def recv_msg(sock):
    ln, = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)
```

Shu yordamchilarni server/klientga qo'shsangiz, xabarlar bo'linmaydi va qo'shilib ketmaydi.

3) TCP client va serverdagi xatoliklarni aniqlash va bartaraf etish Mustahkam klient shabloni (timeout + reconnect)

```
import socket, time, random, struct

HOST, PORT = '127.0.0.1', 5000
BASE, CAP = 0.5, 8.0 # reconnect uchun eksponent backoff

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return buf

def recv_msg(sock):
    ln, = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)

def connect_with_retry():
    delay = BASE
    while True:
        try:
            s = socket.create_connection((HOST, PORT), timeout=5)
            s.settimeout(10) # har bir I/O uchun timeout
            return s
        except (ConnectionRefusedError, TimeoutError, OSError) as e:
            print("Ulanmadi:", e)
            time.sleep(delay + random.random()*0.2) # jitter
            delay = min(delay*2, CAP)
```

```

def run_client():
    s = connect_with_retry()
    try:
        for text in ["ping", "hello", "bye"]:
            try:
                send_msg(s, text.encode('utf-8'))
                print("reply:", recv_msg(s).decode('utf-8'))
            except (socket.timeout, ConnectionError, OSError) as e:
                print("I/O xato:", e, "→ reconnect")
            try: s.shutdown(socket.SHUT_RDWR)
            except OSError: pass
            s.close()
            s = connect_with_retry()
    finally:
        try: s.shutdown(socket.SHUT_RDWR)
        except OSError: pass
        s.close()

if __name__ == "__main__":
    run_client()

```

Serverda ko'p mijozni boshqarish (thread bilan)

```

# qisqa shablon
import socket, threading, struct
HOST, PORT = '127.0.0.1', 5000

def recv_exact(s, n):
    buf=b''
    while len(buf)<n:
        c=s.recv(n-len(buf))
        if not c: raise ConnectionError("peer closed")
        buf+=c
    return buf

def recv_msg(s):
    import struct
    ln,=struct.unpack('!I', recv_exact(s,4))
    return recv_exact(s, ln)

def send_msg(s, b):
    s.sendall(struct.pack('!I', len(b)) + b)

def handle(conn, addr):
    conn.settimeout(20)
    try:
        while True:
            data = recv_msg(conn)
            send_msg(conn, data.upper()) # misol: ECHO
    except (socket.timeout, ConnectionError, OSError):
        pass
    finally:
        try: conn.shutdown(socket.SHUT_RDWR)

```



```

except OSError: pass
conn.close()

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((HOST, PORT)); srv.listen(100)
    print("Listening...")
    while True:
        c, a = srv.accept()
        threading.Thread(target=handle, args=(c, a), daemon=True).start()

```

Tez-tez uchraydigan xatolar va yechimlar

Xato/holat	Sabab	Bartaraf etish
ConnectionRefusedError	Server ishlamaydi, IP/port xato, firewall	Serverni ishga tushiring; IP/portni tekshiring; ruxsat bering
Address already in use	Port band (oldingi jarayon yopilgan)	SO_REUSEADDR yoqing; portni almashtiring; eski jarayonni yoping
socket.timeout	Ulanish/I/O sust	settimeoutni moslang; qayta urinish strategiyasi
ConnectionResetError / BrokenPipeError	Peer to'satdan uzdi	Protokol bo'yicha yopish (shutdown→close); reconnect
Noto'liq yoki qo'shilib ketgan xabarlar	Framing yo'q	Uzunlik-prefiks yoki delimiter joriy qiling
Kodlash muammolari	UTF-8/CP1251 aralash	Har ikki tomonda bir xil kodlash; binar oqimda .decode() qilmaslik

Diagnostika va monitoring

- Port tinglayaptimi?
 - Linux: `ss -tlnp | grep 5000 / lsof -i :5000`
 - Windows: `netstat -ano | find "5000"`
- Soddashtirilgan test: `nc 127.0.0.1 5000` yoki `telnet`.
- Paket darajasi: **Wireshark** — SYN/SYN-ACK/ACK, RST, retransmitlarni ko'ring.
- Loglar: ulanish ID, xabar uzunligi, vaqt, xatolik kodlari.

Nazorat savollari:

1. Server bilan aloqani qanday amalga oshiriladi?
2. `socket.bind()` metodi nima amalni bajaradi?
3. `socket.gethostname()` metodi qanday vazifani bajaradi?
4. `listen()` metodining ishlash tamoyili qanday?
5. `socket.recv()` metodidan nima maqsadda foydalaniladi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

5-mashg'ulot. PyQt5 paketidan foydalanib zamonaviy chat dasturini tuzish.

O'quv savollari:

1. PyQt5 orqali TCP client uchun GUI yaratish.
2. Socket ulanishini boshqarish (connect/send/receive) va xabar almashish.
3. Signal–slot mexanizmi va xatoliklarni GUI'da ko'rsatish.

TCP dasturini client qismini GUI ko'rinishda ishlab chiqish.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Tue Jul 24 13:05:46 2018

@author: JC
"""

import socket
import sys
import threading
import time
import functools
from PyQt5 import QtCore, QtGui
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QMainWindow, QApplication, QWidget, QPushButton
from PyQt5.QtWidgets import QVBoxLayout, QHBoxLayout, QMessageBox,
QTabWidget
from PyQt5.QtWidgets import QGridLayout, QScrollArea, QLabel, QListView
from PyQt5.QtWidgets import QLineEdit, QComboBox, QGroupBox, QAction
from PyQt5.QtGui import QStandardItemModel, QStandardItem, QFont

class MyTableWidget(QWidget):
    def __init__(self, parent):
        super(QWidget, self).__init__(parent)
        #conecion
        self.conn = socket.socket()
        self.connected = False
        #tab UI
        self.layout = QVBoxLayout(self)
        self.tabs = QTabWidget()
        self.tabs.resize(300,200)
        self.tab1 = QWidget()
        self.tab2 = QWidget()
        self.tabs.addTab(self.tab1, "Home")
        self.tabs.addTab(self.tab2, "Chat Room")
        self.tabs.setTabEnabled(1,False)
        #<Home>
        gridHome = QGridLayout()
        self.tab1.setLayout(gridHome)
```

```

self.IPBox = QGroupBox("IP")
self.IPLineEdit = QLineEdit()
self.IPLineEdit.setText("127.0.0.1")
IPBoxLayout = QVBoxLayout()
IPBoxLayout.addWidget(self.IPLineEdit)
self.IPBox.setLayout(IPBoxLayout)
self.portBox = QGroupBox("port")
self.portLineEdit = QLineEdit()
self.portLineEdit.setText("33002")
portBoxLayout = QVBoxLayout()
portBoxLayout.addWidget(self.portLineEdit)
self.portBox.setLayout(portBoxLayout)
self.nameBox = QGroupBox("Name")
self.nameLineEdit = QtWidgets.QLineEdit()
nameBoxLayout = QVBoxLayout()
nameBoxLayout.addWidget(self.nameLineEdit)
self.nameBox.setLayout(nameBoxLayout)
self.connStatus = QLabel("Status", self)
font = QFont()
font.setPointSize(16)
self.connStatus.setFont(font)
self.connBtn = QPushButton("Connect")
self.connBtn.clicked.connect(self.connect_server)
self.disconnBtn = QPushButton("Disconnect")
self.disconnBtn.clicked.connect(self.disconnect_server)
gridHome.addWidget(self.IPBox,0,0,1,1)
gridHome.addWidget(self.portBox,0,1,1,1)
gridHome.addWidget(self.nameBox,1,0,1,1)
gridHome.addWidget(self.connStatus,1,1,1,1)
gridHome.addWidget(self.connBtn,2,0,1,1)
gridHome.addWidget(self.disconnBtn,2,1,1,1)
gridHome.setColumnStretch(0, 1)
gridHome.setColumnStretch(1, 1)
gridHome.setRowStretch(0, 0)
gridHome.setRowStretch(1, 0)
gridHome.setRowStretch(2, 9)
#</Home>
#<Chat Room>
gridChatRoom = QGridLayout()
self.tab2.setLayout(gridChatRoom)
    self.messageRecords = QLabel("<font color='#000000'>Welcome to
chat room</font>", self)
    self.messageRecords.setStyleSheet("background-color: white;");
    self.messageRecords.setAlignment(QtCore.Qt.AlignTop)
    self.messageRecords.setAutoFillBackground(True);
    self.scrollRecords = QScrollArea()
    self.scrollRecords.setWidget(self.messageRecords)
    self.scrollRecords.setWidgetResizable(True)
    self.sendTo = "ALL"
    self.sendChoice = QLabel("Send to :ALL", self)
    self.sendComboBox = QComboBox(self)

```

```

self.sendComboBox.addItem("ALL")
self.sendComboBox.activated[str].connect(self.send_choice)
self.lineEdit = QLineEdit()
self.lineEnterBtn = QPushButton("Enter")
self.lineEnterBtn.clicked.connect(self.enter_line)
self.lineEdit.returnPressed.connect(self.enter_line)
self.friendList = QListView()
self.friendList.setWindowTitle('Room List')
self.model = QStandardItemModel(self.friendList)
self.friendList.setModel(self.model)
self.emojiBox = QGroupBox("Emoji")
self.emojiBtn1 = QPushButton("😊👉👈")
    self.emojiBtn1.clicked.connect(funcutils.partial(self.send_emoji,
"😊👉👈"))
    self.emojiBtn2 = QPushButton("(👉👈)")
self.emojiBtn2.clicked.connect(funcutils.partial(self.send_emoji, "(
👉👈)"))
    self.emojiBtn3 = QPushButton("( ~·3·)")
self.emojiBtn3.clicked.connect(funcutils.partial(self.send_emoji, "(
~·3·)"))
    self.emojiBtn4 = QPushButton("👉(ò_ó~)👈")
    self.emojiBtn4.clicked.connect(funcutils.partial(self.send_emoji,
"👉(ò_ó~)👈"))
    emojiLayout = QHBoxLayout()
    emojiLayout.addWidget(self.emojiBtn1)
    emojiLayout.addWidget(self.emojiBtn2)
    emojiLayout.addWidget(self.emojiBtn3)
    emojiLayout.addWidget(self.emojiBtn4)
    self.emojiBox.setLayout(emojiLayout)
    gridChatRoom.addWidget(self.scrollRecords,0,0,1,3)
    gridChatRoom.addWidget(self.friendList,0,3,1,1)
    gridChatRoom.addWidget(self.sendComboBox,1,0,1,1)
    gridChatRoom.addWidget(self.sendChoice,1,2,1,1)
    gridChatRoom.addWidget(self.lineEdit,2,0,1,3)
    gridChatRoom.addWidget(self.lineEnterBtn,2,3,1,1)
    gridChatRoom.addWidget(self.emojiBox,3,0,1,4)
    gridChatRoom.setColumnStretch(0, 9)
    gridChatRoom.setColumnStretch(1, 9)
    gridChatRoom.setColumnStretch(2, 9)
    gridChatRoom.setColumnStretch(3, 1)
    gridChatRoom.setRowStretch(0, 9)
    #</Chat Room>
    #Initialization
    self.layout.addWidget(self.tabs)
    self.setLayout(self.layout)

def enter_line(self):
    #assure the person still in room before send out
    if self.sendTo != self.sendComboBox.currentText():

```

```

        self.message_display_append("The person left. Private message
not delivered")
        self.lineEdit.clear()
        return
    line = self.lineEdit.text()
    if line == "":#prevent empty message
        return
    if self.sendTo != "ALL":#private message, send to myself first
        #this is a trick Leverage the server sending back a copy to
myself

        send_msg = bytes("{}+self.userName+"}"+line, "utf-8")
        self.conn.send(send_msg)
        time.sleep(0.1) #this is important for not overlapping two
sending

        send_msg = bytes("{}+self.sendTo+"}"+line, "utf-8")
        self.conn.send(send_msg)
        self.lineEdit.clear()
        self.scrollRecords.verticalScrollBar().setValue(self.scrollRecords.
verticalScrollBar().maximum())

    def send_emoji(self, emoji):
        #assure the person still in rooom before send out
        if self.sendTo != self.sendComboBox.currentText():
            self.message_display_append("The person left. Private message
not delivered")
            return
        if self.sendTo != "ALL":#private message, send to myself first
            #this is a trick Leverage the server sending back a copy to
myself

            send_msg = bytes("{}+self.userName+"}"+emoji, "utf-8")
            self.conn.send(send_msg)
            time.sleep(0.1) #this is important for not overlapping two
sending

            send_msg = bytes("{}+self.sendTo+"}"+emoji, "utf-8")
            self.conn.send(send_msg)

    def message_display_append(self, newMessage, textColor = "#000000"):
        oldText = self.messageRecords.text()
        appendText = oldText+"<br /><font
color=\""+textColor+"\">"+newMessage+"</font><font color=\""#000000\"></font>"
        self.messageRecords.setText(appendText)
        time.sleep(0.2) #this helps the bar set to bottom, after all message
already appended
        self.scrollRecords.verticalScrollBar().setValue(self.scrollRecords.
verticalScrollBar().maximum())

    def updateRoom(self):
        while self.connected:
            data = self.conn.recv(1024)
            data = data.decode("utf-8")

```

```

        print(data)
        if data != "":
            if "{CLIENTS}" in data:
                welcome = data.split("{CLIENTS}")
                self.update_send_to_list(welcome[1])
                self.update_room_list(welcome[1])
                if not welcome[0][5:] == "":
                    self.message_display_append(welcome[0][5:])
                    self.scrollRecords.verticalScrollBar().setValue(self
f.scrollRecords.verticalScrollBar().maximum())
                elif data[:5] == "{MSG}": #{MSG} includes broadcast and
server msg
                    self.message_display_append(data[5:], "#006600")
                    self.scrollRecords.verticalScrollBar().setValue(self.sc
rollRecords.verticalScrollBar().maximum())
                else: #private message is NONE format
                    self.message_display_append("{private}"+data, "#cc33cc")
                    self.scrollRecords.verticalScrollBar().setValue(self.sc
rollRecords.verticalScrollBar().maximum())
                time.sleep(0.1) #this is for saving thread cycle time

    def connect_server(self):
        if self.connected == True:
            return
        name = self.nameLineEdit.text()
        if name == "":
            self.connStatus.setText("Status :"+"Please enter your name")
            return
        self.userName = name
        IP = self.IPLineEdit.text()
        if IP == "":
            IP = "127.0.0.1"
        port = self.portLineEdit.text()
        if port == "" or not port.isnumeric():
            self.portLineEdit.setText("33002")
            self.connStatus.setText("Status :"+"Port format invalid")
            return
        else:
            port = int(port)
        try:
            self.conn.connect((IP, port))
        except:
            self.connStatus.setText("Status :"+" Refused")
            self.conn = socket.socket()
            return
        send_msg = bytes("{REGISTER}"+name, "utf-8")
        self.conn.send(send_msg)
        self.connected = True
        self.connStatus.setText("Status :"+" Connected")
        self.nameLineEdit.setReadOnly(True) #This setting is not functional

```

well

```

self.tabs.setTabEnabled(1,True)
self.rT = threading.Thread(target= self.updateRoom)
self.rT.start()

def disconnect_server(self):
    if self.connected == False:
        return
    send_msg = bytes("{QUIT}", "utf-8")
    self.conn.send(send_msg)
    self.connStatus.setText("Status : "+" Disconnected")
    self.nameLineEdit.setReadOnly(False)
    self.nameLineEdit.clear()
    self.tabs.setTabEnabled(1,False)
    self.connected = False
    self.rT.join()
    self.conn.close()
    self.conn = socket.socket()

def update_room_list(self, strList):
    L = strList.split("|")
    self.model.clear()
    for person in L:
        item = QStandardItem(person)
        item.setCheckable(False)
        self.model.appendRow(item)

def update_send_to_list(self, strList):
    L = strList.split("|")
    self.sendComboBox.clear()
    self.sendComboBox.addItem("ALL")
    for person in L:
        if person != self.userName:
            self.sendComboBox.addItem(person)
    previous = self.sendTo
    index = self.sendComboBox.findText(previous)
    print("previous choice:",index)
    if index != -1:
        self.sendComboBox.setCurrentIndex(index) #updating, maintain
receiver
    else:
        self.sendComboBox.setCurrentIndex(0) #updating, the receiver
left, deafault to "ALL"

def send_choice(self,text):
    self.sendTo = text
    print(self.sendTo)
    self.sendChoice.setText("Send to: "+text)

class Window(QMainWindow):
    def __init__(self):
        super(Window, self).__init__()

```

```

        self.setGeometry(50, 50, 500, 300)
        self.setWindowTitle("Chat-Client")
        self.table_widget = MyTableWidget(self)
        self.setCentralWidget(self.table_widget)
        self.show()

    def closeEvent(self, event):
        close = QMessageBox()
        close.setText("You sure?")
        close.setStandardButtons(QMessageBox.Yes | QMessageBox.Cancel)
        close = close.exec()
        if close == QMessageBox.Yes:
            self.table_widget.disconnect_server() #disconnect to server
before exit
            event.accept()
        else:
            event.ignore()

    def run():
        app = QApplication(sys.argv)
        GUI = Window()
        sys.exit(app.exec_())

if __name__ == "__main__":
    run()

```

2. Socket ulanishini boshqarish (connect / send / receive) va xabar almashish

Nazariy qism

Socket — bu tarmoq orqali ikkita dastur (klient va server) oʻrtasida maʼlumot almashish uchun moʻljallangan interfeys. TCP/IP protokoli asosida ishlaganda, socket quyidagi bosqichlarda ishlaydi:

1. Server tomonida:

- socket() — yangi tarmoq socketini yaratadi.
- bind() — IP va portni socketga bogʻlaydi.
- listen() — ulanishlar uchun “tinglash” holatiga oʻtkazadi.
- accept() — klient ulanayotganda ulanishni qabul qiladi.

2. Klient tomonida:

- socket() — socket yaratadi.
- connect() — server bilan ulanadi.
- send() — serverga xabar yuboradi.
- recv() — serverdan xabar oladi.

3. Maʼlumot almashish:

- Ikkala tomon send() va recv() yordamida almashadi.

- TCP holatida oqimli (stream-based) uzatish bo‘ladi — xabar bo‘linmasligi uchun framing ishlatiladi.

Amaliy misol: TCP klient va server

server.py

```
import socket

HOST, PORT = '127.0.0.1', 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(1)
    print(f"Server {HOST}:{PORT} da tinglamoqda...")

    conn, addr = server.accept()
    with conn:
        print("Ulandi:", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            print("Kelgan xabar:", data.decode())
            conn.sendall(data.upper())
```

client.py

```
import socket

HOST, PORT = '127.0.0.1', 5000

with socket.create_connection((HOST, PORT)) as client:
    xabar = input("Serverga xabar yozing: ")
    client.sendall(xabar.encode())
    javob = client.recv(1024)
    print("Serverdan javob:", javob.decode())
```

Natija: Klient “salom” deb yuborsa, server uni “SALOM” qilib qaytaradi.

3. Signal–slot mexanizmi va xatoliklarni GUI’da ko‘rsatish

Nazariy qism

Signal–slot mexanizmi — bu **PyQt5** (yoki **PySide**) da grafik interfeys (GUI) elementlari o‘rtasida aloqa o‘rnatish tizimi.

- **Signal** — biror hodisa sodir bo‘lishi (masalan, tugma bosilishi).
- **Slot** — bu hodisaga javoban bajariladigan funksiya yoki metod.

Misol:

```
self.pushButton.clicked.connect(self.send_message)
```

Bu kodda clicked signali send_message slotiga bog‘langan — tugma bosilganda funksiya ishga tushadi.

Amaliy GUI misol: TCP klient PyQt5’da

client_gui.py

```

import sys
import socket
from PyQt5 import QtWidgets, QtCore

class TcpClient(QtWidgets.QWidget):
    def __init__(self):
        super().__init__()
        self.init_ui()
        self.sock = None

    def init_ui(self):
        self.setWindowTitle("TCP Klient GUI")
        self.resize(400, 250)

        self.ip_label = QtWidgets.QLabel("Server IP:")
        self.ip_input = QtWidgets.QLineEdit("127.0.0.1")

        self.port_label = QtWidgets.QLabel("Port:")
        self.port_input = QtWidgets.QLineEdit("5000")

        self.msg_label = QtWidgets.QLabel("Xabar:")
        self.msg_input = QtWidgets.QLineEdit()

        self.connect_btn = QtWidgets.QPushButton("Ulanish")
        self.send_btn = QtWidgets.QPushButton("Yuborish")
        self.output = QtWidgets.QTextEdit()
        self.output.setReadOnly(True)

        grid = QtWidgets.QGridLayout()
        grid.addWidget(self.ip_label, 0, 0)
        grid.addWidget(self.ip_input, 0, 1)
        grid.addWidget(self.port_label, 1, 0)
        grid.addWidget(self.port_input, 1, 1)
        grid.addWidget(self.msg_label, 2, 0)
        grid.addWidget(self.msg_input, 2, 1)
        grid.addWidget(self.connect_btn, 3, 0)
        grid.addWidget(self.send_btn, 3, 1)
        grid.addWidget(self.output, 4, 0, 1, 2)
        self.setLayout(grid)

        # Signal-slot bog'lanishi
        self.connect_btn.clicked.connect(self.connect_server)
        self.send_btn.clicked.connect(self.send_message)

    def connect_server(self):
        try:
            host = self.ip_input.text()
            port = int(self.port_input.text())
            self.sock = socket.create_connection((host, port), timeout=5)
            self.output.append(f"[OK] Ulandi: {host}:{port}")

```

```

except Exception as e:
    self.output.append(f"[XATO] Ulanishda xato: {e}")

def send_message(self):
    if not self.sock:
        self.output.append("[XATO] Avval serverga ulang!")
        return
    try:
        msg = self.msg_input.text().encode()
        self.sock.sendall(msg)
        reply = self.sock.recv(1024).decode()
        self.output.append(f"Server: {reply}")
    except Exception as e:
        self.output.append(f"[XATO] Jo'natishda yoki qabulda xato: {e}")
        self.sock = None # ulanish uzilgan

```

ishga tushirish:

```

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    window = TcpClient()
    window.show()
    sys.exit(app.exec_())

```

GUI'da xatoliklarni ko'rsatish mexanizmlari**1. TextEdit / QLabel orqali:**

```
self.output.append("[XATO] Ulanishda muammo: Serverga bog'lanmadi.")
```

2. Xabar oynasi orqali:

```
QtWidgets.QMessageBox.warning(self, "Xato", "Serverga ulanib bo'lmadi!")
```

3. Signal orqali xatoni uzatish:

```

self.error_signal = QtCore.pyqtSignal(str)
self.error_signal.connect(self.show_error)
def show_error(self, msg):
    self.output.append(f"[XATO]: {msg}")

```

Xulosa

- **Socket** — tarmoqda ma'lumot almashish interfeysi.
- **connect/send/recv** — TCP orqali xabar yuborish va olish uchun asosiy funksiyalar.
- **Signal-slot** — PyQt5'da hodisalar va funksiyalar o'rtasidagi aloqa.
- GUI'da xatoliklarni foydalanuvchiga ko'rsatish uchun QTextEdit, QMessageBox yoki statusBar ishlatiladi.

Nazorat savollari

1. TCP protokolining asosiy bosqichlari (connect, send, recv) qanday ketma-ketlikda bajariladi?
2. Socket obyektining bind(), listen(), va accept() funksiyalari nimani bajaradi?
3. PyQt5'dagi **signal-slot** mexanizmi nima va u qanday ishlaydi?
4. GUI'da xatoliklarni foydalanuvchiga ko'rsatishning uchta usulini yozing.

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

6-mashg'ulot. Pythonda GUI paketidan foydalanib chat dasturini tuzishni yakunlash

O'quv savollari:

1. TCP client–server dasturini GUI ko'rinishda ishlab chiqish.
2. Xabar almashish oqimini (connect/send/receive) tekshirish va sinovdan o'tkazish.
3. Xatoliklarni qayta ishlash, holatlarni (status/reconnect) GUI'da boshqarish.

TCP client-server dasturini GUI ko'rinishda ishlab chiqish.

```
import argparse
from socket import AF_INET, socket, SOCK_STREAM
from threading import Thread

def accept_incoming_connections():
    """Sets up handling for incoming clients."""
    while True:
        client, client_address = SERVER.accept()
        print("%s:%s has connected." % client_address)
        addresses[client] = client_address
        Thread(target=handle_client, args=(client,)).start()

def handle_client(client): # Takes client socket as argument.
    """Handles a single client connection."""
    name = ""
    prefix = ""

    while True:
        msg = client.recv(BUFSIZ)

        if not msg is None:
            msg = msg.decode("utf-8")

            if msg == "":
                msg = "{QUIT}"

            # Avoid messages before registering
            if msg.startswith("{ALL}") and name:
                new_msg = msg.replace("{ALL}", "{MSG}" + prefix)
                send_message(new_msg, broadcast=True)
                continue

            if msg.startswith("{REGISTER}"):
                name = msg.split(" ")[1]
                welcome = '{MSG>Welcome %s!' % name
                send_message(welcome, destination=client)
```

```

    msg = "{MSG}%s has joined the chat!" % name
    send_message(msg, broadcast=True)
    clients[client] = name
    prefix = name + ": "
    send_clients()
    continue

if msg == "{QUIT}":
    client.close()
    try:
        del clients[client]
    except KeyError:
        pass
    if name:
        send_message("{MSG}%s has left the chat." % name, broadcast=True)
        send_clients()
    break

# Avoid messages before registering
if not name:
    continue

# We got until this point, it is either an unknown message or for an
# specific client...
try:
    msg_params = msg.split("{}")
    dest_name = msg_params[0][1:] # Remove the {
    dest_sock = find_client_socket(dest_name)
    if dest_sock:
        send_message(msg_params[1], prefix=prefix, destination=dest_sock)
    else:
        print("Invalid Destination. %s" % dest_name)
except:
    print("Error parsing the message: %s" % msg)

def send_clients():
    send_message("{CLIENTS}" + get_clients_names(), broadcast=True)

def get_clients_names(separator="|"):
    names = []
    for _, name in clients.items():
        names.append(name)
    return separator.join(names)

def find_client_socket(name):
    for cli_sock, cli_name in clients.items():
        if cli_name == name:
            return cli_sock
    return None

```

```

def send_message(msg, prefix="", destination=None, broadcast=False):
    send_msg = bytes(prefix + msg, "utf-8")
    if broadcast:
        """Broadcasts a message to all the clients."""
        for sock in clients:
            sock.send(send_msg)
    else:
        if destination is not None:
            destination.send(send_msg)

clients = {}
addresses = {}

parser = argparse.ArgumentParser(description="Chat Server")
parser.add_argument(
    '--host',
    help='Host IP',
    default="127.0.0.1"
)
parser.add_argument(
    '--port',
    help='Port Number',
    default=33002
)

server_args = parser.parse_args()

HOST = server_args.host
PORT = int(server_args.port)
BUFSIZ = 2048
ADDR = (HOST, PORT)

stop_server = False

SERVER = socket(AF_INET, SOCK_STREAM)
SERVER.bind(ADDR)

if __name__ == "__main__":
    try:
        SERVER.listen(5)
        print("Server Started at {}:{}".format(HOST, PORT))
        print("Waiting for connection...")
        ACCEPT_THREAD = Thread(target=accept_incoming_connections)
        ACCEPT_THREAD.start()
        ACCEPT_THREAD.join()
        SERVER.close()
    except KeyboardInterrupt:

```

```
print("Closing...")
ACCEPT_THREAD.interrupt()
```

2) Xabar almashish oqimini (connect / send / receive) tekshirish va sinovdan o'tkazish

Nazariya (muxtasar)

- **TCP oqim (stream):** send() va recv() paket chegaralarini saqlamaydi. Shuning uchun **framing** kerak (masalan, 4 baytli uzunlik-prefiks).

- **sendall():** butun bufer jo'natilmaguncha qaytmaydi — qisqa jo'natmalarga yechim.

- **recv():** talab qilingan bayt sonidan kam qaytishi mumkin; kerakli uzunlikka yetguncha yig'ish lozim.

- **Testlash:**

1. Lokal echo-server (127.0.0.1, masalan 5000-port).
2. Klientdan bir nechta xabar yuborib, javoblarni tekshirish.
3. Uzun/past tezlikli sharoitlar (simulyatsiya) — framing sinfi buzilmasligi kerak.

Oddiy echo-server (sinov uchun)

```
# server.py
import socket, struct

HOST, PORT = '127.0.0.1', 5000

def recv_exact(s, n):
    buf=b''
    while len(buf)<n:
        c=s.recv(n-len(buf))
        if not c: raise ConnectionError("peer closed")
        buf+=c
    return buf

def recv_msg(s):
    ln, = struct.unpack('!I', recv_exact(s, 4))
    return recv_exact(s, ln)

def send_msg(s, payload: bytes):
    s.sendall(struct.pack('!I', len(payload)) + payload)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((HOST, PORT)); srv.listen(1)
    print(f"Echo server: {HOST}:{PORT}")
    conn, addr = srv.accept()
    with conn:
        print("Client:", addr)
        try:
```

```

while True:
    data = recv_msg(conn)
    send_msg(conn, data.upper())    # echo + transform
except ConnectionError:
    pass

```

3) Xatoliklarni qayta ishlash, holatlarni (status/reconnect) gui'da boshqarish

Nazariya (muxtasar)

- **GUI bloklanmasin:** tarmoqli ishlar **asosiy GUI oqimidan alohida** (QThread yoki QRunnable) bajarilishi kerak.
- **Signal–slot:** fon ishchi (socket) → GUI'ga **signallar** yuboradi: connected, messageReceived, error, disconnected, statusChanged.
- **Timeout & reconnect:** setTimeout() bilan I/O cheklash, xatoda **eksponent backoff + jitter** bilan qayta ulanish.
- **Holat boshqaruvi:** status-indikator (QLabel/QStatusBar), “Auto reconnect” belgilash, “Connect/Disconnect” tugmalarining faollashuvi.
- **Xatolarni ko'rsatish:** QTextEdit log, kerak bo'lsa QMessageBox (bloklovchi dialogni me'yorida qo'llash).

PyQt5 GUI: klient + fon ishchi (QThread)

Ushbu kod server.py bilan birga ishlaydi. GUI bloklanmaydi, statuslar real-vaqtda yangilanadi, framing ishlatiladi, xatolarda reconnect ixtiyoriy.

```

# client_gui.py
import sys, time, socket, struct, random
from PyQt5 import QtWidgets, QtCore

# ----- Framing yordamchilari -----
def _send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)

def _recv_exact(sock, n):
    buf=b''
    while len(buf)<n:
        c=sock.recv(n-len(buf))
        if not c:
            raise ConnectionError("peer closed")
        buf+=c
    return buf

def _recv_msg(sock) -> bytes:
    ln, = struct.unpack('!I', _recv_exact(sock, 4))
    return _recv_exact(sock, ln)

# ----- Worker: tarmoq QThread'da -----
class SocketWorker(QtCore.QThread):

```



```

connected = QtCore.pyqtSignal()
disconnected = QtCore.pyqtSignal()
messageReceived = QtCore.pyqtSignal(str)
error = QtCore.pyqtSignal(str)
status = QtCore.pyqtSignal(str)

def __init__(self, host, port, auto_reconnect=False, parent=None):
    super().__init__(parent)
    self.host = host
    self.port = port
    self.auto_reconnect = auto_reconnect
    self._sock = None
    self._running = True
    self._outbox = QtCore.QQueue() # xabarlar navbati
    self._mtx = QtCore.QMutex() # navbat uchun
    self._hasData = QtCore.QWaitCondition() # yuborish uyg'otgichi

# Qt'da QQueue yo'q - kichik wrapper: list + mtx + condition
# Qulaylik uchun o'zimiz imitatsiya qilamiz
def queue_put(self, item: bytes):
    self._mtx.lock()
    try:
        if not hasattr(self, "_queue"):
            self._queue = []
        self._queue.append(item)
        self._hasData.wakeAll()
    finally:
        self._mtx.unlock()

def queue_get(self):
    self._mtx.lock()
    try:
        while self._running and (not hasattr(self, "_queue") or not
self._queue):
            self._hasData.wait(self._mtx, 50) # 50 ms
        if not self._running:
            return None
        return self._queue.pop(0)
    finally:
        self._mtx.unlock()

def stop(self):
    self._running = False
    try:
        if self._sock:
            self._sock.shutdown(socket.SHUT_RDWR)
    except OSError:
        pass
    try:
        if self._sock:

```

```

        self._sock.close()
    except OSError:
        pass

def _connect_with_retry(self):
    delay, cap = 0.5, 8.0
    while self._running:
        try:
            s = socket.create_connection((self.host, self.port), timeout=5)
            s.settimeout(10)
            self.status.emit(f"Ulandi: {self.host}:{self.port}")
            self.connected.emit()
            return s
        except (ConnectionRefusedError, TimeoutError, OSError) as e:
            self.error.emit(f"Ulanmadi: {e}")
            if not self.auto_reconnect:
                return None
            self.status.emit(f"Qayta urinish... {delay:.1f}s")
            self.msleep(int((delay + random.random()*0.2) * 1000))
            delay = min(delay*2, cap)
    return None

def run(self):
    while self._running:
        self._sock = self._connect_with_retry()
        if not self._sock:
            self.disconnected.emit()
            break

        try:
            # Asosiy sikl: navbatdan yuborish va soketdan qabul qilish
            while self._running:
                # Yuborish
                item = self.queue_get()
                if item is None:
                    break
                try:
                    _send_msg(self._sock, item)
                except (socket.timeout, OSError, ConnectionError) as e:
                    self.error.emit(f"Yuborishda xato: {e}")
                    raise

                # Qabul
                try:
                    reply = _recv_msg(self._sock).decode('utf-8',
errors='replace')

                    self.messageReceived.emit(reply)
                except (socket.timeout, OSError, ConnectionError) as e:
                    self.error.emit(f"Qabulda xato: {e}")
                    raise

```

```

except Exception as e:
    self.status.emit("Ulanish uzildi")
    self.disconnected.emit()
    try:
        self._sock.close()
    except Exception:
        pass

    if not self.auto_reconnect:
        break
    # auto_reconnect bo'lsa - loop boshiga qaytamiz

self.status.emit("Worker to'xtadi")

# ----- GUI -----
class ClientGUI(QtWidgets.QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TCP Klient (PyQt5)")
        self.resize(520, 360)

        self.hostEdit = QtWidgets.QLineEdit("127.0.0.1")
        self.portEdit = QtWidgets.QLineEdit("5000")
        self.connectBtn = QtWidgets.QPushButton("Connect")
        self.disconnectBtn = QtWidgets.QPushButton("Disconnect")
        self.autoReconnect = QtWidgets.QCheckBox("Auto reconnect")
        self.autoReconnect.setChecked(True)

        self.msgEdit = QtWidgets.QLineEdit()
        self.sendBtn = QtWidgets.QPushButton("Send")
        self.log = QtWidgets.QTextEdit(); self.log.setReadOnly(True)
        self.statusBar = QtWidgets.QLabel("Disconnected")

        grid = QtWidgets.QGridLayout(self)
        grid.addWidget(QtWidgets.QLabel("Host:"), 0, 0)
        grid.addWidget(self.hostEdit, 0, 1)
        grid.addWidget(QtWidgets.QLabel("Port:"), 0, 2)
        grid.addWidget(self.portEdit, 0, 3)
        grid.addWidget(self.connectBtn, 0, 4)
        grid.addWidget(self.disconnectBtn, 0, 5)
        grid.addWidget(self.autoReconnect, 1, 0, 1, 2)
        grid.addWidget(QtWidgets.QLabel("Xabar:"), 2, 0)
        grid.addWidget(self.msgEdit, 2, 1, 1, 4)
        grid.addWidget(self.sendBtn, 2, 5)
        grid.addWidget(self.log, 3, 0, 1, 6)
        grid.addWidget(self.statusBar, 4, 0, 1, 6)

        self.worker = None
        self._set_connected(False)

```

```

# signal-slot
self.connectBtn.clicked.connect(self.on_connect)
self.disconnectBtn.clicked.connect(self.on_disconnect)
self.sendBtn.clicked.connect(self.on_send)

def _set_connected(self, ok: bool):
    self.connectBtn.setEnabled(not ok)
    self.disconnectBtn.setEnabled(ok)
    self.sendBtn.setEnabled(ok)
    if ok:
        self.statusBar.setText("Connected")
    else:
        self.statusBar.setText("Disconnected")

def on_connect(self):
    if self.worker and self.worker.isRunning():
        return
    host = self.hostEdit.text().strip()
    port = int(self.portEdit.text().strip() or "0")
    self.worker = SocketWorker(host, port,
auto_reconnect=self.autoReconnect.isChecked())
    # Worker signallari → GUI slotlari
    self.worker.connected.connect(lambda: (self._set_connected(True),
self.log.append("[OK] Connected")))
    self.worker.disconnected.connect(lambda: (self._set_connected(False),
self.log.append("[i] Disconnected")))
    self.worker.messageReceived.connect(lambda m: self.log.append(f"[SERVER]
{m}"))
    self.worker.error.connect(lambda e: self.log.append(f"[ERROR] {e}"))
    self.worker.status.connect(self.statusBar.setText)
    self.worker.start()

def on_disconnect(self):
    if self.worker:
        self.worker.auto_reconnect = False
        self.worker.stop()
        self.worker.wait(1500) # yumshoq to'xtash
        self._set_connected(False)
        self.log.append("[i] Manual disconnect")

def on_send(self):
    if not self.worker or not self.worker.isRunning():
        self.log.append("[WARN] Avval ulanib oling")
        return
    payload = self.msgEdit.text().encode('utf-8')
    if not payload:
        self.log.append("[WARN] Bo'sh xabar yuborilmaydi")
        return
    self.worker.queue_put(payload)

```

```

self.log.append(f"[YOU] {self.msgEdit.text()}")
self.msgEdit.clear()

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    w = ClientGUI()
    w.show()
    sys.exit(app.exec_())

```

Nimalar bu yerda hal qilingan?

- **Framing:** 4 baytli uzunlik-prefiks (!!).
- **GUI bloklanmaydi:** soket ishlari QThread ichida.
- **Status/reconnect:** holat QLabel orqali, ixtiyoriy **Auto reconnect** bilan.
- **Xatolar:** error signali orqali log'ga tushadi, status yangilanadi.
- **Yopish:** shutdown → close, stop() va wait() bilan yumshoq to'xtash.

Oqimni testlash bo'yicha tezkor chek-list

1. python server.py — echo-serverni ishga tushiring.
2. python client_gui.py — GUI'ni ishga tushiring.
3. **Connect** bosing → “Connected”.
4. Bir nechta xabar yuboring → serverdan katta harfda qaytadi.
5. Serverni to'xtating → GUI'da Disconnected va/ yoki **Auto reconnect** bo'lsa qayta urinishlar log'da ko'rinadi.
6. Qayta ishga tushirilgan serverga GUI avtomatik ulanadi.

Nazorat savollari

1. Nega TCP'da framing (masalan, uzunlik-prefiks) zarur va recv() nega xabarni bo'lib yuborishi mumkin?
2. sendall() va send() o'rtasidagi farq nimada? Qaysi holatda sendall() kerak bo'ladi?
3. PyQt'da nega soket ishlarini QThread (yoki QRunnable) ga chiqarish tavsiya etiladi?
4. “Auto reconnect” ni qanday strategiya bilan amalga oshirish ma'qul (timeout, backoff, jitter)?
5. GUI'da xatolik va holatlarni foydalanuvchiga ko'rsatishning uchta amaliy usulini sanang (log, status, dialog va h.k.).

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

7-mashg'ulot. Python ilovasini maxsus paket yordami.

O'quv savollari:

1. Python ilovasini maxsus paket yordamida yuklanuvchi fayl ko'rinishiga keltirish.

2. PyInstaller, cx_Freeze kabi paketlarning imkoniyatlari bilan tanishish.

3. Dasturiy ta'minotni turli operatsion tizimlarda sinovdan o'tkazish.

4. Yaratilgan yuklanuvchi faylni foydalanuvchi uchun qulay shaklda taqdim etish.

1. Python ilovasini maxsus paket yordamida yuklanuvchi fayl ko'rinishiga keltirish.

Ushbu bo'limda pythonda yaratilgan loyihalarning yuklanuvchi faylini hosil qilish bo'yicha bilim va ko'nikmalarga berib o'tiladi.

Dasturchi yaratgan dasturini kimgadir taqdim etishi uchun albatda butun bir loyihasini emas abalki butun loyihani bitta yuklanuvchi faylga birlashtirib, keyin taqdim etadi. Bunday bitta faylga birlashtirishning turli usullari mavjud.

Pythonning PYINSTALLER paketi va uning imkoniyatlari

PyInstaller paketi yordamida yuklanuvchi faylni hosil qilishni bir nechta qadamlarga bo'lish mumkin:

1-qadam: PyInstaller paketini kompyuterga o'rnatish.

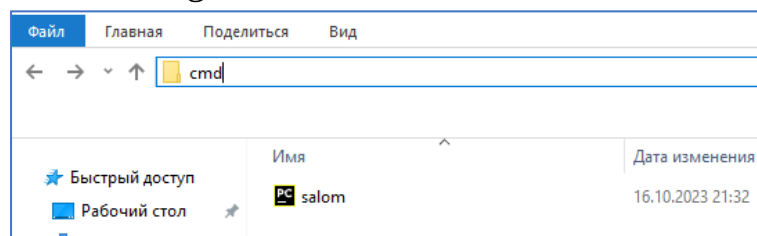
PyInstaller paketini kompyuterga o'rnatish uchun quyidagi buyruqni buyruklar satriga kiritiladi:

```
pip install pyinstaller
```

Shundan so'ng ushbu paketdan fodalanish mumkin bo'ladi.

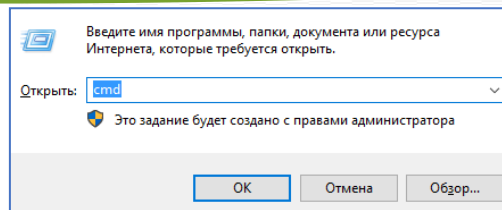
2-qadam: PyInstaller yordamida bajariladigan fayl hosil qilish.

Endi *PyInstaller* paketi yordamida Python tilida yozilgan loyihani bitta faylga jamlash kerak. Buning uchun project joylashgan papkaga kirish va buyruq qatoriga **CMD** buyrug'ini kiritib, enter tugmasini bosiladi:



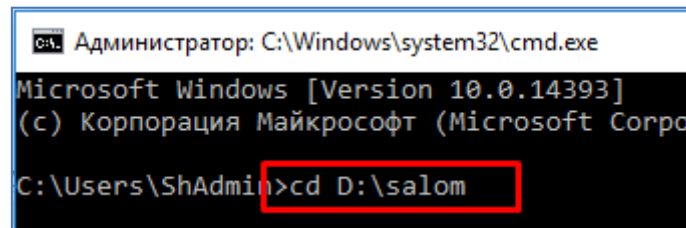
80-rasm. Buyruq qatoriga CMD buyrug'ini kiritish

Yoki ikkinchi usuli kompyuteringinzdan PUSK+R tugmasi bosiladi va hosil bo'lgan «**Выполнить**» oynasiga “CMD” buyrug'i kiritiladi.



81-rasm. «Выполнить» oynasi

Va hosil bo'lgan konsol oynasiga **cd** buyrug'i orqali kerakli papkaga kirish mumkin. Buning uchun **cd** va undan keyin Python skriptingiz saqlanadigan joy quyidagi ko'rinishda kiritiladi va enter tugmasi bosiladi va ko'rsatilgan kotalokga kiriladi:

82-rasm. **cd** buyrug'i orqali kerakli papkaga kirish

Keyin bajariladigan amal faylni yaratish uchun quyidagi shablondan to'g'ri foydalanish qilishdan iborat:

```
pyinstaller --onefile pythonFileName.py
```

Ushbu shablonda *pythonFileName* "**salom**" (va fayl kengaytmasi albatta .py) bo'lgani uchun bajariladigan faylni yaratish buyrug'i quyidagicha bo'ladi:

```
pyinstaller --onefile salom.py
```

ENTER tugmasini bosiladi va pyinstaller paketi avtomatik ravishda ushbu papka ichida dist papkasini hosil qiladi. Yuklanuvchi exe fayl esa ushbu papkaning ichida joylashgan bo'ladi. Endi ushbu exe faylni python dasturi o'rnatilmagan kompyuterlarda ham ishga tushirish mumkin.

Pythonning AUTO-PY-TO-EXE paketi va uning imkoniyatlari

Pythonda yaratilgan loyihalarni yagona .exe faylga yig'ishning yana bir usuli – bu auto-py-to-exe paketidan foydalangan holda judayam funksiyanal va visual tarzda amalga oshirish mumkin. Buning uchun ushbu paketni kompyuterga o'rnatish kerak bo'ladi.

auto-py-to-exe paketini kompyuterga o'rnatish. Kompyuterga python paketlarini o'rnatishning bir nechta usullari mavjud. Bullardan **pip**, **github** hamda **offline** holda o'rnatish mumkin. Odatda pip orqali o'rnatiladi. Chunki bu usul qulay hisoblanadi.

Pip orqali o'rnatish. *Auto-py-to-exe* paketini kompyuterga Pip orqali o'rnatish quyidagicha amalga oshiriladi: Birinchi navbatda “**Команда строка**” oynasi ochib

olinadi va u joyga pip install auto-py-to-exe buyrug'i kiritiladi va enter tugmasi bosiladi. Shundan so'ng avtomatik ravishda kompyuterga ushbu paket o'rnatiladi:

```
$ pip install auto-py-to-exe
```

GitHubdan o'rnatish. Ushbu paketni to'g'ridan-to'g'ri GitHubdan o'rnatish mumkin. GitHubdan *auto-py-to-exe* paketini o'rnatish uchun avval GitHub omborini klonlash kerak bo'ladi. U quyidagi kod orqali amalga oshiriladi:

```
git clone https://github.com/brentvollebrecht/auto-py-to-exe.git
```

Keyin auto-py-to-exe papkasiga cd buyrug'i orqali o'tish kerak.

```
$ cd auto-py-to-exe
```

Endi python buyrug'i orqali setup.py faylini ishga tushirish kerak.

```
$ python setup.py install
```

Quyidagi buyruq yordamida versiyani ham tekshirish mumkin:

```
auto-py-to-exe --version
```

```
C:\Users\ShAdmin>auto-py-to-exe --version
auto-py-to-exe 2.42.0
```

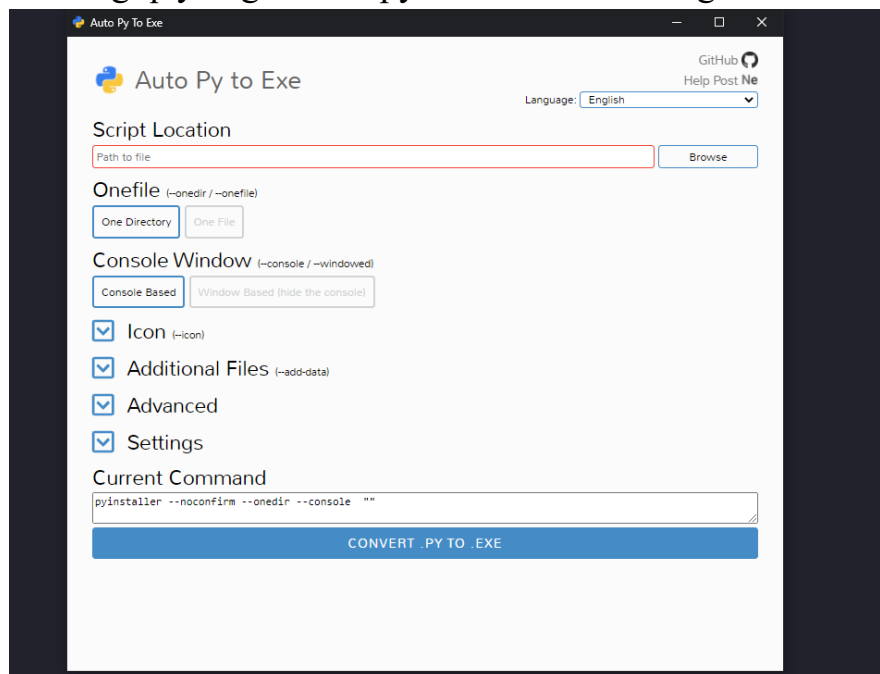
83-rasm. Paketning versiyasini bilish

Auto py to exe ning joriy versiyasi 2.42.0 va u endi kompyuterga o'rnatilgan.

Ilovani ishga tushirish. auto-py-to-exe ilivasini ishga tushirish uchun terminalda quyidagi buyruqni bajarish kerak bo'ladi:

```
$ auto-py-to-exe
```

Shundan so'ng quyidagi "Auto py to exe" dasturi ishga tushadi:

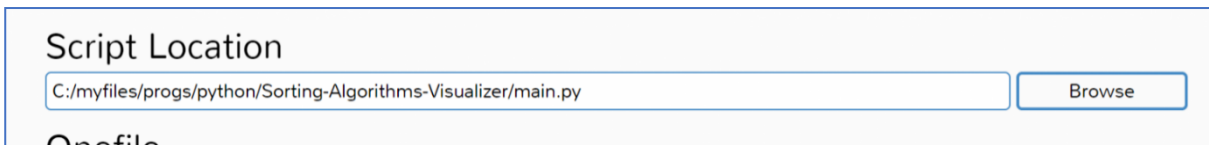


84-rasm. avto-py-to-exe GUI interfeysi

Endi ushbu interfeysdan .py faylni yuklanuvchi .exe fayliga aylantirish uchun foydalaniladi.

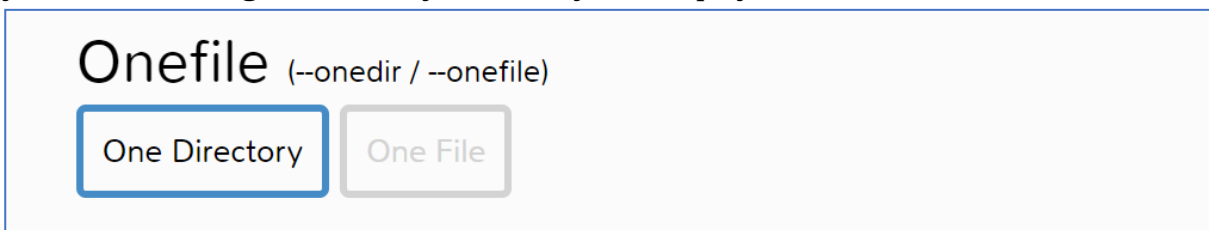
Konvertatsiya jarayoni. 1-qadam. Fayl joylashuvini qo'shish. Python faylini .exe fayliga o'zgartirish uchun avval uning yo'lini ko'rsatish kerak bo'ladi. Buning

uchun konvertatsiya qilinadigan faylning manziliga o'tiladi va keyin yo'lni ko'rsatib, qo'shib qo'yiladi:



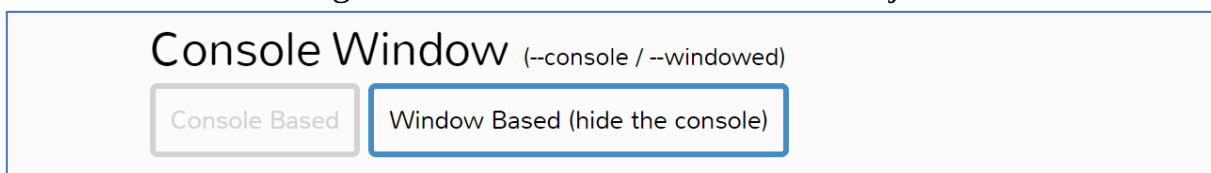
85-rasm. Faylni yo'lni ko'rsatish

2-qadam: "One Directory" yoki "One File" ni tanlash. Interfeysda "One Directory" yoki "One File" ni tanlash imkoniyati mavjud. Odatda ko'pchilik katta Python loyihalar bir nechta fayllardan tashkil topgan bo'ladi, shuning uchun "One Directory" ni tanlash afzalroq. Ushbu parametr barcha kerakli fayllar bilan papkani yaratadi, shuningdek .exe faylini ham yaratib qo'yadi.



86-rasm. "One Directory" yoki "One File" ni tanlash

3-qadam. "Console Based" yoki "Windows Based" ni tanlash. Yuqoridagi amallarni bajarilganidan so'ng dastur turini tanlash kerak: **Console** yoki **Windows** asoslangan. Agar "Windows Based" ni tanlansa, bu dasturning barcha konsol natijalarini yashiradi. Agar yaratilgan dastur konsol chiqishiga asoslangan holda ishlaydigan bo'lsa, unda "Console Based" ni tanlash maqsadga muvofiq bo'ladi. Agar dasturda GUI ilovasi bo'lsa yoki foydalanuvchiga konsol chiqishini ko'rsatish shart bo'lmasa, unda "Windows Based" ni tanlash tavsiya qilinadi. Bu yerda ikkinchi ikkinchi variantni tanlangan, chunki ushbu dasturda GUI mavjud.



87-rasm. Konsolga asoslangan yoki oynaga asoslangan

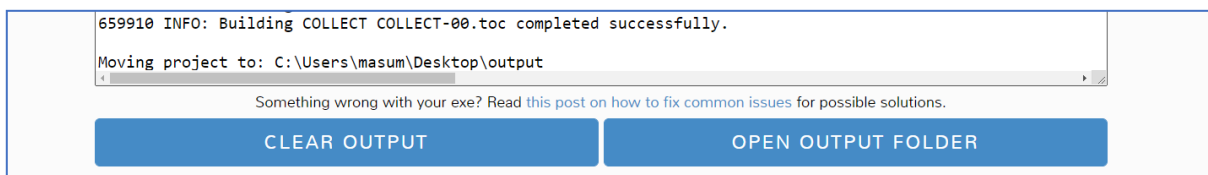
4-qadam. Konvertatsiya qilish. Yuqoridagi barcha amallarni bajarilganidan keyin qolgan sozlamalari masalan, piktogramma, qo'shimcha fayllar va hk. ni o'z standart holatida qoldirib, konvertatsiya qilish mumkin. Buning uchun faqat **CONVERT .PY TO .EXE** tugmasini bosish kifoya.



88-rasm. Konvertatsiya qilish uchun tugma.

Jarayonni yakunlash uchun biroz kutishingiz kerak bo'ladi.

Open output folder. Konvertatsiya jarayoni tugaganidan so'ng, "Open output folder" tugmasini bosish orqali konvertatsiya qilingan fayl joylashgan papkani ochish mumkin.

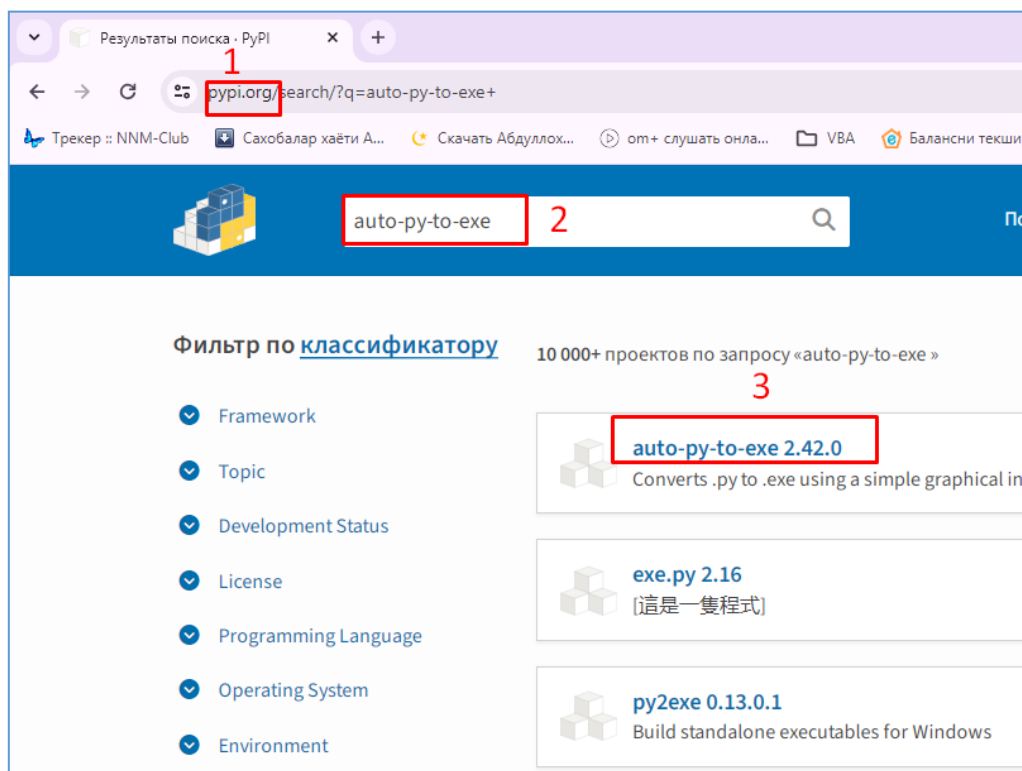


89-rasm. Open output folder tugmasi

Shundan so'ng .exe fayl joylashgan papka ochiladi. Endi uni Python-ni o'rnatmasdan boshqa kompyuterlarda ham bemaolol ishga tushirib foydalanish mumkin.

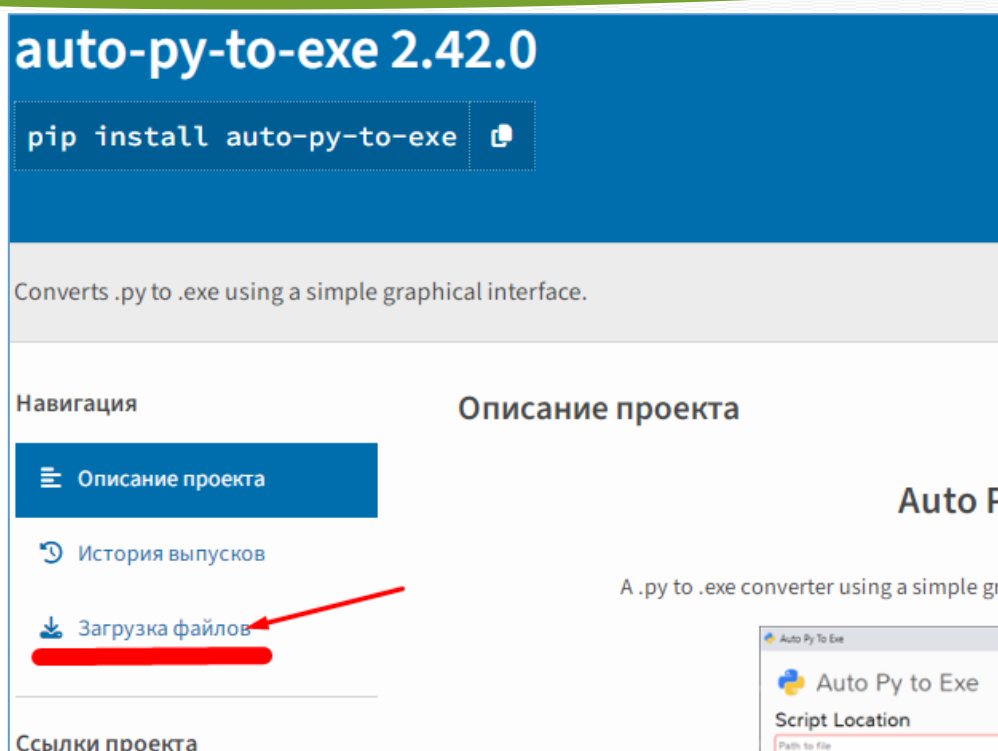
Python paketlarini offline holatda kompyuterga o'rnatish

Ba'zi hollarda internet tarmog'iga ulanmagan kompyuterlarga ham python paketlarini o'rnatishga zarurat paydo bo'lib qoladi. Bunday vaziyatlarda kompyuterga pythonning kerakli paketini olib kelib, o'rnatiladi. Buning uchun boshqa bir internetga ulangan kompyuterdan **pypi.org** saytiga kirib, kerakli paketni qidirib topiladi va yuklab olinadi. Masalan:



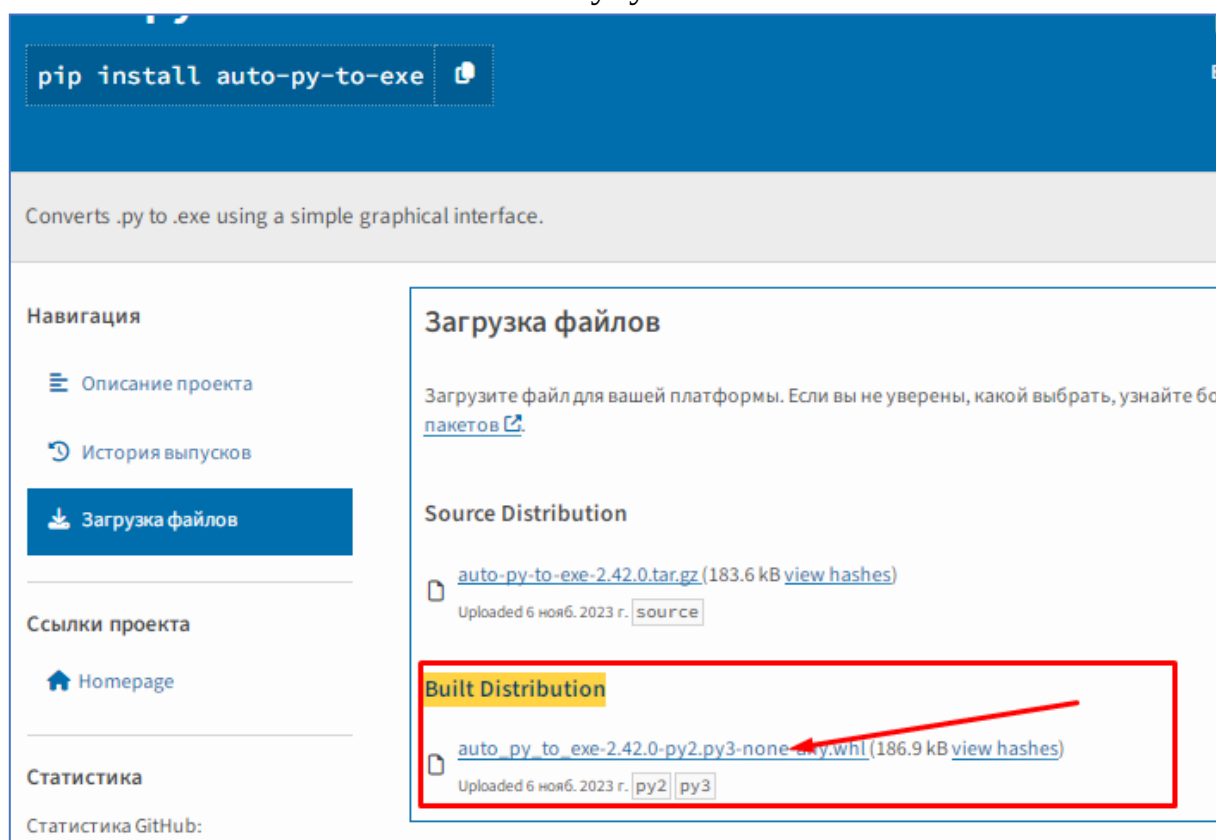
90-rasm. Pypi.org sayti bilan ishlash

Ushbu kerakli paketni topib, uni tanlanganda paket haqida ko'plab kerakli ma'lumotlar beruvchi oyna ochiladi. U oynada paketni yuklab olish tugmasi ham mavjud va yuklab olish "Загрузка файлов" bo'limini tanlanlash orqali amalga oshiriladi.



91-rasm. PyPI.org saytidan kerakli paketni yuklab olish

Shundan so‘ng paketni yuklab olish uchun “**Загрузка файлов**” oynasi ochiladi. U yerdan “**Built Distribution**” bo‘limidagi faylni yuklab olish uchun maxsus ssilkani bosiladi va avtomatik ravishda fayl yuklab olinadi.



92-rasm. Kerakli paketni yuklab olish

Ushbu yuklab olingan faylni boshqa bir kompyuterga ko‘chirib o‘tkaziladi. Endi ushbu kerakli yukanuvchi faylni kompyuterga o‘rnatish kerak. Buning uchun fayl

oddiy .exe fayllar kabi emas balki maxsus usulda o'rnatilishi kerak bo'ladi. Ushbu amal "**py -m pip install file name**" shabloni yordamida amalga oshiriladi. Bizning misolimizda file Downloads papkasida joylashganligi uchun terminaldan osha papkaga kirib borib, undan so'ng o'rnatish kodini teriladi va enter tugmasi bosish orqali o'rnatish amalga oshiriladi.

```
Microsoft Windows [Version 10.0.14393]
(c) Корпорация Майкрософт (Microsoft Corporation), 2016. Все права защищены.

C:\Users\ShAdmin\Downloads>py -m pip install auto py to exe-2.42.0-py2.py3-none-any.whl
```

93-rasm. Kerakli paketni lokal holatda o'rnatish

Shundan so'ng, agarda hech qanday xatolik ro'y bermasa, paket kompyuterga muvoffaqiyatli o'tnatiladi.

Barcha paketlar kompyuterga lokal holatda o'rnatilishi mumkin.

2) pyinstaller va cx_freeze imkoniyatlari

Minimal namuna (test qilinadigan kichik dastur)

```
app.py:
import sys
def main():
    print("Hello from packaged app!")
    if len(sys.argv) > 1:
        print("Args:", sys.argv[1:])
if __name__ == "__main__":
    main()
```

PyInstaller (eng ommabop, juda moslashuvchan)

O'rnatish

```
pip install pyinstaller
```

Tez start (CLI ilova)

```
pyinstaller --onefile app.py
# dist/app(.exe) yaratiladi
```

GUI rejim (Windows'da konsol oynasiz)

```
pyinstaller --onefile --noconsole --name MyApp app.py
```

Ma'lumot/fayllarni qo'shish (rasmlar, JSON va h.k.)

Eslatma: --add-data ajratgich Windows'da ;, macOS/Linux'da :.

```
# Windows
pyinstaller --onefile --add-data "assets\logo.png;assets" app.py
# macOS/Linux
pyinstaller --onefile --add-data "assets/logo.png:assets" app.py
```

Kodni ichida nisbiy yo'lni topish:

```
import sys, os
def resource_path(rel):
    base = getattr(sys, "_MEIPASS", os.path.dirname(__file__))
    return os.path.join(base, rel)
logo_path = resource_path("assets/logo.png")
```

Maxsus .spec bilan ilg'orroq sozlash

app.spec (asosiy qismlar):

```
# pyinstaller app.spec
block_cipher = None
a = Analysis(['app.py'],
             datas=[('assets/logo.png', 'assets/logo.png')],
             hiddenimports=['pkgutil'], # kerak bo'lsa
             ...)
exe = EXE(a.pure, a.zipped_data, a.scripts,
         name='MyApp',
         console=False) # True bo'lsa konsol chiqadi
```

Afzalliklar

- --onefile (bitta exe) yoki onedir (papka) rejimlari
- **Hidden import** va **hooklar** orqali murakkab kutubxonalarni (PyQt, requests, numpy, ...) to'g'ri yig'ish
- UPX siqish (agar o'rnatilgan bo'lsa): --upx-dir
- Mac'da .app bundle yaratadi, Linux'da ELF

Tipik muammolar (va yechimlar)

- **Yashirin importlar** topilmaydi → --hidden-import yoki hiddenimports=[] ga qo'shing
- **Qt plugin** topilmaydi → --add-data bilan platforms papkasini qo'shing (PyQt/PySide)
- **Antivirus false-positive** → exe'ni **imzolash** (Windows), yoki fayl nomi/hususiyatlarini o'zgartirish
- **Katta fayl** → onedir yengilroq; UPX qo'llash; keraksiz kutubxonalarni chiqarib tashlash

cx_Freeze (o'rnatkichlar bilan yaxshi integratsiya)

O'rnatish

```
pip install cx-Freeze
```

Setup skripti

setup.py:

```
from cx_Freeze import setup, Executable

build_options = {
    "packages": [],
    "excludes": [],
    "include_files": [("assets/logo.png", "assets/logo.png")],
}

executables = [Executable("app.py", base="console", target_name="MyApp")]

setup(
    name="MyApp",
    version="1.0.0",
    description="Sample app",
    options={"build_exe": build_options},
    executables=executables,
)
```

Build

```
python setup.py build
# natija: build/ .../ MyApp(.exe)
```

Afzalliklar

- setup.py orqali bildirishnoma, resurslar boshqaruvi
- MSI o'rnatkich (Windows) bilan integratsiya (bdist_msi)
- PyInstaller bilan yechilmaydigan ayrim import yo'llari/muammolarda barqaror bo'lishi mumkin

Tipik muammolar

- Qt/SDL/... kabi native kutubxonalar yo'llari → include_files bilan qo'shish
- VCRuntime DLL'lari → yangi Python distributsiyasi bilan odatda to'g'ri keladi, lekin ba'zan vcruntime140.dllni ham qo'shish kerak bo'ladi

3) TURLI OPERATSION TIZIMLARDA SINOVDAN O'TKAZISH

Asosiy qoida: *Python paketlari ko'pincha cross-compile qilmaydi.* Har bir OS uchun **o'sha OSda** build qiling va sinang.

Test strategiyasi (bosqichma-bosqich)

1. **Toza muhit:** har OS'da venv yarating, faqat kerakli paketlarni o'rinating.

2. **Build:** PyInstaller/cx_Freeze bilan mos binarni yarating.

3. **Run-time test:**

- CLI: --help, oddiy buyruqlar, fayl I/O sinovlari
- GUI: ochilish va yopilish, menyular, "open/save file", net funksiyalar

4. **Sistema bog'liqligi:**

- Windows: Visual C++ Redistributable kerakmi? (ko'pincha Python o'zi bilan olib keladi)
- macOS: codesign, notarization, Gatekeeper sinovi
- Linux: glibc versiyasi (ko'pincha eski distro'da build qiling → kengroq moslik), ldd bilan dependensiyalar

5. **Avtomatlashtirish** (ixtiyori): GitHub Actions yoki GitLab CI'da **matrix build:**

- runs-on: [windows-latest, macos-latest, ubuntu-latest]
- har birida pip install -r requirements.txt → build → artefakt sifatida yuklab qo'yish

6. **Qo'lda qurilma testlari:**

- FullHD/4K ekranlar, scaling (125%, 150%)

- Tarmoq cheklovlari (firewall off/on)
- Foydalanuvchi huquqlari (admin vs oddiy user), yozish ruxsatlari

Tezkor buyruqlar

Windows

```
# DLL tekshirish
where MyApp.exe
# Process tez diagnostika
tasklist | findstr MyApp
# Event Viewer (Application) loglarini tekshirish
```

macOS

```
# karantin bit'lari va imzo
spctl --assess --type execute dist/MyApp.app
codesign -dv --verbose=4 dist/MyApp.app
```

Linux

```
ldd dist/MyApp | grep "not found"      # yo'q kutubxonalar
ldd dist/MyApp | sort | uniq
```

4) Foydalanuvchi uchun qulay tarzda taqdim etish (installer/portable)

Windows

- **Portable ZIP:** --onefile exe yoki onedir papkani zip qilib tarqating.
- **MSI/EXE o'rnatkich:**
 - **cx_Freeze:** python setup.py bdist_msi
 - **Inno Setup** yoki **NSIS:** PyInstaller dist¥MyApp¥* dan installer yasang (yorliqlar, Uninstall, Start Menu).
- **Imzolash** (tavsiya): signtool.exe sign /tr http://timestamp ... MyApp.exe
Bu SmartScreen ogohlantirishlarini kamaytiradi.

macOS

- **.app bundle** (PyInstaller avtomatik yaratadi), tarqatish:
 - .dmg (Disk image) yoki .pkg
- **codesign** va **notarize** (Apple Developer ID talab qilinadi):
 - imzolash → notarizatsiya → stapling → Gatekeeper'da tekshirish
- Gatekeeper uchun foydalanuvchiga ko'rsatma (birinchi ishga tushirishda "Open Anyway" bo'lishi mumkin)

Linux

- **AppImage** (portable'ga o'xshaydi) yoki **.deb/.rpm** (distroga xos)
- Minimal talablar: glibc mosligi; kerakli .solar paketga kiritilgan bo'lsin yoki distro repolaridan o'rnatilishi ko'rsatilgan bo'lsin
- Sodaroq yo'l: tar.gz (install skript bilan)

Avto-yangilash (ixtiyoriy)

- Windows: **Squirrel.Windows**, **WinSparkle**
- macOS: **Sparkle**

- Ochiq internetga chiqadigan ilovalar uchun versiya tekshiruvchi kichik modul (JSON manba) yozib, "Update available" dialog ko'rsatish.

Foydalanuvchi tajribasi (UX) uchun tavsiyalar

- **Versiya va build raqami** ilova "About" oynasida ko'rinsin
- **Loglar:** ~/.myapp/logs yoki %LOCALAPPDATA%\MyApp\logs
- **Birlamchi sozlama fayllari:** foydalanuvchi papkasida saqlansin (admin huquqsiz)
- **Bir bosishda o'rnatish:** "Next → Next → Finish", default yo'llar
- **Uninstall** imkoniyati aniq ko'rinsin
- **Rasmiy qo'llanma** (PDF yoki onlayn) va "FAQ" linkini installerda ham ko'rsating

Qo'shimcha demo: PyInstaller bilan PyQt5 GUI'ni yig'ish

```
gui.py (oddiy oynacha):
import sys
from PyQt5 import QtWidgets

class Main(QtWidgets.QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My GUI")
        btn = QtWidgets.QPushButton("Hello")
        btn.clicked.connect(lambda:
QtWidgets.QMessageBox.information(self, "Hi", "Packaged!"))
        lay = QtWidgets.QVBoxLayout(self); lay.addWidget(btn)

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    w = Main(); w.show()
    sys.exit(app.exec_())
```

Yig'ish (Windows misoli):

```
pyinstaller --onefile --noconsole --name MyGUI gui.py
# PyQt pluginlari kerak bo'lsa:
# pyinstaller --onefile --noconsole --name MyGUI --add-data "venv\Lib\site-
packages\PyQt5\Qt\plugins\platforms;PyQt5\Qt\plugins\platforms" gui.py
```

Nazorat savollari:

1. Pyinstaller paketining vazifasi qanday?
2. Pyinstaller paketini o'rnatish qanday amalga oshiriladi?
3. Pyinstaller paketidan qanday foydalaniladi?
4. Python paketlarini kompyuterga qanday usullardan foydalanib, o'rnatish mumkin?
5. Kerakli paketlarni lokal holda o'rnatish qanday amalga oshiriladi?
6. Kompyuterga paketlarni lokal holada o'rnatishning qanday afzalliklari bor?
7. Auto-py-to-exe paketi kompyuterga qanday o'rnatiladi?
8. Auto-py-to-exe paketining qanday sozlamalari mavjud?